

GASZTROTÜKÖR



VIZSGAREMEK

BGÉSZC GANZ ÁBRAHÁM KÉT TANÍTÁSI NYELVŰ TECHNIKUM

KÉSZÍTETTE: CSORDÁS PATRIK,
SZŐKE DÁNIEL,
MÁRTON BENCE

SZAKMA MEGNEVEZÉSE:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ

TÉMAVEZETŐ: SZALAINÉ TÖRÖK EDIT,
MÉHES JÓZSEF

TARTALOMJEGYZÉK

1.	Bevezetés.....	6
1.1.	Projekt célja.....	6
1.2.	Téma kiválasztásának indoklása	6
1.3.	Célközönség	7
1.4.	Csoport tagjai és munkamegosztás	7
1.4.1.	Munkamegosztás	7
1.4.1.1.	Közös fejlesztések	7
1.4.1.2.	Fejlesztők egyéni hozzájárulásai	8
2.	Fejlesztői dokumentáció	9
2.1.	Alkalmazás áttekintés.....	9
2.2.	Felhasznált szoftverek.....	9
2.3.	Felhasznált programozási nyelvek	10
2.4.	Domain és tárhely	10
2.4.1.	Domain név konfiguráció	11
2.4.1.1.	Domain kiválasztása és vásárlása	11
2.4.1.2.	DNS-beállítások módosítása	11
2.4.2.	Azure konfiguráció	11
2.4.2.1.	Statikus webalkalmazás létrehozása (Frontend).....	11
2.4.2.2.	Webalkalmazás (API)	13
2.4.2.3.	SQL Adatbázis (GasztroTukorDB).....	13
2.5.	Zoho Mail.....	15
2.5.1.	Regisztráció és domain hozzáadása	15
2.6.	Github.....	16
2.7.	Adatbázis.....	16
2.7.1.	Adatbázis szerkezete.....	16
2.7.2.	Táblák részletes leírása	17
2.7.2.1.	Users tábla (Felhasználók)	17
2.7.2.2.	MenuCategories tábla (Menü kategóriák).....	17
2.7.2.3.	Restaurants tábla (Éttermek)	18
2.7.2.4.	MenuItems tábla (Menü elemek).....	19
2.7.2.5.	Reviews tábla (Vélemények).....	20
2.7.3.	Kapcsolatok	21
2.8.	Fő funkciók	22
2.9.	Projekt struktúra	22
2.9.1.	Fő könyvtárak	22

2.10. Adatmodellek	23
2.10.1. User (Felhasználó)	23
2.10.2. Restaurant (Étterem)	23
2.10.3. MenuItem (Menüelem)	23
2.10.4. MenuCategory (Menükategória)	24
2.10.5. Review (Értékelés)	24
2.10.6. ApplicationDbContext	24
2.10.6.1. Környezetspecifikus konfiguráció	25
2.11. Controllers (Vezérlők)	26
2.11.1. RestaurantController	26
2.11.2. UserController	28
2.11.3. ReviewController	30
2.11.4. MenuItemController	32
2.11.5. MenuCategoryController	34
2.11.6. EmailController	35
2.12. Services (Szolgáltatások)	38
2.12.1. RestaurantService	38
2.12.2. MenuItemService	39
2.12.3. MenuCategoryService	40
2.12.4. ReviewService	40
2.12.5. UserService	41
2.12.6. EmailService	42
2.12.7. AuthServiceProviderService	42
2.12.7.1. Hitelesítési állapot kezelése	42
2.12.7.2. Felhasználói hitelesítés lekérdezése	43
2.12.7.3. Felhasználói bejelentkezés értesítése	43
2.12.7.4. Felhasználói kijelentkezés értesítése	44
2.12.7.5. UserSession osztály	44
2.12.7.6. Használata az alkalmazásban	44
2.12.7.7. Munkamenet kezelés	44
2.12.7.8. Biztonság	45
2.13. web/Program.cs	45
2.13.1. Névterek és függőségek	45
2.13.2. WebAssembly host konfigurálása	45
2.13.3. HTTP kliens regisztrálása	46
2.13.4. Adatbázis és szolgáltatások regisztrálása	46
2.14. API/Program.cs	46

2.14.1.	WebApplication Builder létrehozása	46
2.14.2.	Szolgáltatások regisztrálása	46
2.14.3.	CORS (Cross-Origin Resource Sharing) beállítása	47
2.14.4.	Alkalmazás kiépítése és konfigurálása	47
2.14.5.	Middleware konfigurálása	47
2.15.	NavMenu.razor.....	48
2.15.1.	HTML struktúra	48
2.15.2.	JavaScript integráció.....	50
2.15.3.	C# Metódusok.....	51
2.16.	MainLayout.razor.....	57
2.16.1.	HTML struktúra	57
2.16.2.	JavaScript integráció.....	58
2.16.3.	C# Metódusok.....	59
2.17.	Home.razor.....	62
2.17.1.	HTML struktúra	62
2.17.2.	JavaScript integráció.....	63
2.17.3.	C# Metódusok.....	65
2.18.	Site.razor	66
2.18.1.	HTML Struktúra	66
2.18.2.	JavaScript integráció.....	68
2.18.3.	C# Metódusok.....	69
2.19.	Owner.razor.....	72
2.19.1.	HTML struktúra	72
2.19.2.	JavaScript integráció.....	74
2.19.3.	C# Metódusok.....	74
2.20.	Administrator.razor	78
2.20.1.	HTML struktúra	78
2.20.2.	JavaScript integráció.....	80
2.20.3.	C# Metódusok.....	81
2.21.	Verification.razor.....	83
2.21.1.	HTML struktúra	83
2.21.2.	JavaScript integráció.....	85
2.21.3.	C# Metódusok.....	85
2.22.	Routes.json.....	87
2.22.1.	Struktúra és beállítások.....	87
2.22.1.1.	Routes (Útvonalak).....	87
2.22.1.2.	NavigationFallback (Navigációs tartalék).....	87

2.22.1.3.	ResponseOverrides (Válasz felülbírálosok)	88
2.22.2.	Az Azure Static Web Apps miatt szükséges	88
2.23.	Képernyőtervek	89
2.24.	Bejelentkezési adatok	95
2.25.	Fejlesztési lehetőségek	96
2.26.	Tesztelési terv	97
2.26.1.	Tesztelési célok	97
2.26.2.	Tesztelési típusok	97
2.26.2.1.	Funkcionális tesztek	97
2.26.2.2.	Egységtesztek	97
2.26.2.3.	Integrációs tesztek	97
2.26.3.	Tesztelési környezet	98
2.26.4.	Tesztesetek	98
2.26.4.1.	Regisztráció/Bejelentkezés	98
2.26.4.2.	Étterem keresés / listázás	99
2.26.4.3.	Értékelések	99
2.26.4.4.	Profil beállítások / Értékelések módosítása	100
2.26.4.5.	Étterem kezelése	101
3.	Felhasználói dokumentáció	103
3.1.	A program céljának és lényegesebb funkcióinak összefoglalása	103
3.2.	Szükséges hardvereszközök és szoftverek felsorolása	103
3.3.	Telepítés és indítás lépéseinek ismertetése	104
3.4.	A program részletes bemutatása	104
3.4.1.	Vendég funkciók:	104
3.4.2.	Keresőmező működése	104
3.4.2.1.	Regisztráció új vendégként	105
3.4.2.2.	Sikeres regisztráció	105
3.4.2.3.	Bejelentkezés	105
3.4.2.4.	Elfelejtett jelszó	106
3.4.2.5.	Jelszó megváltoztatása	107
3.4.2.6.	Fiók törlése	107
3.4.2.7.	Értékelés küldésének lépései	107
3.4.3.	Étterem tulajdonos funkciók	109
3.4.3.1.	Étterem Regisztráció	109
3.4.3.2.	Étterem kezelése	111
3.4.3.3.	Étterem adatainak módosítása	111
3.4.3.4.	Étterem tulajdonos beállítások	112

3.4.3.5.	Étterem tulajdonos Fiók törlése.....	113
3.4.3.6.	Étterem feltöltése adatokkal	113
3.4.3.7.	Étel feltöltésének lépései	114
3.4.3.8.	Feltöltött étel szerkesztése	115
3.4.3.9.	Étterem tulajdonos vélemények szekció	115
3.4.3.10.	Étterem vélemények mentése	116
3.4.4.	Adminisztrátor funkciók	117
3.4.4.1.	Adminisztrátor bejelentkezés	117
3.4.4.2.	Adminisztrátor profil beállítások.....	117
3.4.4.3.	Adminisztrátor felhasználók kezelése	118
3.4.4.4.	Adminisztrátor felhasználók törlése	118
3.4.4.5.	Adminisztrátori felület kezelése	119
3.4.4.6.	Adminisztrátori felület Étterem adatainak kezelése	119
3.4.4.7.	Étterem szerkesztése és annak lépései.....	120
3.4.4.8.	Adminisztrátori felület Étterem menüjének kezelése.....	121
3.4.4.9.	A szerkesztés lépéseinek folyamata.....	121
3.4.4.10.	Adminisztrátori felület vendégvélemény kezelése.....	122
3.4.5.	Az oldalt segítő tájékoztatói opciók	123
3.4.5.1.	GasztroTükör gomb és jelentősége	123
3.4.5.2.	Munkamenet figyelő.....	123
3.4.5.3.	Munkamenet lejárt és annak következménye.....	124
3.4.6.	Kapcsolattartás.....	124
4.	Összefoglalás.....	125
4.1.	Köszönetnyilvánítás	125
4.2.	Elkészült projektünk értékelése.....	125
5.	Irodalomjegyzék.....	126

1. BEVEZETÉS

1.1. PROJEKT CÉLJA

A projekt elsődleges célja egy olyan átfogó rendszer megalkotása, amely a helyi, kezdő éttermek számára biztosít lehetőséget láthatóságuk növelésére és hatékonyabb kapcsolódásra a potenciális vendégekkel. A rendszer lehetőséget biztosít a felhasználók számára helyi éttermek keresésére és értékelések olvasására böngészőn keresztül, míg az éttermek tulajdonosai kezelhetik a menüiket és elemezhetik a vendégértékeléseket, ezzel megkönnyítve működésüket és javítva a vendégek elégedettségét.

Az általunk fejlesztett program különlegessége, hogy kifejezetten a magyar piacra fókuszál, és lehetőséget biztosít a beregisztrált éttermek értékelésére különböző kategóriák szerint. Ez részletesebb és átfogóbb visszajelzéseket eredményez, ami elősegíti az éttermek fejlődését. Programunk célja az is, hogy áthidalja a piackutatás során feltárt hiányosságokat: például a diningcity.hu oldalon nehéz rátalálni az értékelési lehetőségre, az etterem.hu pedig nem biztosít kategóriák szerinti értékelést, csak szöveges véleményt. Platformunk kiemelt figyelmet fordít az átláthatóságra és a letisztult dizájnra, így felhasználóbarátabb és hatékonyabb élményt kínál.

Emellett programunk könnyen kezelhető adminisztrációs eszközöket biztosít, amelyek megkönnyítik a menükezelést és a vendégértékelések nyomon követését. Ezek az eszközök nemcsak a mindennapi működés hatékonyságát növelik, hanem hozzájárulnak az éttermek hosszú távú sikeréhez is.

1.2. TÉMA KIVÁLASZTÁSÁNAK INDOKLÁSA

Minden egy esős péntek estén kezdődött, amikor a barátainkkal egy új éttermet próbáltunk találni a belvárosban. Éhesek voltunk, de nem akartunk a szokásos helyekre menni, valami újat szerettünk volna kipróbálni. Google-ztunk, de semmi igazán meggyőzőt nem találtunk. Aztán megláttunk egy kis, hangulatos bistro-t egy mellékutcában, amelyről még soha nem hallottunk.

Beléptünk, és azonnal megcsapott a friss fűszerek és az ínycsiklandó étel illata. Az étel mennyei volt, a séf pedig maga szolgálta fel, lelkesen mesélve az ízek mögötti történetekről. Mikor rákérdeztünk, miért nem ismertebb a hely, csak sóhajtott:

"Nehéz versenyezni a nagyobb éttermek marketingjével. A minőség itt van, de kevés az eszköz, hogy elérjük az embereket."

Ez a mondat megmaradt bennünk. Miért nincs egy olyan rendszer, amely a kisebb, helyi éttermeket segíti, hogy könnyebben kapcsolatba lépjenek az emberekkel? Egy platform, ahol a vendégek nem csak éttermet kereshetnek, hanem valódi, megbízható véleményeket olvashatnak, az éttermek pedig láthatóbbá válhatnak?

Ekkor jött az ötlet.

A következő hetekben elkezdtünk ötletelni, tervezni, és végül megszületett a projekt: egy átfogó rendszer, amely összeköti a helyi éttermeket és a vendégeket.

1.3. CÉLKÖZÖNSÉG

A célközönség elsődlegesen azok a helyi, kezdő éttermek, amelyek még nem rendelkeznek ismert márkanevvel vagy jelentős vendégkörrel. Ezek az éttermek különösen nehéz helyzetben vannak a telített piacon, és nagy szükségük van arra, hogy növeljék láthatóságukat és hatékonyabban kapcsolódjanak a potenciális vendégeikkel.

Másodsorban a rendszer célközönségébe tartoznak azok a felhasználók is, akik helyi éttermeket keresnek, szeretnék értékeléseket olvasni, valamint új, érdekes helyeket felfedezni a saját környezetükben.

1.4. CSOPORT TAGJAI ÉS MUNKAMEGOSZTÁS

A projektet mi, Csordás Patrik, Szőke Dániel és Márton Bence készítettük.

1.4.1. MUNKAMEGOSZTÁS

1.4.1.1. KÖZÖS FEJLESZTÉSEK

- **ApplicationDbContext.cs:** Központi adatbázis kontextus
- **MenuCategory.cs:** Menü kategóriák modellje
- **MenuItem.cs:** Menüelemek modellje
- **Restaurant.cs:** Éttermek modellje
- **Review.cs:** Értékelések modellje
- **User.cs:** Felhasználók modellje
- **MenuCategoryController.cs:** Menü kategóriák kezelése
- **MenuItemController.cs:** Menüelemek kezelése
- **RestaurantController.cs:** Éttermek kezelése
- **ReviewController.cs:** Értékelések kezelése
- **UserController.cs:** Felhasználók kezelése

- **Frontend elrendezés:** Az oldal egységes megjelenése és szerkezete
- **MainLayout.razor** és **NavMenu.razor:** Fő elrendezés és navigációs komponensek

1.4.1.2. FEJLESZTŐK EGYÉNI HOZZÁJÁRULÁSAI

Csordás Patrik

- **EmailController.cs** és **EmailService.cs:** Email funkcionalitás
- **Home.razor** és **Site.razor:** Nyilvános főoldal és étterem oldal
- **AuthStateProviderService.cs:** Hitelesítési állapot kezelése

Márton Bence

- **Administrator.razor:** Adminisztrátori felület implementálása
- **Verification.razor:** Éttermek és menüelemek ellenőrzési és jóváhagyási folyamatainak fejlesztése

Szóke Dániel

- **Owner.razor:** Étteremtulajdonosi kezelőfelület fejlesztése
- **Notification.razor:** Felhasználói értesítési rendszer implementálása

A projekt struktúrája olyan hatékony csapatmunkára épült, amely lehetővé tette számunkra, hogy mindenki a saját részére összpontosítson, miközben közös architektúrát és adatbázismodelleket használtunk.

2. FEJLESZTŐI DOKUMENTÁCIÓ

2.1. ALKALMAZÁS ÁTTEKINTÉS

A GasztroTükör egy étterem értékelő és menedzsment weboldal, amely Blazor WebAssembly technológiával készült. Az alkalmazás .NET 8 keretrendszert használ, és három fő felhasználói szerepkört támogat: vendég (customer), étterem tulajdonos (owner) és adminisztrátor (admin).

2.2. FELHASZNÁLT SZOFTVEREK

A fejlesztési folyamat során az alábbi szoftvereket használtuk különböző feladatokra:

- **Visual Studio Code**

A Visual Studio Code egy könnyű, de erőteljes forráskódszerkesztő, amelyet elsősorban a HTML alapok szerkesztésére és tesztelésére használtunk. Kiemelkedő funkciói közé tartozik a szintaxiskiemelés, a kódkiegészítés, valamint a beépített terminál, amely megkönnyíti a fejlesztési munkát.

- **Visual Studio 2022**

A Blazor WebAssembly Standalone App és az ASP.NET Core Web API fejlesztése C# nyelven történt, Visual Studio 2022-ben. Ez a fejlesztői környezet számos beépített eszközt biztosít a hatékony backend- és frontend-fejlesztéshez, például kódlépésenkénti hibakeresést (debugging), NuGet csomagkezelést, valamint a .NET projektek könnyű kezelhetőségét.

- **SQL Server Management Studio 20**

Az adatbázisok létrehozásához, kezeléséhez és karbantartásához az SQL Server Management Studio (SSMS) 20 verzióját használtuk Microsoft SQL Server (MSSQL) adatbázis-kezelővel. Az MSSQL választása azért történt, mert erős integrációt biztosít a .NET alapú alkalmazásokkal, kiváló teljesítményt és skálázhatóságot nyújt nagyobb adatmennyiségek kezelésekor tranzakciókezelési funkciókkal rendelkezik. Ezzel a szoftverrel hatékonyan kezelhetők az SQL szerverek, lekérdezések futtathatók, adatbázisok módosíthatók, valamint optimalizálhatók az adatok tárolására és lekérdezésére szolgáló folyamatok.

2.3. FELHASZNÁLT PROGRAMOZÁSI NYELVEK

A fejlesztés során az alábbi programozási nyelveket alkalmaztuk:

- **HTML (HyperText Markup Language)** – A weboldal szerkezetének és tartalmának meghatározására szolgáló jelölőnyelv.
- **CSS (Cascading Style Sheets)** – A weboldal vizuális megjelenésének kialakításához használt stíusleíró nyelv.
- **JavaScript** – A weboldal interaktív elemeinek és dinamikus funkcióinak megvalósítására szolgáló programozási nyelv.
- **C#** – A Blazor WebAssembly alkalmazás és az ASP.NET Core Web API fejlesztéséhez használt, objektumorientált programozási nyelv, amely hatékony és biztonságos backend megoldásokat biztosít.
- **MSSQL (Microsoft SQL Server Query Language)** – Az adatbázisok kezeléséhez, lekérdezések futtatásához és adatok manipulálásához használt SQL dialektus a Microsoft SQL Server környezetben.

2.4. DOMAIN ÉS TÁRHELY

A GasztroTükör weboldal tárhelye és fő szolgáltatásai a Microsoft Azure felhőalapú környezetében kerültek kialakításra. A rendszer három fő Azure-szolgáltatásra épül:

- **GasztroTukorDB:** SQL-alapú relációs adatbázis az alkalmazás adatainak tárolására.
- **gasztrotukor-api:** Webalkalmazás, amely a háttérrendszer API-végpontjait szolgáltatja.
- **Statikus webalkalmazás:** A weboldal statikus tartalmi (HTML, CSS, JavaScript) egy Azure Static Web Apps szolgáltatáson keresztül kerülnek kiszolgálásra. Az alkalmazás eredeti Azure-címe:

<https://salmon-tree-07d97a703.6.azurestaticapps.net>

A weboldal publikus, felhasználók számára elérhető domain-je:

www.gasztrotukor.hu

Ezt a doménnevet a Rackhost.hu szolgáltatón keresztül szereztük be. A domain név átirányítása az Azure-ban hosztolt statikus webalkalmazásra megfelelően be lett állítva, így a felhasználók a Rackhost által kezelt doménnéven keresztül érik el az Azure-infrastruktúrán működő weboldalt.

2.4.1. DOMAIN NÉV KONFIGURÁCIÓ

2.4.1.1. DOMAIN KIVÁLASZTÁSA ÉS VÁSÁRLÁSA

- A kiválasztott domain: gasztrotukor.hu



- A vásárlás után a Rackhost felületén elérhetővé vált a domain DNS-kezelése.

2.4.1.2. DNS-BEÁLLÍTÁSOK MÓDOSÍTÁSA

A domain regisztrációját követően a Rackhost adminfelületén a DNS-zónát módosítottuk annak érdekében, hogy a domain az Azure Static Web App szolgáltatáshoz csatlakozzon.

CNAME rekord hozzáadása:

- Hozsnév:** www
- Érték:** salmon-tree-07d97a703.6.azurestaticapps.net
- TTL:** Alapértelmezett (3600 másodperc)

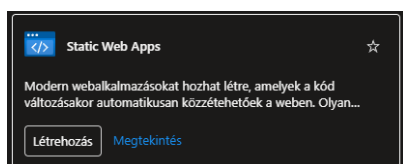
A, CNAME, TXT rekordok				Új rekord
HOSZTNÉV	TÍPUS	ÉRTÉK	TTL	
www.gasztrotukor.hu	CNAME	salmon-tree-07d97a703.6.azurestaticapps.net	3600	

Ez a beállítás lehetővé teszi, hogy a felhasználók a www.gasztrotukor.hu címen keresztül érjék el az Azure-ban tárolt statikus weboldalt.

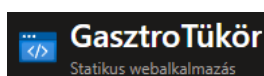
2.4.2. AZURE KONFIGURÁCIÓ

2.4.2.1. STATIKUS WEBALKALMAZÁS LÉTREHOZÁSA (FRONTEND)

- Szolgáltatás:** Azure Static Web Apps



- Projekt neve:** GasztroTükör



- Automatikus build:** GitHub Actions integráción keresztül történik

master_gasztrotukor-api.yml

- **Webhely címe (alapértelmezett):**
https://salmon-tree-07d97a703.6.azurestaticapps.net

Egyéni tartomány hozzáadása:

Más DNS-beli egyéni tartomány
Egyéni tartomány hozzáadása külső DNS-szolgáltató használatával

- A Static Web App beállításai között hozzáadtuk a www.gasztrotukor.hu címet.

Egyéni tartomány hozzáadása

1 Tartomány megadása 2 **Ellenőrzés és hozzáadás**

Tartománynév: www.gasztrotukor.hu

Tartomány tulajdonjogának érvényesítése

Állomásnév rekordtípusa: CNAME

Másolja a CNAME állomásnévrekordot, és használja a tartomány tulajdonjogának megerősítéséhez a DNS-szolgáltatójánál. A DNS-bejegyzés módosításainak érvénybe léptetése akár 48 órát is igénybe vehet.

Típus	Gazda...	Érték	Állapot
CNAME	www	salmon-tree-07d97a703.6.azurestaticapps.net	

- Az Azure automatikusan ellenőrzi a DNS CNAME rekord meglétét, majd engedélyezi az egyéni domain használatát.

Típus	Gazda...	Érték	Állapot
CNAME	www	salmon-tree-07d97a703.6.azurestaticapps.net	Ellenőrizve

- **HTTPS aktiválása:** Azure automatikusan telepíti az SSL tanúsítványt, így a domain biztonságosan, HTTPS-en keresztül érhető el.

Tanúsítványmegtekintő: www.gasztrotukor.hu

Általános Részletek

Tulajdonos

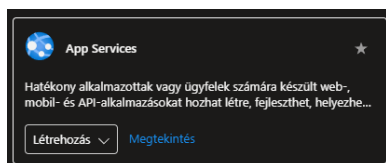
Név (CN)	www.gasztrotukor.hu
Szervezet (O)	<Nem része a tanúsítványnak>
Szervezeti egység (OU)	<Nem része a tanúsítványnak>

Kiállította:

Név (CN)	GeoTrust Global TLS RSA4096 SHA256 2022 CA1
Szervezet (O)	DigiCert, Inc.
Szervezeti egység (OU)	<Nem része a tanúsítványnak>

2.4.2.2. WEBALKALMAZÁS (API)

- **Szolgáltatás:** App Services



- **Alkalmazás neve:** gasztrotukor-api

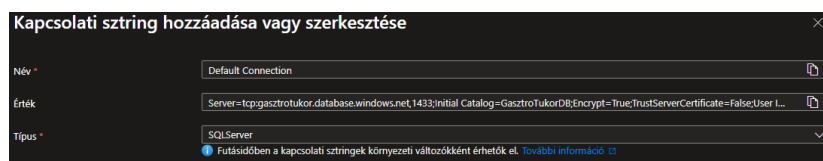


- **Használat célja:** REST API endpointok szolgáltatása a frontend számára
- **Deployment:** CI/CD pipeline segítségével GitHub Actions integráción keresztül történik

`azure-static-web-apps-salmon-tree-07d97a703.yml`

- **Konfiguráció:**

- Környezeti változók: kapcsolati sztring az adatbázishoz.

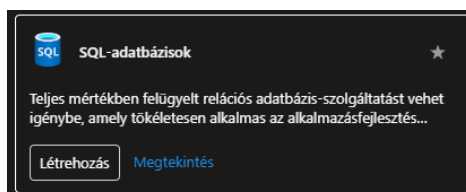


- Termékváltozat és méret: Free (F1)

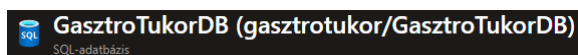
Termékváltozat és méret **Free (F1)**

2.4.2.3. SQL ADATBÁZIS (GASZTROTUKORDB)

- **Szolgáltatás:** Azure SQL Database



- **Adatbázis neve:** GasztroTukorDB



- **Kapcsolódás:**

- A gasztrotukor-api alkalmazás a backendkódon keresztül SQL kapcsolati sztringgel kapcsolódik az adatbázishoz.
- A kapcsolat engedélyezése az Azure felületén történik, a SQL-kiszolgáló tűzfalbeállításain keresztül.

- **Tűzfalbeállítás – fontos konfiguráció:**

- "Kiszolgáló elérésének engedélyezése az Azure-szolgáltatások és -erőforrások számára" opció engedélyezve lett.

☒ A kiszolgáló elérésének engedélyezése az Azure-szolgáltatások és -erőforrások számára

- Ezzel a beállítással a SQL Server tűzfala lehetővé teszi a kapcsolatfelvételt bármely Azure-szolgáltatásból vagy erőforrásból.

- **Biztonság:**

- A kapcsolat jelszóval védett SQL-felhasználói fiókkal történik.

- A tűzfal más publikus IP-k számára továbbra is zárt marad, kivéve az engedélyezett adminisztrációs IP-tartományokat.

Szabály neve	Kezdő IPv4-cím	Záró IPv4-cím
CsoPa04		
Danika		
Jimbo		

- **Termékváltozat és méret**

Tarifacsomag : Alapszintű

2.5. Zoho MAIL

Hivatalos e-mail kommunikáció lebonyolítása a `gasztrotukor.hu` domain alatt (pl. kapcsolatfelvétel, email visszajelzések)

2.5.1. REGISZTRÁCIÓ ÉS DOMAIN HOZZÁADÁSA

- **Szolgáltatás:** Zoho Mail (Forever Free Plan)



Forever Free Plan

Up to five users, 5GB/User, 25MB attachment limit.

Web access and free mobile apps*.

Email hosting for single domain.

SIGN UP NOW

- **Fiók neve:** info@gasztrotukor.hu

Betöltés... info@gasztrotukor.hu



- **Deployment:**
 - A domain-hez szükséges DNS rekordok manuálisan kerültek konfigurálásra a domain szolgáltató felületén.
 - Az SMTP hitelesítési adatok titkosítva kerülnek be a CI/CD pipeline-ba, így az API-k képesek automatikusan e-mailt küldeni.
- **Konfiguráció:**
 - **MX rekordok hozzáadása:**

MX rekordok				Új MX rekord	
HOSZTNÉV	LEVELEZŐ SZERVER	PRIORITÁS	TTL		
gasztrotukor.hu	mx.zoho.eu	10	3600		
gasztrotukor.hu	mx2.zoho.eu	20	3600		
gasztrotukor.hu	mx3.zoho.eu	50	3600		

- **TXT rekord hozzáadása:**

A, CNAME, TXT rekordok				Új rekord	
HOSZTNÉV	TÍPUS	ÉRTÉK	TTL		
gasztrotukor.hu	TXT	v=spf1 include:zoho.eu include:_spf.google.com ~all	3600		

2.6. GITHUB

A projekt gördülékeny lebonyolítása érdekében létrehoztunk egy közös fejlesztési környezetet GitHub-on. Minden résztvevő a saját **@ganziskola.hu** végződésű emailcímével regisztrált, így biztosítva az egységes hozzáférést és az egyszerű együttműködést.

A fejlesztéshez létrejött egy külön repository **vizsgaremek** néven, amely az alábbi címen érhető el: <https://github.com/CsoPa04/vizsgaremek>.

Ebben a tárhelyben minden résztvevő rögzítésre került, így a kódváltozások nyomon követhetők, a közös munkafolyamat pedig átláthatóvá vált. A projekt során létrejött fájlokat – legyen szó forráskódról, adatbázisról vagy dokumentációról – mindenki egy helyen elérheti.

A fejlesztők saját gépükön dolgoztak, és rendszeresen feltöltötték munkáikat a közös tárhelyre, így biztosítva a folyamatos verziókövetést és a csapatszintű integrációt.

2.7. ADATBÁZIS

A **GasztroTükörDB** egy olyan adatbázis, amelyet egy online étterem- és ételminősítő platformhoz terveztek. Az adatbázis tárolja az étteremeket, azok menüit, felhasználókat és az értékeléseket.

2.7.1. ADATBÁZIS SZERKEZETE

A **GasztroTükörDB** egy relációs adatbázis, amely az alábbi főbb táblákból áll:

- **Users** (Felhasználók): A rendszerbe regisztrált felhasználókat tárolja.
- **MenuCategories** (Menü kategóriák): Az étlap kategóriák szerinti felosztását kezeli.
- **Restaurants** (Éttermek): Az éttermek adatait tartalmazza.
- **MenuItems** (Menü elemek): Az étlap konkrét tételeit tárolja.
- **Reviews** (Vélemények): A felhasználók által adott értékeléseket és véleményeket tartalmazza.

2.7.2. TÁBLÁK RÉSZLETES LEÍRÁSA

2.7.2.1. USERS TÁBLA (FELHASZNÁLÓK)

Ez a tábla a felhasználói fiókokat tárolja.

Oszlop	Típus	Leírás
user_id	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
email	NVARCHAR(255)	Felhasználó e-mail címe
password	VARBINARY(255)	Titkosított jelszó (SHA256)
role	NVARCHAR(255)	Felhasználó szerepköre (admin, tulajdonos, vendég)
is_deleted	BIT	Törölt-e az elem
created_at	DATETIME	Létrehozás dátuma

Users	
	user_id
	email
	password
	role
	is_deleted
	created_at

2.7.2.2. MENU CATEGORIES TÁBLA (MENÜ KATEGÓRIÁK)

Ez a tábla az étlap elemeinek kategóriáit tartalmazza.

Oszlop	Típus	Leírás
category_id	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
category_name	NVARCHAR(255)	A kategória neve

MenuCategories	
	category_id
	category_name

2.7.2.3. RESTAURANTS TÁBLA (ÉTTERMEK)

Ez a tábla az étteremek adatait tárolja.

Oszlop	Típus	Leírás
restaurant_id	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
owner_id	INT	Hivatkozás az Users táblára
name	NVARCHAR(255)	Étterem neve
cuisine	NVARCHAR(255)	Konyha típusa
phone	VARCHAR(255)	Telefonszám
address	NVARCHAR(255)	Cím
opening_time	TIME	Nyitási idő
closing_time	TIME	Zárási idő
thumbnail	NVARCHAR(MAX)	A kép Base64 kódolt formátumban
description	NVARCHAR(255)	Étterem rövid leírása
is_approved	BIT	Admin jóváhagyta-e
is_deleted	BIT	Törölt-e az elem
created_at	DATETIME	Létrehozás dátuma

Restaurants	
	restaurant_id
	owner_id
	name
	cuisine
	phone
	address
	opening_time
	closing_time
	thumbnail
	description
	is_approved
	is_deleted
	created_at

2.7.2.4. MENUITEMS TÁBLA (MENÜ ELEMÉK)

Ez a tábla az étlapon szereplő ételeket tárolja.

Oszlop	Típus	Leírás
item_id	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
restaurant_id	INT	Hivatkozás az Restaurants táblára
category_id	INT	Hivatkozás az MenuCategories táblára
name	NVARCHAR(255)	Az étel neve
description	NVARCHAR(255)	Az étel rövid leírása
price	INT	Az étel ára
thumbnail	NVARCHAR(MAX)	
is_available	BIT	Elérhető-e az étel
is_approved	BIT	Admin jóváhagyta-e
is_deleted	BIT	Törölt-e az elem
created_at	DATETIME	Létrehozás dátuma

Menuitems	
	item_id
	restaurant_id
	category_id
	name
	description
	price
	thumbnail
	is_available
	is_approved
	is_deleted
	created_at

2.7.2.5. REVIEWS TÁBLA (VÉLEMÉNYEK)

Ez a tábla a felhasználói értékeléseket tárolja.

Oszlop	Típus	Leírás
review_id	INT (IDENTITY)	Egyedi azonosító, elsődleges kulcs
user_id	INT	Hivatkozás az Users táblára
restaurant_id	INT	Hivatkozás az Restaurants táblára
food_rating	INT	Étel minőségének pontszáma (1-5)
service_rating	INT	Szolgáltatás pontszáma (1-5)
cleanliness_rating	INT	Tisztaság pontszáma (1-5)
atmosphere_rating	INT	Hangulat pontszáma (1-5)
avg_rating	FLOAT	Az átlagos értékelés (automatikusan számolva)
review_text	NVARCHAR(255)	Szöveges vélemény
is_deleted	BIT	Törölt-e az elem
created_at	DATETIME	Létrehozás dátuma

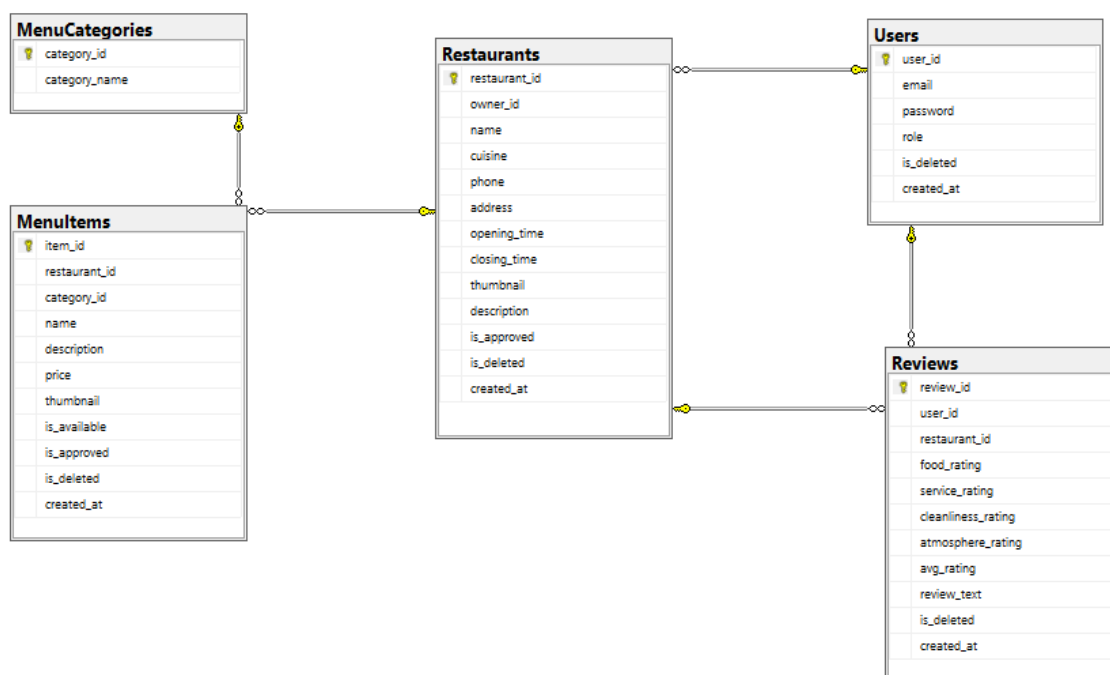
Reviews	
	review_id
	user_id
	restaurant_id
	food_rating
	service_rating
	cleanliness_rating
	atmosphere_rating
	avg_rating
	review_text
	is_deleted
	created_at

2.7.3. KAPCSOLATOK

Az egyes táblák az IDEGEN KULCSOK által kapcsolódnak egymáshoz, biztosítva az adatok konzisztenciáját.

Az egyes táblák közötti kapcsolatok:

- **1:1 kapcsolat:**
 - **Users ↔ Restaurants:** Egy tulajdonos felhasználóhoz csak egy étterem tartozhat
- **1:N kapcsolat:**
 - **Users ↔ Reviews:** Egy felhasználó írhat több értékelést különböző éttermekre, de egy értékelés pontosan egy felhasználóhoz tartozik.
 - **Restaurants ↔ Reviews:** Egy étterem kaphat több értékelést különböző felhasználóktól, de egy értékelés pontosan egy étteremre vonatkozik.
 - **Restaurants ↔ MenuItems:** Egy étteremnek lehet több menüeleme, de egy menüelem pontosan egy étteremhez tartozik
 - **MenuCategories ↔ MenuItems:** Egy menükategóriának lehet több menüeleme, de egy menüelem pontosan egy kategóriába tartozik.



2.8. FŐ FUNKCIÓK

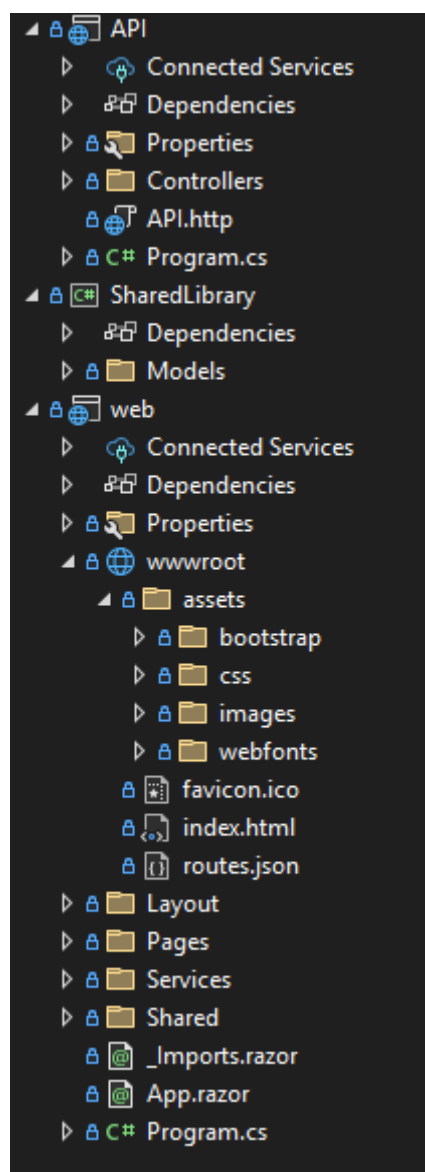
- **Vendégek számára:** Éttermek böngészése, értékelések megtekintése, éttermek értékelése
- **Tulajdonosok számára:** Éttermek adatainak kezelése, menü szerkesztése, értékelések megtekintése, adatok exportálása
- **Adminisztrátorok számára:** Éttermek jóváhagyása, felhasználók kezelése, rendszerfelügyelet

2.9. PROJEKT STRUKTÚRA

A megoldás egy Blazor WebAssembly alkalmazásból és egy közös könyvtárból (SharedLibrary) áll, amely az adatmodelleket tartalmazza.

2.9.1. FŐ KÖNYVTÁRAK

- **API:**
 - **Controllers:** Az alkalmazás backendjének vezérlői
- **Web:** Blazor WebAssembly alkalmazás
 - **Pages:** Razor komponensek (oldalak)
 - **Services:** Szolgáltatások implementációja
 - **Shared:** Újrafelhasználható komponensek
 - **Layout:** Elrendezés sablonok
- **SharedLibrary:** Közös modellek és interfészek
 - **Models:** Adatmodellek



2.10. ADATMODELLEK

A rendszer a következő fő entitásokat használja:

2.10.1. USER (FELHASZNÁLÓ)

```
public partial class User {  
    [Key]  
    public int user_id { get; set; }  
    public string email { get; set; }  
    public byte[] password { get; set; } // Hashelt jelszó  
    public string role { get; set; }     // "customer", "owner", "admin"  
    public bool is_deleted { get; set; }  
    public DateTime created_at { get; set; }  
}
```

2.10.2. RESTAURANT (ÉTTEREM)

```
public partial class Restaurant {  
    [Key]  
    public int restaurant_id { get; set; }  
    public int owner_id { get; set; }  
    public string name { get; set; }  
    public string cuisine { get; set; }  
    public string phone { get; set; }  
    public string address { get; set; }  
    public TimeOnly opening_time { get; set; }  
    public TimeOnly closing_time { get; set; }  
    public string thumbnail { get; set; } // Base64 formátumú kép  
    public string description { get; set; }  
    public bool is_approved { get; set; } // Admin jóváhagyás státusza  
    public bool is_deleted { get; set; }  
    public DateTime created_at { get; set; }  
}
```

2.10.3. MENUITEM (MENÜELEM)

```
public partial class MenuItem {  
    [Key]  
    public int item_id { get; set; }  
    public int restaurant_id { get; set; }  
    public int category_id { get; set; }  
    public string name { get; set; }  
    public string description { get; set; }  
    public int price { get; set; }  
    public string thumbnail { get; set; } // Base64 formátumú kép  
    public bool is_available { get; set; }  
    public bool is_approved { get; set; }  
    public bool is_deleted { get; set; }  
    public DateTime created_at { get; set; }  
}
```

2.10.4. MENUCATEGORY (MENÜKATEGÓRIA)

```
public partial class MenuCategory {
    [Key]
    public int category_id { get; set; }
    public string category_name { get; set; }
}
```

2.10.5. REVIEW (ÉRTÉKELÉS)

```
public partial class Review {
    [Key]
    public int review_id { get; set; }
    public int user_id { get; set; }
    public int restaurant_id { get; set; }
    public int food_rating { get; set; } // 1-5 csillag értékelés
    public int service_rating { get; set; } // 1-5 csillag értékelés
    public int cleanliness_rating { get; set; } // 1-5 csillag értékelés
    public int atmosphere_rating { get; set; } // 1-5 csillag értékelés
    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public decimal avg_rating { get; set; } // A 4 értékelés átlaga
    public string review_text { get; set; }
    public bool is_deleted { get; set; }
    public DateTime created_at { get; set; }
}
```

2.10.6. APPLICATIONDBCONTEXT

Az ApplicationDbContext osztály az adatbázis-kapcsolatot és adatmodelleket kezeli. Ez az osztály az Entity Framework Core **DbContext** osztály kiterjesztése, amely a szolgáltatások és a vezérlők számára biztosítja az adatbázis-hozzáférést.

```
namespace SharedLibrary.Models
{
    public partial class ApplicationDbContext : DbContext
    {
        public virtual DbSet<MenuCategory> MenuCategories { get; set; }
        public virtual DbSet<MenuItem> MenuItems { get; set; }
        public virtual DbSet<Restaurant> Restaurants { get; set; }
        public virtual DbSet<Review> Reviews { get; set; }
        public virtual DbSet<User> Users { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            var isLocalhost = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT") == "Development";
            if (isLocalhost)
            {
                optionsBuilder.UseSqlServer("Server=localhost\\SQLEXPRESS;Database=GasztróTukorDB;User Id=sa;Password=1A2w3e4F;TrustServerCertificate=True");
            }
            else
            {
                optionsBuilder.UseSqlServer("Server=tcp:gasztrotukor.database.windows.net,1433;Initial Catalog=GasztróTukorDB;Encrypt=True;TrustServerCertificate=False;User Id=gasztrotukor;Password=1A2w3e4F;");
            }
        }
    }
}
```

2.10.6.1.KÖRNYEZETSPECIFIKUS KONFIGURÁCIÓ

Az adatbáziskapcsolat automatikusan a megfelelő konfigurációt választja:

Fejlesztői környezet

- Helyi SQL Express szerveret használ
- Adatbázis: GasztroTukorDB
- Kapcsolati sztring:
Server=localhost\SQLEXPRESS;
Database=GasztroTukorDB;
User Id=sa;
Password=1A2w3e4F;
TrustServerCertificate=True

Éles környezet

- Azure SQL adatbázist használ
- Adatbázis: GasztroTukorDB
- Kapcsolati sztring:
Server=tcp:gasztrotukor.database.windows.net,1433;
Initial Catalog=GasztroTukorDB;
Encrypt=True;
TrustServerCertificate=False;
User Id=gasztrotukor;
Password=1A2w3e4F

2.11. CONTROLLERS (VEZÉRLŐK)

Ezek a vezérlők felelősek az API végpontok kezeléséért, valamint az adatok visszaadásáért a kliens számára. A vezérlők a szolgáltatásokkal (Services) működnek együtt, hogy elérjék az adatbázist.

2.11.1. RESTAURANTCONTROLLER

A RestaurantController felelős az éttermekkel kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class RestaurantController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public RestaurantController()
        {
            _context = new ApplicationDbContext();
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Restaurant>>> GetAllRestaurants()
        {
            return await _context.Restaurants.ToListAsync();
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<Restaurant>> GetRestaurantById(int id)
        {
            var restaurant = await _context.Restaurants.FindAsync(id);

            if (restaurant == null)
            {
                return NotFound();
            }

            return restaurant;
        }

        [HttpGet("byName/{name}")]
        public async Task<ActionResult<Restaurant>> GetRestaurantByName(string name)
        {
            var restaurant = await _context.Restaurants.FirstOrDefaultAsync(r => r.name == name);
            if (restaurant == null)
            {
                return NotFound();
            }
            return restaurant;
        }

        [HttpGet("byOwner/{ownerId}")]
        public async Task<ActionResult<Restaurant>> GetRestaurantByOwnerId(int ownerId)
        {
            var restaurant = await _context.Restaurants.FirstOrDefaultAsync(r => r.owner_id == ownerId);
            if (restaurant == null)
            {
                return NotFound();
            }
            return restaurant;
        }

        [HttpPost]
        public async Task<ActionResult<Restaurant>> CreateRestaurant(Restaurant restaurant)
        {
            _context.Restaurants.Add(restaurant);
            await _context.SaveChangesAsync();

            return CreatedAtAction(nameof(GetRestaurantById), new { id = restaurant.restaurant_id }, restaurant);
        }
    }
}
```



```
[HttpPut("{id}")]
public async Task<IActionResult> UpdateRestaurant(int id, Restaurant restaurant)
{
    if (id != restaurant.restaurant_id)
    {
        return BadRequest();
    }
    _context.Entry(restaurant).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Restaurants.Any(r => r.restaurant_id == id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return Ok(restaurant);
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteRestaurant(int id)
{
    var restaurant = await _context.Restaurants.FindAsync(id);
    if (restaurant == null)
    {
        return NotFound();
    }

    restaurant.is_deleted = true;
    _context.Entry(restaurant).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return NoContent();
}
}
```

Végpontok:

- **GET /api/Restaurant** - Összes étterem lekérése
- **GET /api/Restaurant/{id}** - Étterem lekérése azonosító alapján
- **GET /api/Restaurant/ByName/{name}** - Étterem keresése név alapján
- **GET /api/Restaurant/byOwner/{ownerId}** - Étterem keresése tulajdonos azonosítója alapján
- **POST /api/Restaurant** - Új étterem létrehozása
- **PUT /api/Restaurant/{id}** - Étterem adatainak frissítése
- **DELETE /api/Restaurant/{id}** - Étterem törlése (logikai törlés)

Megjegyzés:

- A DELETE művelet az **is_deleted** mezőt állítja **true** értékre

2.11.2. USERCONTROLLER

A UserController felelős a felhasználókkal kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public UserController()
        {
            _context = new ApplicationDbContext();
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<User>>> GetAllUsers()
        {
            return await _context.Users.ToListAsync();
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<User>> GetUserById(int id)
        {
            var user = await _context.Users.FindAsync(id);

            if (user == null)
            {
                return NotFound();
            }

            return user;
        }

        [HttpGet("byEmail/{email}")]
        public async Task<ActionResult<User>> GetUserByEmail(string email)
        {
            var user = await _context.Users.FirstOrDefaultAsync(u => u.email == email && u.is_deleted == false);

            if (user == null)
            {
                return NotFound();
            }

            return user;
        }

        [HttpPost]
        public async Task<ActionResult<User>> CreateUser(User user)
        {
            _context.Users.Add(user);
            await _context.SaveChangesAsync();

            return CreatedAtAction(nameof(GetUserById), new { id = user.user_id }, user);
        }
    }
}
```

```
[HttpPut("{id}")]
public async Task<IActionResult> UpdateUser(int id, User user)
{
    if (id != user.user_id)
    {
        return BadRequest();
    }

    _context.Entry(user).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Users.Any(u => u.user_id == id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }

    return NoContent();
}
```

```
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteUser(int id)
{
    var user = await _context.Users.FindAsync(id);
    if (user == null)
    {
        return NotFound();
    }

    user.is_deleted = true;
    _context.Entry(user).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return NoContent();
}
}
```

Végpontok:

- **GET /api/User** - Összes felhasználó lekérése
- **GET /api/User/{id}** - Felhasználó lekérése azonosító alapján
- **GET /api/User/byEmail/{email}** - Felhasználó keresése email cím alapján
- **POST /api/User** - Új felhasználó létrehozása
- **PUT /api/User/{id}** - Felhasználói adatok frissítése
- **DELETE /api/User/{id}** - Felhasználó törlése (logikai törlés)

Megjegyzések:

- A DELETE művelet nem törli véglegesen a felhasználót, csak az **is_deleted** mezőt állítja **true** értékre
- Az email alapú lekérdezés csak a nem törölt felhasználókat adja vissza

2.11.3. REVIEWCONTROLLER

A ReviewController felelős a véleményekkel kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ReviewController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public ReviewController()
        {
            _context = new ApplicationDbContext();
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<Review>>> GetAllReviews()
        {
            return await _context.Reviews.ToListAsync();
        }

        [HttpGet("byRestaurant/{restaurantId}")]
        public async Task<ActionResult<IEnumerable<Review>>> GetReviewsByRestaurantId(int restaurantId)
        {
            var reviews = await _context.Reviews
                .Where(r => r.restaurant_id == restaurantId)
                .ToListAsync();
            return reviews;
        }

        [HttpGet("byUserId/{userId}")]
        public async Task<ActionResult<IEnumerable<Review>>> GetReviewsByUserId(int userId)
        {
            var reviews = await _context.Reviews
                .Where(r => r.user_id == userId)
                .ToListAsync();
            return reviews;
        }

        [HttpPost]
        public async Task<ActionResult<Review>> CreateReview(Review review)
        {
            _context.Reviews.Add(review);
            await _context.SaveChangesAsync();

            return Ok(review);
        }
    }
}
```

```
[HttpPut("{id}")]
public async Task<IActionResult> UpdateReview(int id, Review review)
{
    if (id != review.review_id)
    {
        return BadRequest();
    }
    _context.Entry(review).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.Reviews.Any(i => i.review_id == id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return Ok(review);
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteReview(int id)
{
    var review = await _context.Reviews.FindAsync(id);
    if (review == null)
    {
        return NotFound();
    }

    review.is_deleted = true;
    _context.Entry(review).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return NoContent();
}
}
```

Végpontok:

- **GET /api/Review** - Összes értékelés lekérése
- **GET /api/Review/byRestaurant/{restaurantId}** - Értékelések lekérése étterem azonosító alapján
- **GET /api/Review/byUserId/{userId}** - Értékelések lekérése felhasználó azonosító alapján
- **POST /api/Review** - Új értékelés létrehozása
- **PUT /api/Review/{id}** - Értékelés frissítése
- **DELETE /api/Review/{id}** - Értékelés törlése (logikai törlés)

Megjegyzés:

- A DELETE művelet az **is_deleted** mezőt állítja **true** értékre

2.11.4. MENUITEMCONTROLLER

A MenuItemController felelős az étlap elemekkel kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MenuItemController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public MenuItemController()
        {
            _context = new ApplicationDbContext();
        }

        [HttpGet("byRestaurantId/{id}")]
        public async Task<ActionResult<IEnumerable<MenuItem>>> GetMenuItemsByRestaurantId(int id)
        {
            var items = await _context.MenuItems.Where(i => i.restaurant_id == id).ToListAsync();
            if (items == null)
            {
                return NotFound();
            }
            return Ok(items);
        }

        [HttpGet("{id}")]
        public async Task<ActionResult<IEnumerable<MenuItem>>> GetMenuItemById(int id)
        {
            var item = await _context.MenuItems.FirstOrDefaultAsync(i => i.item_id == id);
            if (item == null)
            {
                return NotFound();
            }
            return Ok(item);
        }

        [HttpPost]
        public async Task<ActionResult<IEnumerable<MenuItem>>> CreateMenuItem(MenuItem item)
        {
            _context.MenuItems.Add(item);
            await _context.SaveChangesAsync();

            return CreatedAtAction(nameof(GetMenuItemById), new { id = item.item_id }, item);
        }
    }
}
```

```
[HttpPut("{id}")]
public async Task<IActionResult> UpdateMenuItem(int id, MenuItem item)
{
    if (id != item.item_id)
    {
        return BadRequest();
    }
    _context.Entry(item).State = EntityState.Modified;

    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!_context.MenuItems.Any(i => i.item_id == id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return Ok(item);
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteMenuItem(int id)
{
    var item = await _context.MenuItems.FindAsync(id);
    if (item == null)
    {
        return NotFound();
    }
    else
    {
        item.is_deleted = true;
        _context.Entry(item).State = EntityState.Modified;
    }

    await _context.SaveChangesAsync();
    return NoContent();
}
}
```

Végpontok:

- **GET /api/MenuItem/byRestaurantId/{id}** - Menüelemek lekérése étterem azonosító alapján
- **GET /api/MenuItem/{id}** - Menüelem lekérése azonosító alapján
- **POST /api/MenuItem** - Új menüelem létrehozása
- **PUT /api/MenuItem/{id}** - Menüelem adatainak frissítése
- **DELETE /api/MenuItem/{id}** - Menüelem törlése (logikai törlés)

Megjegyzés:

- A DELETE művelet az **is_deleted** mezőt állítja **true** értékre

2.11.5. MENUCATEGORYCONTROLLER

A MenuCategoryController felelős a menü kategóriákkal kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MenuCategoryController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public MenuCategoryController()
        {
            _context = new ApplicationDbContext();
        }

        [HttpGet]
        public async Task<ActionResult<IEnumerable<MenuCategory>>> GetAllCategories()
        {
            return await _context.MenuCategories.ToListAsync();
        }
    }
}
```

Végpont:

- GET /api/MenuCategory - Összes menükategória lekérése

2.11.6. EMAILCONTROLLER

Az EmailController felelős az emaillel kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]

    public class EmailController : ControllerBase
    {
        private const string FromEmail = "info@gasztrotukor.hu";
        private const string FromDisplayName = "GasztroTükör";
        private const string FromPassword = "UwdR8wSu9y9h";
        private const string Smtphost = "smtp.zoho.eu";
        private const int Smtport = 587;
        private const string BrandName = "GasztroTükör";

        [HttpPost("sendRegistrationEmail")]
        0 references
        public async Task<IActionResult> SendEmail([FromBody] EmailRequest emailRequest)
        {
            var fromAddress = new MailAddress(FromEmail, FromDisplayName);
            var toAddress = new MailAddress(emailRequest.ToEmail);
            const string subject = $"Sikerés regisztráció | {BrandName}";

            string body = @$"
            <!DOCTYPE html>
            <html>
            <head>
                <meta charset='utf-8'>
                <meta name='color-scheme' content='light'>
                <meta name='supported-color-schemes' content='light'>
            </head>
            <body style='margin: 0; padding: 0; background-color: #ffffff;'>
                <div style='font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px; color: #333333;'>
                    <div style='background-color: #c41e3a; padding: 15px; text-align: center;'>
                        <h1 style='color: white; margin: 0; font-size: 24px;'>{BrandName}</h1>
                    </div>
                    <div style='padding: 20px; border: 1px solid #dddddd; border-top: none; background-color: #ffffff;'>
                        <p style='font-size: 16px; color: #333333; margin-bottom: 15px;'>Kedves <span style='font-weight: bold; color: #333333;'>{emailRequest.ToEmail}</span>!</p>
                        <p style='font-size: 16px; line-height: 1.5; color: #333333; margin-bottom: 15px;'>Köszönjük, hogy regisztrált a <span style='color: #c41e3a; font-weight: bold;'>{BrandName}</span> oldalra. Reméljük, hogy élvezni fogja az oldal nyújtotta lehetőségeket.</p>

                    if (emailRequest.IsOwner)
                    {
                        body += @"<p style='font-size: 16px; line-height: 1.5; background-color: #faf3d1; padding: 10px; border-left: 4px solid #f3d55b; color: #333333;'>Mivel tulajdonosként regisztrált, az étterme nem jelenik meg nyilvánosan, amíg az oldal egy adminisztrátora jóvá nem hagyja.</p>";
                    }

                    body += @$"
                    <p style='font-size: 16px; margin-top: 30px; color: #333333;'>Üdvözléssel,<br>
                    <span style='color: #c41e3a; font-weight: bold;'>A {BrandName} csapata</span></p>
                    </div>
                    <div style='text-align: center; padding: 10px; font-size: 12px; color: #777777;'>
                        <p>© {DateTime.Now.Year} {BrandName} - Minden jog fenntartva</p>
                        <p style='border-top: 1px solid #dddddd; padding-top: 10px;'>Ez egy automatikusan küldött üzenet. Kérjük, ne válaszoljon erre az e-mailre!</p>
                    </div>
                </div>
            </body>
            </html>";

            var smtp = new SmtpClient
            {
                Host = Smtphost,
                Port = Smtport,
                EnableSsl = true,
                DeliveryMethod = SmtpDeliveryMethod.Network,
                UseDefaultCredentials = false,
                Credentials = new NetworkCredential(fromAddress.Address, FromPassword)
            };

            using (var message = new MailMessage(fromAddress, toAddress)
            {
                Subject = subject,
                Body = body,
                IsBodyHtml = true
            })
            {
                await smtp.SendMailAsync(message);
            }

            return Ok("Regisztrációs email sikeresen elküldve.");
        }
    }
}
```



```

[HttpPost("sendForgotPassword")]

public async Task<IActionResult> SendForgotPassword([FromBody] ForgotPasswordRequest request)
{
    var fromAddress = new MailAddress(FromEmail, FromDisplayName);
    var toAddress = new MailAddress(request.ToEmail);
    const string subject = $"Elfelejtett jelszó | {BrandName}";

    string body = @$"
<!DOCTYPE html>
<html>
<head>
    <meta charset='utf-8'>
    <meta name='color-scheme' content='light'>
    <meta name='supported-color-schemes' content='light'>
</head>
<body style='margin: 0; padding: 0; background-color: #ffffff;'>
    <div style='font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px; color:
    #333333;'>
        <div style='background-color: #c4le3a; padding: 15px; text-align: center;'>
            <h1 style='color: white; margin: 0; font-size: 24px;'>{BrandName}</h1>
        </div>
        <div style='padding: 20px; border: 1px solid #dddddd; border-top: none; background-color: #ffffff;'>
            <p style='font-size: 16px; color: #333333; margin-bottom: 15px;'>Kedves <span style='font-weight:
            bold; color: #333333;'>{request.ToEmail}</span>!</p>
            <p style='font-size: 16px; line-height: 1.5; color: #333333; margin-bottom: 15px;'>Az új jelszava:</
            p>
            <div style='background-color: #f5f5f5; padding: 15px; margin: 20px 0; border-radius: 4px; text-
            align: center;'>
                <span style='font-family: monospace; font-size: 18px; font-weight: bold; letter-spacing: 1px;
                color: #333333;'>{request.NewPassword}</span>
            </div>
            <p style='font-size: 14px; color: #777777; margin-bottom: 15px;'>Ez egy automatikusan generált
            jelszó. A biztonság érdekében javasoljuk, hogy bejelentkezés után a Beállítások menüpontban
            változtassa meg.</p>
            <p style='font-size: 16px; margin-top: 30px; color: #333333;'>Üdvözlettel,<br>
            <span style='color: #c4le3a; font-weight: bold;'>A {BrandName} csapata</span></p>
        </div>
        <div style='text-align: center; padding: 10px; font-size: 12px; color: #777777;'>
            <p>© {DateTime.Now.Year} {BrandName} - Minden jog fenntartva</p>
            <p style='border-top: 1px solid #dddddd; padding-top: 10px;'>Ez egy automatikusan küldött üzenet.
            Kérjük, ne válaszoljon erre az e-mailre!</p>
        </div>
    </div>
</body>
</html>";

    var smtp = new SmtpClient
    {
        Host = SmtpHost,
        Port = SmtpPort,
        EnableSsl = true,
        DeliveryMethod = SmtpDeliveryMethod.Network,
        UseDefaultCredentials = false,
        Credentials = new NetworkCredential(fromAddress.Address, FromPassword)
    };

    using (var message = new MailMessage(fromAddress, toAddress)
    {
        Subject = subject,
        Body = body,
        IsBodyHtml = true
    })
    {
        await smtp.SendMailAsync(message);
    }

    return Ok("Elfelejtett jelszó email sikeresen elküldve.");
}

public class EmailRequest
{
    public string ToEmail { get; set; }

    public bool IsOwner { get; set; }
}

public class ForgotPasswordRequest
{
    public string ToEmail { get; set; }

    public string NewPassword { get; set; }
}
}

```

Végpontok:

- **POST /api/Email/sendRegistrationEmail**
 - Sikeres regisztráció után üdvözlő emailt küld a felhasználónak
 - Ha a felhasználó étterem tulajdonos, akkor tájékoztatást kap a jóváhagyási folyamatról
 - A küldés a Zoho SMTP szerverén keresztül történik
- **POST /api/Email/sendForgotPassword**
 - Új, automatikusan generált jelszót küld a felhasználónak
 - Biztonsági figyelmeztetést tartalmaz a jelszó megváltoztatásáról
 - Szintén a Zoho SMTP szerverén keresztül küldi az emailt

A kontroller két adatmodellt használ:

- **EmailRequest:** Regisztrációs email küldéséhez
 - ToEmail: A címzett email címe
 - IsOwner: Jelzi, hogy a felhasználó étterem tulajdonos-e
- **ForgotPasswordRequest:** Elfelejtett jelszó emailhez
 - ToEmail: A címzett email címe
 - NewPassword: Az új, automatikusan generált jelszó

2.12. SERVICES (SZOLGÁLTATÁSOK)

2.12.1. RESTAURANTSERVICE

A RestaurantService osztály felelős az éttermekkel kapcsolatos API hívásokért:

```
public class RestaurantService
{
    private readonly HttpClient _httpClient;

    public RestaurantService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<List<Restaurant>> GetAllRestaurants()
    {
        return await _httpClient.GetFromJsonAsync<List<Restaurant>>("api/Restaurant/");
    }

    public async Task<Restaurant> GetRestaurantById(int id)
    {
        return await _httpClient.GetFromJsonAsync<Restaurant>($"api/Restaurant/{id}");
    }

    public async Task<Restaurant> GetRestaurantByName(string name)
    {
        return await _httpClient.GetFromJsonAsync<Restaurant>($"api/Restaurant/byName/{name}");
    }

    public async Task<Restaurant> GetRestaurantByOwnerId(int ownerId)
    {
        return await _httpClient.GetFromJsonAsync<Restaurant>($"api/Restaurant/byOwner/{ownerId}");
    }

    public async Task CreateRestaurant(Restaurant restaurant)
    {
        await _httpClient.PostAsJsonAsync("api/Restaurant", restaurant);
    }

    public async Task<MenuItem> UpdateRestaurant(Restaurant restaurant)
    {
        var response = await _httpClient.PutAsJsonAsync($"api/Restaurant/{restaurant.restaurant_id}", restaurant);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<MenuItem>();
    }

    public async Task DeleteRestaurant(int restaurantId)
    {
        await _httpClient.DeleteAsync($"api/Restaurant/{restaurantId}");
    }
}
```

- **GetAllRestaurants()**: Visszaadja az összes éttermet a rendszerben
- **GetRestaurantById(int id)**: Egy adott étterem lekérdezése azonosító alapján
- **GetRestaurantByName(string name)**: Egy adott étterem lekérdezése név alapján
- **GetRestaurantByOwnerId(int ownerId)**: Egy adott tulajdonoshoz tartozó étterem lekérdezése
- **CreateRestaurant(Restaurant restaurant)**: Új étterem létrehozása
- **UpdateRestaurant(Restaurant restaurant)**: Étterem adatainak frissítése
- **DeleteRestaurant(int restaurantId)**: Étterem törlése azonosító alapján

2.12.2. MENUITEMSERVICE

A MenuItemService osztály felelős a menüelemekkel kapcsolatos API hívásokért:

```
public class MenuItemService
{
    private readonly HttpClient _httpClient;

    public MenuItemService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<MenuItem>> GetMenuItemsByRestaurantId(int restaurantId)
    {
        return await _httpClient.GetFromJsonAsync<IEnumerable<MenuItem>>($"api/MenuItem/restaurant/{restaurantId}");
    }

    public async Task<MenuItem> GetMenuItem(int itemId)
    {
        return await _httpClient.GetFromJsonAsync<MenuItem>($"api/MenuItem/{itemId}");
    }

    public async Task<MenuItem> CreateMenuItem(MenuItem menuItem)
    {
        var response = await _httpClient.PostAsJsonAsync("api/MenuItem", menuItem);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<MenuItem>();
    }

    public async Task<MenuItem> UpdateMenuItem(MenuItem menuItem)
    {
        var response = await _httpClient.PutAsJsonAsync($"api/MenuItem/{menuItem.item_id}", menuItem);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<MenuItem>();
    }

    public async Task DeleteMenuItem(int itemId)
    {
        await _httpClient.DeleteAsync($"api/MenuItem/{itemId}");
    }
}
```

- **GetMenuItemsByRestaurantId(int restaurantId):** Egy étterem összes menüelemének lekérdezése
- **GetMenuItem(int itemId):** Menüelem lekérdezése azonosító alapján
- **CreateMenuItem(MenuItem menuItem):** Új menüelem létrehozása
- **UpdateMenuItem(MenuItem menuItem):** Menüelem frissítése
- **DeleteMenuItem(int itemId):** Menüelem törlése azonosító alapján

2.12.3. MENUCATEGORYSERVICE

A MenuCategoryService osztály felelős a menü kategóriákkal kapcsolatos API hívásokért:

```
public class MenuCategoryService
{
    private readonly HttpClient _httpClient;

    public MenuCategoryService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<MenuCategory>> GetAllCategories()
    {
        return await _httpClient.GetFromJsonAsync<IEnumerable<MenuCategory>>("api/MenuCategory");
    }

    public async Task<MenuCategory> GetCategory(int categoryId)
    {
        return await _httpClient.GetFromJsonAsync<MenuCategory>($"api/MenuCategory/{categoryId}");
    }
}
```

- **GetAllCategories():** Az összes menükategória lekérdezése
- **GetCategory(int categoryId):** Egy adott menükategória lekérdezése azonosító alapján

2.12.4. REVIEWSERVICE

A ReviewService osztály felelős az értékelésekkel kapcsolatos API hívásokért:

```
public class ReviewService
{
    private readonly HttpClient _httpClient;

    public ReviewService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<List<Review>> GetReviewsByRestaurantId(int restaurantId)
    {
        return await _httpClient.GetFromJsonAsync<List<Review>>($"api/Review/restaurant/{restaurantId}");
    }

    public async Task<Review> GetReview(int id)
    {
        return await _httpClient.GetFromJsonAsync<Review>($"api/Review/{id}");
    }

    public async Task<Review> CreateReview(Review review)
    {
        var response = await _httpClient.PostAsJsonAsync("api/Review", review);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<Review>();
    }

    public async Task<bool> DeleteReview(int id)
    {
        var response = await _httpClient.DeleteAsync($"api/Review/{id}");
        return response.IsSuccessStatusCode;
    }
}
```

- **GetReviewsByRestaurantId(int restaurantId):** Egy étteremhez tartozó összes értékelés lekérdezése

- **GetReview(int id):** Értékelés lekérdezése azonosító alapján
- **CreateReview(Review review):** Új értékelés létrehozása
- **DeleteReview(int id):** Értékelés törlése azonosító alapján

2.12.5. USERSERVICE

A UserService osztály felelős a felhasználókkal kapcsolatos API hívásokért:

```
public class UserService
{
    private readonly HttpClient _httpClient;

    public UserService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<User> GetUserById(int id)
    {
        return await _httpClient.GetFromJsonAsync<User>($"api/User/{id}");
    }

    public async Task<User> GetUserByEmail(string email)
    {
        return await _httpClient.GetFromJsonAsync<User>($"api/User/email/{email}");
    }

    public async Task<User> CreateUser(User user)
    {
        var response = await _httpClient.PostAsJsonAsync("api/User", user);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<User>();
    }

    public async Task<User> UpdateUser(User user)
    {
        var response = await _httpClient.PutAsJsonAsync($"api/User/{user.user_id}", user);
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<User>();
    }

    public async Task DeleteUser(int id)
    {
        await _httpClient.DeleteAsync($"api/User/{id}");
    }
}
```

- **GetUserById(int id):** Felhasználó lekérdezése azonosító alapján
- **GetUserByEmail(string email):** Felhasználó lekérdezése email cím alapján
- **CreateUser(User user):** Új felhasználó létrehozása
- **UpdateUser(User user):** Felhasználó adatainak frissítése
- **DeleteUser(int id):** Felhasználó törlése azonosító alapján

2.12.6. EMAILSERVICE

Az EmailService osztály felelős az email küldéssel kapcsolatos API hívásokért:

```
public class EmailService
{
    private readonly HttpClient _httpClient;

    public EmailService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task SendRegistrationEmailAsync(string email, bool isOwner)
    {
        var request = new { ToEmail = email, IsOwner = isOwner };
        await _httpClient.PostAsJsonAsync("api/Email/sendRegistrationEmail", request);
    }

    public async Task SendForgotPasswordEmailAsync(string email, string newPassword)
    {
        var request = new { ToEmail = email, NewPassword = newPassword };
        await _httpClient.PostAsJsonAsync("api/Email/sendForgotPassword", request);
    }
}
```

- **SendRegistrationEmailAsync(string email, bool isOwner):** Regisztrációs email küldése
- **SendForgotPasswordEmailAsync(string email, string newPassword):** Elfelejtett jelszó email küldése

2.12.7. AUTHSTATEPROVIDERSERVICE

Az AuthStateProviderService egy olyan szolgáltatás a Blazor WebAssembly projektben, amely a felhasználói hitelesítési állapot kezelését végzi. Ez a szolgáltatás a Microsoft által biztosított **AuthenticationStateProvider** absztrakt osztályból származik, amely a Blazor alkalmazások hitelesítési infrastruktúrájának központi eleme.

2.12.7.1. HITELESÍTÉSI ÁLLAPOT KEZELÉSE

```
public class AuthStateProviderService : AuthenticationStateProvider
{
    private readonly ILocalStorageService _localStorage;
    private readonly ClaimsPrincipal _anonymous = new ClaimsPrincipal(new ClaimsIdentity());
    public const int LoginTimeLimitMinutes = 45;
}
```

- Az osztály a **AuthenticationStateProvider** absztrakt osztályt implementálja
- Használja a **Blazored.LocalStorage** csomagot az adatok helyi tárolásához
- Az alapértelmezett munkamenet időkorlát 45 perc

2.12.7.2. FELHASZNÁLÓI HITELESÍTÉS LEKÉRDEZÉSE

```
public override async Task<AuthenticationState> GetAuthenticationStateAsync()
{
    try
    {
        var storedToken = await _localStorage.GetItemAsync<string>("authToken");

        if (string.IsNullOrEmpty(storedToken))
        {
            return new AuthenticationState(_anonymous);
        }

        var userData = JsonSerializer.Deserialize<UserSession>(storedToken);

        if (userData == null || DateTime.UtcNow - userData.LoginTime > TimeSpan.FromMinutes(LoginTimeLimitMinutes))
        {
            await NotifyUserLogout();
            return new AuthenticationState(_anonymous);
        }

        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, userData.Email),
            new Claim(ClaimTypes.Role, userData.Role),
            new Claim(ClaimTypes.NameIdentifier, userData.UserId.ToString())
        };

        var identity = new ClaimsIdentity(claims, "Custom");
        var user = new ClaimsPrincipal(identity);

        return new AuthenticationState(user);
    }
    catch
    {
        return new AuthenticationState(_anonymous);
    }
}
```

- Ez a metódus ellenőrzi a localStorage-ban tárolt "authToken" értékét
- Ha nincs token vagy lejárt, anonim felhasználót ad vissza
- Ha van érvényes token, visszaállítja a felhasználó adatait és szerepkörét

2.12.7.3. FELHASZNÁLÓI BEJELENTKEZÉS ÉRTESÍTÉSE

```
public async Task NotifyUserAuthentication(ClaimsPrincipal user)
{
    if (user == null || !user.Identity.IsAuthenticated)
    {
        throw new ArgumentException("Érvénytelen felhasználó vagy a felhasználó nincs hitelesítve.");
    }

    var userData = new UserSession
    {
        Email = user.FindFirst(ClaimTypes.Name)?.Value,
        Role = user.FindFirst(ClaimTypes.Role)?.Value,
        UserId = int.Parse(user.FindFirst(ClaimTypes.NameIdentifier)?.Value),
        LoginTime = DateTime.UtcNow
    };

    var serializedData = JsonSerializer.Serialize(userData);
    await _localStorage.SetItemAsync("authToken", serializedData);

    NotifyAuthenticationStateChanged(Task.FromResult(new AuthenticationState(user)));
}
```

- Ez a metódus létrehoz egy felhasználói munkamenetet és eltárolja a localStorage-ban
- Létrehozza a felhasználó jogosultsági adatait (claims) tartalmazó objektumot
- Értesíti az alkalmazás többi részét a bejelentkezésről

2.12.7.4. FELHASZNÁLÓI KIJELENTKEZÉS ÉRTESÍTÉSE

```
public async Task NotifyUserLogout()
{
    await _localStorage.RemoveItemAsync("authToken");
    NotifyAuthenticationStateChanged(Task.FromResult(new AuthenticationState(_anonymous)));
}
```

- Eltávolítja a tárolt tokent a localStorage-ból
- Értesíti az alkalmazás többi részét a kijelentkezésről

2.12.7.5. USERSESSION OSZTÁLY

```
public class UserSession
{
    public string Email { get; set; }
    public string Role { get; set; }
    public int UserId { get; set; }
    public DateTime LoginTime { get; set; }
}
```

- Ez a beágyazott osztály tárolja a munkamenettel kapcsolatos felhasználói adatokat
- A localStorage-ban JSON formátumban tárolódik
- Tartalmazza a felhasználó e-mail címét, szerepkörét, azonosítóját és a bejelentkezés időpontját

2.12.7.6. HASZNÁLATA AZ ALKALMAZÁSBAN

Az AuthServiceProviderService az alkalmazás számos részén felhasználásra kerül:

- **Navigációs menüben:** Figyeli a hitelesítési állapot változását és frissíti a felhasználói felületet
- **Munkamenet időzítőben:** Nyomon követi a felhasználói munkamenet időkorlátját
- **Komponensekben:** Az [Authorize] attribútum és az <AuthorizeView> komponens ezen keresztül ellenőrzi a jogosultságokat
- **Oldalakon:** Jogosultság alapján szűri a megjeleníthető tartalmakat

2.12.7.7. MUNKAMENET KEZELÉS

- A munkamenet alapértelmezés szerint 45 percre áll
- A NavMenu komponensben egy visszaszámláló jelzi a munkamenet hátralévő idejét
- Ha a munkamenet lejár, a felhasználó automatikusan kijelentkezik
- Az időzítő a bejelentkezéskor indul és másodpercenként frissül

2.12.7.8. BIZTONSÁG

- A felhasználói adatok helyi tárolóban (localStorage) vannak elmentve
- A jelszavak soha nem kerülnek tárolásra, csak szerver oldalon történik a jelszó ellenőrzése
- A munkamenet korlátozott ideig érvényes, ami után újra be kell jelentkezni
- A jogosultságokat mindenhol ellenőrzi a rendszer

2.13. WEB/PROGRAM.CS

Ez a fájl a Blazor WebAssembly kliens alkalmazás belépési pontja, amely a böngészőben fut. Inicializálja a Blazor WebAssembly alkalmazást és konfigurálja a függőségi injektálás szolgáltatásait.

2.13.1. NÉVTEREK ÉS FÜGGŐSÉGEK

```
using Microsoft.AspNetCore.Components.Web;  
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;  
using web.Services;  
using Microsoft.AspNetCore.Components.Authorization;  
using SharedLibrary.Models;  
using Blazored.LocalStorage;
```

- **Microsoft.AspNetCore.Components.Web:** Biztosítja a Blazor webalkalmazásokhoz szükséges komponenseket
- **Microsoft.AspNetCore.Components.WebAssembly.Hosting:** A WebAssembly host környezet beállításához szükséges
- **web.Services:** Tartalmazza az alkalmazás által használt szolgáltatásokat
- **Microsoft.AspNetCore.Components.Authorization:** Az autentikációs állapot kezeléséhez szükséges
- **SharedLibrary.Models:** Megosztott modellek, beleértve az adatbázis kontextust
- **Blazored.LocalStorage:** Helyi tárolás kezeléséhez szükséges külső könyvtár

2.13.2. WEBASSEMBLY HOST KONFIGURÁLÁSA

```
var builder = WebAssemblyHostBuilder.CreateDefault(args);  
builder.RootComponents.Add<App>("body");  
builder.RootComponents.Add<HeadOutlet>("head::after");
```

- Létrehozza az alapértelmezett WebAssembly host builder-t
- Hozzáadja az **App** komponenst a HTML body elemhez
- Hozzáadja a **HeadOutlet** komponenst, amely lehetővé teszi dinamikus tartalom hozzáadását a head elemhez

2.13.3. HTTP KLIENS REGISZTRÁLÁSA

```
builder.Services.AddScoped(sp => new HttpClient
{
    BaseAddress = new Uri(builder.HostEnvironment.IsDevelopment()
        ? "https://localhost:7129/"
        : "https://gasztrotukor-api.azurewebsites.net/")
});
```

HTTP kliens regisztrálása különböző alap URL-ekkel fejlesztési és éles környezetben:

- Fejlesztési környezetben: **https://localhost:7129/**
- Éles környezetben: **https://gasztrotukor-api.azurewebsites.net/**

2.13.4. ADATBÁZIS ÉS SZOLGÁLTATÁSOK REGISZTRÁLÁSA

```
builder.Services.AddBlazoredLocalStorage();
builder.Services.AddAuthorizationCore();
builder.Services.AddScoped<AuthStateProviderService>();
builder.Services.AddScoped<AuthenticationStateProvider>(provider =>
    provider.GetRequiredService<AuthStateProviderService>());
builder.Services.AddDbContext<ApplicationDbContext>();
builder.Services.AddScoped<RestaurantService>();
builder.Services.AddScoped<UserService>();
builder.Services.AddScoped<EmailService>();
builder.Services.AddScoped<MenuItemService>();
builder.Services.AddScoped<MenuCategoryService>();
builder.Services.AddScoped<ReviewService>();
```

2.14. API/PROGRAM.CS

Ez a fájl az API projekt belépési pontja, amely egy ASP.NET Core Web API alkalmazást konfigurál és futtat. Ez az API szolgáltatja az adatokat a Blazor WebAssembly kliensnek.

2.14.1. WEBAPPLICATION BUILDER LÉTREHOZÁSA

```
var builder = WebApplication.CreateBuilder(args);
```

- Létrehozza az ASP.NET Core WebApplication builder-t a parancssori argumentumok alapján

2.14.2. SZOLGÁLTATÁSOK REGISZTRÁLÁSA

```
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

- **AddControllers:** Hozzáadja a controller alapú routing támogatást
- **AddEndpointsApiExplorer:** Az API végpontok automatikus feltárását teszi lehetővé
- **AddSwaggerGen:** Swagger dokumentáció generálásához szükséges szolgáltatás

2.14.3. CORS (CROSS-ORIGIN RESOURCE SHARING) BEÁLLÍTÁSA

```
var corsPolicy = "allowedCorsPolicy";
builder.Services.AddCors(options =>
{
    options.AddPolicy(corsPolicy, policyBuilder =>
    {
        policyBuilder.WithOrigins("https://www.gasztrotukor.hu", "https://localhost:7272")
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});
```

CORS házirend konfigurálása, amely meghatározza, mely forrásokból érkehetnek kérések az API-hoz:

- Engedélyezett eredetek: **https://www.gasztrotukor.hu** és **https://localhost:7272**
- Bármely HTTP fejléc engedélyezése
- Bármely HTTP metódus engedélyezése
- Hitelesítési adatok (pl. cookie-k) engedélyezése

2.14.4. ALKALMAZÁS KIÉPÍTÉSE ÉS KONFIGURÁLÁSA

```
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

- Létrehozza az alkalmazást a konfigurált szolgáltatásokkal
- Fejlesztési környezetben engedélyezi a Swagger API dokumentációt

2.14.5. MIDDLEWARE KONFIGURÁLÁSA

```
app.UseHttpsRedirection();
app.UseCors(corsPolicy);
app.UseAuthorization();
app.MapControllers();
```

- **UseHttpsRedirection**: Automatikus átirányítás HTTPS-re
- **UseCors**: A korábban definiált CORS házirend alkalmazása
- **UseAuthorization**: Autorizációs middleware alkalmazása
- **MapControllers**: Végpontok konfigurálása a controller-ek alapján

2.15. NAVMENU.RAZOR

A NavMenu.razor fájl a GasztroTükör webalkalmazás navigációs menüjét tartalmazza. Ez a komponens kezeli a bejelentkezett és nem bejelentkezett felhasználók navigációs elemeit, valamint különböző felhasználói műveletek végrehajtását (jelszómódosítás, fiók törlése).

2.15.1. HTML STRUKTÚRA

Az alkalmazás az `<AuthorizeView>` Blazor komponenszt használja a hitelesített és nem hitelesített felhasználók számára különböző navigációs felületek megjelenítésére:

```
<AuthorizeView>
  <Authorized>
    <!-- Bejelentkezett felhasználók számára megjelenő UI -->
  </Authorized>
  <NotAuthorized>
    <!-- Nem bejelentkezett felhasználók számára megjelenő UI -->
  </NotAuthorized>
</AuthorizeView>
```

Bejelentkezett felhasználók navigációs sávja

```
<section class="navigation-section">
  <nav class="navbar navbar-expand-lg navbar-dark authenticated">
    <div class="container">
      <!-- Logo és branding -->
      <NavLink class="navbar-brand" href="/">
        <i class="fas fa-utensils me-2"></i>GasztroTükör
      </NavLink>

      <!-- Munkamenet időzítő (mobil nézet) -->
      <div class="d-lg-none session-timer text-light text-center me-2 @({timeLeft?.TotalMinutes < 1 ? "timer-warning" : ""})">
        <!-- Időzítő ikon és szöveg -->
      </div>

      <!-- Hamburger menü gomb -->
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <!-- Navigációs elemek -->
      <div class="collapse navbar-collapse" id="navbarNav">
        <!-- Mobil és asztali nézetekhez különböző elemek -->
      </div>
    </div>
  </nav>
</section>
```

Nem bejelentkezett felhasználók navigációs sávja

```
<section class="navigation-section">
  <nav class="navbar navbar-expand-sm navbar-dark">
    <div class="container">
      <!-- Logo és branding -->
      <NavLink class="navbar-brand" href="/">
        <i class="fas fa-utensils me-2"></i>GasztroTükör
      </NavLink>

      <!-- Hamburger menü gomb -->
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <!-- Bejelentkezés és regisztrációs gombok (mobil és asztali nézet) -->
      <div class="collapse navbar-collapse" id="navbarNav">
        <!-- Különböző elrendezés mobil és asztali nézethez -->
      </div>
    </div>
  </nav>
</section>
```

A komponens több Bootstrap modált is tartalmaz:

1. Felhasználói beállítások modál:

```
<div class="modal fade login-modal" id="userDataModal" tabindex="-1" aria-labelledby="userDataModallabel" aria-hidden="true">
  <div class="modal-dialog modal-lg">
    <div class="modal-content">
      <!-- Modál fejléce -->
      <div class="modal-header">
        <h5 class="modal-title" id="userDataModallabel">Beállítások</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Bezárás"></button>
      </div>

      <!-- Modál tartalma: tab navigáció különböző funkciókhoz -->
      <div class="modal-body">
        <!-- Tab navigáció -->
        <ul class="nav nav-tabs mb-3" id="userDataTabs" role="tablist">
          <!-- Tabfülek -->
        </ul>

        <div class="tab-content" id="userDataTabContent">
          <!-- Különböző tabok tartalma -->
        </div>
      </div>
    </div>
  </div>
</div>
```

2. Értékelés szerkesztése modál:

```
<div class="modal fade login-modal" id="editReviewModal" tabindex="-1" aria-labelledby="editReviewModallabel" aria-hidden="true">
  <!-- Értékelés szerkesztési űrlap -->
</div>
```

Tab rendszer

A felhasználói beállítások modálban tab rendszer segítségével szerveződik a tartalom:

```
<!-- Tab navigáció -->
<ul class="nav nav-tabs mb-3" id="userDataTabs" role="tablist">
  <li class="nav-item" role="presentation">
    <button class="nav-link active" id="profile-tab" data-bs-toggle="tab"
      data-bs-target="#profile-tab-pane" type="button" role="tab">
      Profiladatok
    </button>
  </li>
  <!-- További tabok szerepkör alapján -->
</ul>

<div class="tab-content" id="userDataTabContent">
  <!-- Profil adatok tab -->
  <div class="tab-pane fade show active" id="profile-tab-pane" role="tabpanel" aria-labelledby="profile-tab" tabindex="0">
    <!-- Profiladatok űrlap -->
  </div>

  <!-- További tab panelek -->
</div>
```


Feltételes megjelenítés szerepkörök alapján

A HTML tartalom feltételesen jelenik meg a felhasználói szerepkörtől függően:

```
@if (context.User.IsInRole("admin"))
{
    <NavLink href="/admin" class="btn btn-outline-warning me-2">
        <i class="fa-solid fa-shield-quartered me-1"></i>Adminisztrátori felület
    </NavLink>
}
else if (context.User.IsInRole("owner"))
{
    <NavLink href="/myrestaurant" class="btn btn-outline-warning me-2">
        <i class="fa-regular fa-store me-1"></i>Éttermem kezelése
    </NavLink>
}
```

2.15.2. JAVASCRIPT INTEGRÁCIÓ

`window.addEventListener('scroll', function...):`

Figyeli a görgetést és hozzáad egy 'scrolled' osztályt a navigációs sávhoz görgetéskor.

```
window.addEventListener('scroll', function () {
    const navbar = document.querySelector('.navbar');
    if (window.scrollY > 50) {
        navbar.classList.add('scrolled');
    } else {
        navbar.classList.remove('scrolled');
    }
});
```

`window.togglePassword:`

```
window.togglePassword = function (inputId, iconId) {
    const input = document.getElementById(inputId);
    const icon = document.getElementById(iconId);
    if (input.type === 'password') {
        input.type = 'text';
        icon.classList.remove('fa-eye');
        icon.classList.add('fa-eye-slash');
    } else {
        input.type = 'password';
        icon.classList.remove('fa-eye-slash');
        icon.classList.add('fa-eye');
    }
};
```

2.15.3. C# METÓDUSOK

OnInitializedAsync():

A komponens inicializálásakor fut le, beállítja a bejelentkezett felhasználót és elindítja a munkamenet időzítőt.

```
protected override async Task OnInitializedAsync()
{
    var authState = await AuthenticationStateProvider.GetAuthenticationStateAsync();
    if (authState.User.Identity.IsAuthenticated)
    {
        currentUserEmail = authState.User.Identity.Name;
        StartTimer();
    }

    AuthStateProviderService.AuthenticationStateChanged += async (task) =>
    {
        var state = await task;
        if (state.User.Identity.IsAuthenticated)
        {
            currentUserEmail = state.User.Identity.Name;
            StartTimer();
        }
        else
        {
            currentUserEmail = string.Empty;
            StopTimer();
        }
        StateHasChanged();
    };
}
```

StartTimer():

Elindítja a munkamenet időzítőt, amely figyeli a bejelentkezés érvényességét.

```
private async void StartTimer()
{
    try
    {
        var token = await LocalStorage.GetItemAsync<string>("authToken");
        if (!string.IsNullOrEmpty(token))
        {
            var userData = System.Text.Json.JsonSerializer.Deserialize<UserSession>(token);
            sessionEndTime = userData.LoginTime.AddMinutes(AuthStateProviderService.LoginTimeLimitMinutes);

            timer?.Dispose();
            timer = new Timer(1000);
            timer.Elapsed += (s, e) => UpdateTimer();
            timer.Start();
            UpdateTimer();
        }
    }
    catch { }
}
```


UpdateTimer():

Frissíti a munkamenet hátralévő idejét és kezeli a lejárt munkamenetet.

```
private void UpdateTimer()
{
    timeLeft = sessionEndTime - DateTime.UtcNow;

    if (timeLeft?.TotalSeconds <= 0)
    {
        InvokeAsync(async () =>
        {
            await HandleLogout();
            notification?.ShowMessage("A munkamenet lejárt. Kérjük, jelentkezzen be újra.", "yellow");
        });
        return;
    }

    InvokeAsync(StateHasChanged);
}
```

FormatTimeRemaining(TimeSpan? span):

Formázza a hátralévő időt olvasható formátumra (perc:másodperc).

```
private string FormatTimeRemaining(TimeSpan? span)
{
    if (span == null) return "00:00";
    return $"{(int)span.Value.TotalMinutes:d2}:{span.Value.Seconds:d2}";
}
```

StopTimer():

Leállítja és eltávolítja a munkamenet időzítőt.

```
private void StopTimer()
{
    timer?.Stop();
    timer?.Dispose();
    timer = null;
    timeLeft = null;
}
```

HandleLogout():

Kezeli a felhasználó kijelentkezését.

```
private async Task HandleLogout()
{
    await AuthStateProviderService.NotifyUserLogout();
    StopTimer();
    currentUserEmail = string.Empty;
    notification?.ShowMessage("Sikeres kijelentkezés.", "green");
}
```

Dispose():

```
public void Dispose()
{
    timer?.Dispose();
    AuthStateProviderService.AuthenticationStateChanged -= async (task) =>
    {
        var state = await task;
        if (state.User.Identity.IsAuthenticated) StartTimer();
        else StopTimer();
        StateHasChanged();
    };
}
```

ValidatePassword(ChangeEventArgs e):

Ellenőrzi a megadott új jelszó megfelelőségét.

```
private void ValidatePassword(ChangeEventArgs e)
{
    newPassword = e?.Value?.ToString() ?? string.Empty;
    passwordErrorMessage = GetPasswordValidationError(newPassword);
    ValidatePasswordConfirm(new ChangeEventArgs { Value = confirmPassword });
}
```

ValidatePasswordConfirm(ChangeEventArgs e):

Ellenőrzi, hogy a jelszó megerősítése egyezik-e az új jelszóval.

```
private void ValidatePasswordConfirm(ChangeEventArgs e)
{
    confirmPassword = e?.Value?.ToString() ?? string.Empty;
    if (newPassword != confirmPassword)
    {
        passwordConfirmErrorMessage = "A jelszavak nem egyeznek.";
    }
    else
    {
        passwordConfirmErrorMessage = string.Empty;
    }
    passwordErrorMessage = GetPasswordValidationError(newPassword);
}
```

GetPasswordValidationError(string password)

Ellenőrzi a jelszó megfelelését a biztonsági követelményeknek.

```
private string GetPasswordValidationError(string password)
{
    if (string.IsNullOrEmpty(password))
    {
        return string.Empty;
    }
    if (password.Length < 8 || password.Length > 20)
    {
        return "A jelszónak 8 és 20 karakter közötti hosszúnak kell lennie.";
    }
    if (!System.Text.RegularExpressions.Regex.IsMatch(password, "[A-Z]"))
    {
        return "A jelszónak tartalmaznia kell legalább egy nagybetűt.";
    }
    if (!System.Text.RegularExpressions.Regex.IsMatch(password, "[a-z]"))
    {
        return "A jelszónak tartalmaznia kell legalább egy kisbetűt.";
    }
    if (!System.Text.RegularExpressions.Regex.IsMatch(password, "\\d"))
    {
        return "A jelszónak tartalmaznia kell legalább egy számot.";
    }
    if (!System.Text.RegularExpressions.Regex.IsMatch(password, "[!@#%&*._-]"))
    {
        return "A jelszónak tartalmaznia kell legalább egy speciális karaktert (!@#%&*._-).";
    }
    return string.Empty;
}
```

HandleUserDataUpdate():

Kezeli a felhasználói jelszó módosítását.

```
private async Task HandleUserDataUpdate()
{
    if (!string.IsNullOrEmpty(passwordErrorMessage) || !string.IsNullOrEmpty(passwordConfirmErrorMessage))
    {
        return;
    }

    var user = await UserService.GetUserByEmail(currentUserEmail);
    using (var sha256 = System.Security.Cryptography.SHA256.Create())
    {
        var hashedCurrentPassword = sha256.ComputeHash(System.Text.Encoding.UTF8.GetBytes(currentPassword));
        if (Convert.ToBase64String(hashedCurrentPassword) != Convert.ToBase64String(user.password))
        {
            notification.ShowMessage("Jelszó módosítás sikertelen: A jelenlegi jelszó helytelen.", "red");
            return;
        }

        var hashedNewPassword = sha256.ComputeHash(System.Text.Encoding.UTF8.GetBytes(newPassword));
        user.password = hashedNewPassword;
        await UserService.UpdateUser(user);
    }

    await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('userDataModal')).hide()");
    await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('userDataForm').reset()");
    notification.ShowMessage("A jelszó sikeresen módosítva.", "green");

    currentPassword = string.Empty;
    newPassword = string.Empty;
    confirmPassword = string.Empty;
    passwordErrorMessage = string.Empty;
    passwordConfirmErrorMessage = string.Empty;
}
```

DeleteAccount():

Admin funkcióként törli a megadott azonosítójú felhasználót.

```
private async Task DeleteUserId(int userId)
{
    var confirmed = await notification.ShowConfirmation("Biztosan törölni szeretné ezt a felhasználót? Ez a művelet végleges és nem vonható vissza!");

    if (confirmed)
    {
        try
        {
            var user = await UserService.GetUserById(userId);
            if (user != null)
            {
                if (user.role == "customer")
                {
                    var reviews = await ReviewService.GetReviewsByUserId(user.user_id);
                    foreach (var review in reviews)
                    {
                        await ReviewService.DeleteReview(review.review_id);
                    }
                }

                if (user.role == "owner")
                {
                    var restaurant = await RestaurantService.GetRestaurantByOwnerId(user.user_id);
                    var reviews = await ReviewService.GetReviewsByRestaurantId(restaurant.restaurant_id);
                    foreach (var review in reviews)
                    {
                        await ReviewService.DeleteReview(review.review_id);
                    }

                    var menuItems = await MenuItemService.GetMenuItemsByRestaurantId(restaurant.restaurant_id);
                    foreach (var menuItem in menuItems)
                    {
                        await MenuItemService.DeleteMenuItem(menuItem.item_id);
                    }

                    if (restaurant != null)
                    {
                        await RestaurantService.DeleteRestaurant(restaurant.restaurant_id);
                    }
                }

                await UserService.DeleteUser(userId);

                LoadAllUsers();

                notification.ShowMessage("A felhasználó sikeresen törölve.", "green");
            }
        }
        catch (Exception ex)
        {
            notification.ShowMessage($"Hiba történt a felhasználó törlése során: {ex.Message}", "red");
        }
    }
}
```

ShowEditReviewModal(Review review):

Megjeleníti az értékelés szerkesztéséhez használt modalt.

```
private void ShowEditReviewModal(Review review)
{
    editReview = review;
    JSRuntime.InvokeVoidAsync("eval", "new bootstrap.Modal(document.getElementById('editReviewModal')).show()");
}
```

HandleEditReview():

Kezeli az értékelés szerkesztését és mentését.

```
private async Task HandleEditReview()
{
    await ReviewService.UpdateReview(editReview);
    userReviews = await ReviewService.GetReviewsByUserId(currentUserId);
    await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('editReviewModal')).hide()");
    notification.ShowMessage("Az értékelés sikeresen frissítve.", "green");
}
```

LoadUserReviews():

Betölti a bejelentkezett felhasználó értékeléseit.

```
private async Task LoadUserReviews()
{
    if (userReviews == null)
    {
        var authState = await AuthenticationStateProvider.GetAuthenticationStateAsync();
        if (authState.User.Identity.IsAuthenticated)
        {
            var user = await UserService.GetUserByEmail(authState.User.Identity.Name);
            if (user != null)
            {
                currentUserId = user.user_id;
                userReviews = await ReviewService.GetReviewsByUserId(currentUserId);

                userReviews = userReviews.Where(r => r.is_deleted == false).ToList();

                foreach (var review in userReviews)
                {
                    if (!restaurantNames.ContainsKey(review.restaurant_id))
                    {
                        var restaurant = await RestaurantService.GetRestaurantById(review.restaurant_id);
                        if (restaurant != null)
                        {
                            restaurantNames[review.restaurant_id] = restaurant.name;
                        }
                    }
                }
            }
        }
    }
}
```

LoadAllUsers():

Admin funkcióként betölti az összes felhasználó adatait.

```
private async Task LoadAllUsers()
{
    allUsers = (await UserService.GetAllUsers()).Where(user => !user.is_deleted).ToList();
    StateHasChanged();
}
```

DeleteUserReview(int reviewId):

Törli a felhasználó értékelését.

```
private async Task DeleteUserReview(int reviewId)
{
    var confirmed = await notification.ShowConfirmation("Biztosan törölni szeretné ezt az értékelést? A művelet nem visszafordítható.");

    if (confirmed)
    {
        try
        {
            await ReviewService.DeleteReview(reviewId);

            userReviews = userReviews.Where(r => r.review_id != reviewId).ToList();

            notification.ShowMessage("Az értékelés sikeresen törölve.", "green");
        }
        catch (Exception ex)
        {
            notification.ShowMessage($"Hiba történt az értékelés törlése során: {ex.Message}", "red");
        }
    }
}
```

2.16. MAINLAYOUT.RAZOR

A MainLayout.razor fájl a GasztroTükör Blazor WebAssembly alkalmazás fő elrendezését határozza meg. Ez a komponens az alkalmazás alapstruktúráját biztosítja, beleértve a fejléct, navigációs menüt, tartalmi területet és lábléct. Emellett a felhasználói regisztrációs és bejelentkezési modálokat is tartalmazza.

2.16.1. HTML STRUKTÚRA

A főoldal elrendezése három fő részre oszlik:

```
<div class="page-container">
  <!-- Navigációs komponens -->
  <div>
    <NavMenu />
  </div>

  <!-- Fő tartalom terület -->
  <main class="content-wrap">
    <!-- Modális ablakok -->
    <!-- ...regisztrációs és bejelentkezési modálok... -->

    <!-- Gyermek komponensek renderelése -->
    @Body
  </main>

  <!-- Lábléc -->
  <footer class="footer-section">
    <!-- Lábléc tartalom -->
  </footer>
</div>
```

Regisztrációs modál:

```
<div class="modal fade registration-modal" id="registrationModal" tabindex="-1" aria-labelledby="registrationModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="registrationModalLabel">Regisztráció</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Bezárás"></button>
      </div>
      <div class="modal-body">
        <form id="registrationForm" @onsubmit="HandleRegistration">
          <!-- Alapvető felhasználói adatok -->

          <!-- Felhasználó típus választó -->
          <div class="mb-3">
            <label class="form-label">Felhasználó típusa</label>
            <div class="d-flex gap-3">
              <!-- Rádió gombok: vendég/tulajdonos -->
            </div>
          </div>

          <!-- Tulajdonos-specifikus mezők (alapértelmezetten rejtve) -->
          <div id="ownerFields" style="display: none;">
            <!-- Étterem adatok mezői -->
          </div>

          <!-- Regisztráció gomb -->
        </form>
      </div>
    </div>
  </div>
</div>
```

Bejelentkezési modál:

```

<div class="modal fade login-modal" id="loginModal" tabindex="-1" aria-labelledby="loginModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="loginModalLabel">Bejelentkezés</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Bezárás"></button>
      </div>
      <div class="modal-body">
        <form id="loginForm" @onsubmit="HandleLogin">
          <!-- Email és jelszó mezők -->
          <button type="submit" class="btn btn-primary w-100">Bejelentkezés</button>
        </form>
        <!-- Elfelejtett jelszó link -->
        <div class="text-center mt-3">
          <a @onclick="HandleForgotPassword" style="cursor: pointer">Elfelejtett jelszó?</a>
        </div>
      </div>
    </div>
  </div>
</div>

```

2.16.2. JAVASCRIPT INTEGRÁCIÓ

```

<script>
  // Tulajdonos mezők megjelenítése/elrejtése a választott típus alapján
  document.querySelectorAll('input[name="userType"]').forEach(radio => {
    radio.addEventListener('change', function () {
      const ownerFields = document.getElementById('ownerFields');
      ownerFields.style.display = this.value === 'owner' ? 'block' : 'none';

      // Kötelező mezők dinamikus kezelése
      const ownerInputs = ownerFields.querySelectorAll('input');
      ownerInputs.forEach(input => {
        input.required = this.value === 'owner';
      });
    });
  });

  // Jelszó láthatóság kapcsoló funkció
  function togglePassword(inputId, iconId) {
    // Jelszó mező típusának és ikonjának váltása
  }

  // Modál elrejtése
  window.hideModal = (modalId) => {
    const modal = bootstrap.Modal.getInstance(document.getElementById(modalId));
    modal.hide();
  };

  // Űrlap visszaállítása
  window.resetForm = (formId) => {
    document.getElementById(formId).reset();
  };
</script>

```

2.16.3. C# METÓDUSOK

OnInitializedAsync():

A komponens inicializálásakor fut le, és beállítja a bejelentkezett felhasználói adatokat.

```
protected override async Task OnInitializedAsync()
{
    var authState = await AuthenticationStateTask;
    var userClaims = authState.User;

    if (userClaims.Identity is not null && userClaims.Identity.IsAuthenticated)
    {
        var authUser = await UserService.GetUserByEmail(userClaims.Identity.Name);
        user = new User
        {
            email = authUser.email,
            role = authUser.role,
            user_id = authUser.user_id
        };
    }
}
```

HandleRegistration():

Feldolgozza a felhasználói regisztrációt, létrehozza a felhasználót és éttermet (ha a felhasználó tulajdonos).

```
private async Task HandleRegistration()
{
    // Email, jelszó és felhasználói típus kinyerése a DOM-ból
    var email = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('emailInput').value");
    var password = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('passwordInput').value");
    var passwordConfirm = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('passwordConfirmInput').value");
    var userType = await JSRuntime.InvokeAsync<string>("eval", "document.querySelector('input[name=\"userType\"]:checked').value");

    // Jelszó validálás
    ValidatePassword(new ChangeEventArgs { Value = password });
    ValidatePasswordConfirm(new ChangeEventArgs { Value = passwordConfirm });

    if (!string.IsNullOrEmpty(passwordErrorMessage) || !string.IsNullOrEmpty(passwordConfirmErrorMessage))
    {
        return;
    }

    // További validáció tulajdonos esetén
    if (userType == "owner")
    {
        var phone = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('phone').value");
        ValidatePhone(new ChangeEventArgs { Value = phone });

        if (!string.IsNullOrEmpty(phoneErrorMessage))
        {
            return;
        }
    }

    // Felhasználó létrehozása
    // ... (ellenőrzés, hogy a felhasználó már létezik-e)
    // ... (új felhasználó létrehozása a megfelelő szereppel)

    // Étterem létrehozása tulajdonos esetén
    if (userType == "owner")
    {
        // ... (étterem adatok kinyerése és validációja)
        // ... (étterem létrehozása)
    }

    // Értesítő email küldése
    await SendRegistrationEmail(email, userType == "owner");

    // UI visszaállítás, értesítés megjelenítése
    await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('registrationModal')).hide()");
    await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('registrationForm').reset()");
    notification.ShowMessage("A regisztráció sikeres volt. Kérjük, jelentkezzen be.", "green");
}
```


HandleLogin():

Kezeli a felhasználó bejelentkezését, ellenőrzi a hitelesítő adatokat és beállítja az autentikációs állapotot.

```
private async Task HandleLogin()
{
    // Email és jelszó kinyerése a DOM-ból
    var email = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('loginEmail').value");
    var password = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('loginPassword').value");

    // Felhasználó lekérése email alapján
    User existingUser = null;
    try
    {
        existingUser = await UserService.GetUserByEmail(email);
    }
    catch (HttpRequestException ex) when (ex.StatusCode == System.Net.HttpStatusCode.NotFound)
    {
        // Felhasználó nem található, értesítés küldése
        notification.ShowMessage("Ez az email cím még nincs regisztrálva. Kérjük, regisztráljon.", "yellow");
        await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('loginModal')).hide()");
        await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('loginForm').reset()");
        return;
    }

    // Jelszó ellenőrzése
    if (existingUser != null)
    {
        using (var sha256 = System.Security.Cryptography.SHA256.Create())
        {
            var hashedPassword = sha256.ComputeHash(System.Text.Encoding.UTF8.GetBytes(password));
            var hashedPasswordString = Convert.ToBase64String(hashedPassword);

            if (hashedPasswordString == Convert.ToBase64String(existingUser.password))
            {
                // Claim-ek létrehozása a hitelesítéshez
                var claims = new List<Claim>
                {
                    // ... (felhasználói azonosítók és szerepkörök hozzáadása)
                };

                // Felhasználó autentikációjának beállítása
                await AuthStateProviderService.NotifyUserAuthentication(principal);

                // Modál elrejtése, értesítés megjelenítése
                notification.ShowMessage($"Sikeres bejelentkezés: {email}.", "green");
                await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('loginModal')).hide()");
                await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('loginForm').reset()");
                StateHasChanged();
            }
            else
            {
                notification.ShowMessage("Helytelen email cím és/vagy jelszó.", "red");
            }
        }
    }
}
}
```

UploadThumbnail(InputFileChangeEventArgs e):

Kezeli az étterem borítóképének feltöltését, ellenőrzi a méretét és konvertálja base64 formátumba.

```
private async Task UploadThumbnail(InputFileChangeEventArgs e)
{
    var file = e.File;
    if (file != null)
    {
        // Fájl méret ellenőrzése (max 500KB)
        if (file.Size > 512000)
        {
            notification.ShowMessage("A fájl mérete nem haladhatja meg az 500 KB-ot.", "yellow");
            await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('thumbnail').value = ''");
            return;
        }

        // Fájl konvertálása base64 formátumba
        using var stream = new MemoryStream();
        await file.OpenReadStream().CopyToAsync(stream);
        thumbnailBase64 = $"data:{file.ContentType};base64,{Convert.ToBase64String(stream.ToArray())}";
    }
}
```

SendRegistrationEmail(string email, bool isOwner):

Regisztrációs értesítő email küldése a felhasználónak.

```
private async Task SendRegistrationEmail(string email, bool isOwner)
{
    await EmailService.SendRegistrationEmailAsync(email, isOwner);
}
```

HandleForgotPassword():

Kezeli az elfelejtett jelszó funkciót, új véletlenszerű jelszót generál és küldi el a felhasználónak.

```
private async Task HandleForgotPassword()
{
    // Email cím kinyerése
    var email = await JSRuntime.InvokeAsync<string>("eval", "document.getElementById('loginEmail').value");

    if (string.IsNullOrEmpty(email))
    {
        notification.ShowMessage("Kérjük, adja meg az email címét.", "yellow");
        return;
    }

    // Felhasználó ellenőrzése
    User existingUser = null;
    try
    {
        existingUser = await UserService.GetUserByEmail(email);
    }
    catch (HttpRequestException ex) when (ex.StatusCode == System.Net.HttpStatusCode.NotFound)
    {
        notification.ShowMessage("Ez az email cím még nincs regisztrálva. Kérjük, regisztráljon.", "yellow");
        return;
    }

    // Megerősítés kérése
    var confirmed = await notification.ShowConfirmation("Biztosan szeretne új jelszót igényelni?");

    if (confirmed)
    {
        // Új jelszó generálása
        var newPassword = GenerateRandomPassword();

        // Jelszó hash-elése és mentése
        using (var sha256 = System.Security.Cryptography.SHA256.Create())
        {
            var hashedPassword = sha256.ComputeHash(System.Text.Encoding.UTF8.GetBytes(newPassword));
            existingUser.password = hashedPassword;
            await UserService.UpdateUser(existingUser);
        }

        // Email küldése az új jelszóval
        await SendForgotPasswordEmail(email, newPassword);

        // Értesítés és modál elrejtése
        notification.ShowMessage("Az új jelszót elküldtük az email címére.", "green");
        await JSRuntime.InvokeVoidAsync("hideModal", "loginModal");
    }
}
```

GenerateRandomPassword():

Véletlenszerű, biztonságos jelszót generál az elfelejtett jelszó funkcióhoz.

```
private string GenerateRandomPassword()
{
    const string validChars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#%&*._-";
    var random = new Random();
    return new string(Enumerable.Repeat(validChars, 12)
        .Select(s => s[random.Next(s.Length)]).ToArray());
}
```

2.17. HOME.RAZOR

2.17.1. HTML STRUKTÚRA

Hero szekció:

A hero szekció a főoldal tetején található, és tartalmazza a keresési mezőt.

```
<section class="hero-section text-white text-center">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-12 col-md-10 col-lg-8">
        <h1 class="display-4 fw-bold mb-4">
          Ízleld meg a <span class="golden-text">kiválóságot</span>
        </h1>
        <p class="lead mb-5">Gasztronómiai kalandod itt kezdődik</p>
        <div class="search-container">
          <div class="input-group">
            <input type="text" id="searchInput" class="form-control form-control-lg py-3 px-4"
              placeholder="Keress éttermek, konyhák vagy kulcsszavak között..."
              aria-label="keresés">
            <button class="btn btn-lg px-4 py-3" type="button" id="searchButton">
              <i class="fas fa-search"></i>
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
```

- Keresési mező és gomb a felhasználók számára.
- A keresési mező valós idejű keresést tesz lehetővé az éttermek között.

Kiemelt éttermek szekció:

```
<section class="featured-restaurants-section py-3 bg-light" id="featuredSection">
  <div class="container my-4">
    <h2 class="text-center mb-4 text-warning">Kiemelt Éttermek</h2>
    <div class="row" id="featuredRestaurants">
      <@foreach (var restaurant in topRestaurants)>
        {
          <div class="col-md-4">
            <div class="card" data-restaurant-id="@restaurant.restaurant_id">
              
              <div class="card-body">
                <span class="category-badge mb-2">@restaurant.cuisine</span>
                <h5 class="card-title">@restaurant.name</h5>
                <div class="text-warning mb-2">
                  <@for (int i = 0; i < 5; i++)>
                    {
                      <i class="fa@((i < Math.Floor(averageRating) ? "s" : (i < averageRating ? "s fa-star-half-alt" : "r")) fa-star)></i>
                    }
                  <span class="rating-value ms-2">(@Math.Round(averageRating, 2))</span>
                </div>
                <p class="card-text">@restaurant.description</p>
                <NavLink href="@($"/restaurants/{restaurant.name.Replace(" ", "-").ToLower()}")" class="btn btn-custom">Bővebben...</NavLink>
              </div>
            </div>
          </div>
        }
      </div>
    </div>
  </section>
```

- A legjobbra értékelt éttermek megjelenítése.
- Az éttermekhez tartozó értékelések és vélemények száma is látható.

Éttermek szekció:

```
<section class="restaurants-section">
  <div class="container my-4">
    <h2 class="text-center mb-4">Éttermek</h2>
    <div class="row" id="regularRestaurants">
      @foreach (var restaurant in restaurants)
      {
        <div class="col-md-4">
          <div class="card" data-restaurant-id="@restaurant.restaurant_id">
            
            <div class="card-body">
              <span class="category-badge mb-2">@restaurant.cuisine</span>
              <h5 class="card-title">@restaurant.name</h5>
              <div class="text-warning mb-2">
                @for (int i = 0; i < 5; i++)
                {
                  <i class="fa@(< Math.Floor(averageRating) ? "s" : (i < averageRating ? "s fa-star-half-alt" : "r")) fa-star"></i>
                }
              <span class="rating-value ms-2">@Math.Round(averageRating, 2)</span>
            </div>
            <p class="card-text">@restaurant.description</p>
            <NavLink href="@($"/restaurants/{restaurant.name.Replace(" ", "-").ToLower()}")" class="btn btn-custom">Bővebben...</NavLink>
          </div>
        </div>
      }
    </div>
  </div>
</section>
```

- Az összes elérhető étterem megjelenítése.
- Az éttermekhez tartozó értékelések és vélemények száma is látható.

2.17.2. JAVASCRIPT INTEGRÁCIÓ

initializeAdminSearchFunctions():

Kezeli a keresési funkciókat, és a valós idejű keresést.

```
function initializeHomeSearchFunctions() {
  const searchButton = document.getElementById('searchButton');
  if (searchButton) {
    searchButton.addEventListener('click', function () {
      const searchTerm = document.getElementById('searchInput').value.toLowerCase();
      const featuredCards = document.querySelectorAll('#featuredRestaurants .card');
      const regularCards = document.querySelectorAll('#regularRestaurants .card');
      const featuredSection = document.getElementById('featuredSection');
      const noResultsMessage = document.getElementById('noResultsMessage');

      let featuredVisible = false;
      let regularVisible = false;

      const featuredRestaurantIds = new Set();

      if (searchTerm === '') {
        featuredCards.forEach(card => {
          card.closest('.col-md-4').style.display = '';
        });
        regularCards.forEach(card => {
          card.closest('.col-md-4').style.display = '';
        });
        if (noResultsMessage) {
          noResultsMessage.style.display = 'none';
        }
        if (featuredSection) {
          featuredSection.style.display = '';
        }
        return;
      }

      featuredCards.forEach(card => {
        const title = card.querySelector('.card-title').textContent.toLowerCase();
        const category = card.querySelector('.category-badge').textContent.toLowerCase();
        const description = card.querySelector('.card-text').textContent.toLowerCase();
        const restaurantId = card.getAttribute('data-restaurant-id');

        if (restaurantId) {
          featuredRestaurantIds.add(restaurantId);
        }

        if (title.includes(searchTerm) || category.includes(searchTerm) || description.includes(searchTerm)) {
          card.closest('.col-md-4').style.display = '';
          featuredVisible = true;
        } else {
          card.closest('.col-md-4').style.display = 'none';
        }
      });
    });
  }
}
```

```
regularCards.forEach(card => {
  const title = card.querySelector('.card-title').textContent.toLowerCase();
  const category = card.querySelector('.category-badge').textContent.toLowerCase();
  const description = card.querySelector('.card-text').textContent.toLowerCase();
  const restaurantId = card.getAttribute('data-restaurant-id');

  if (restaurantId && featuredRestaurantIds.has(restaurantId) && featuredVisible) {
    card.closest('.col-md-4').style.display = 'none';
    return;
  }

  if (title.includes(searchTerm) || category.includes(searchTerm) || description.includes(searchTerm)) {
    card.closest('.col-md-4').style.display = '';
    regularVisible = true;
  } else {
    card.closest('.col-md-4').style.display = 'none';
  }
});

if (!featuredVisible && !regularVisible) {
  if (!noResultsMessage) {
    const messageElement = document.createElement('div');
    messageElement.id = 'noResultsMessage';
    messageElement.className = 'col-12 text-center';
    messageElement.innerHTML = '<p><h4>Nincs találat a megadott keresési feltételre.</h4></p>';
    const restaurantsRow = document.querySelector('.restaurants-section .row');
    if (restaurantsRow) {
      restaurantsRow.appendChild(messageElement);
    }
  } else {
    noResultsMessage.style.display = 'block';
  }
} else if (noResultsMessage) {
  noResultsMessage.style.display = 'none';
}

if (featuredVisible && featuredSection) {
  featuredSection.style.display = '';
} else if (featuredSection) {
  featuredSection.style.display = 'none';
}
});
}
```

- Keresési feltételek alapján szűri a jóváhagyott és jóváhagyásra váró éttermeket.
- Üzenetet jelenít meg, ha nincs találat.
- Az input mező változására automatikusan meghívja a keresési funkciót.

2.17.3. C# METÓDUSOK

OnInitializedAsync:

Éttermek és értékelések betöltése a backend szolgáltatásokból, és kiemelt éttermek kiválasztása az értékelések alapján.

```
protected override async Task OnInitializedAsync()
{
    try
    {
        isLoading = true;
        StateHasChanged();

        restaurants = (await RestaurantService.GetAllRestaurants())
            .Where(r => r.is_deleted == false && r.is_approved == true)
            .ToList();

        foreach (var restaurant in restaurants)
        {
            var restaurantReviews = (await ReviewService.GetReviewsByRestaurantId(restaurant.restaurant_id))
                .Where(r => r.is_deleted == false)
                .ToList();

            restaurantReviewCounts[restaurant.restaurant_id] = restaurantReviews.Count;

            if (restaurantReviews.Any())
            {
                restaurantRatings[restaurant.restaurant_id] = restaurantReviews.Average(r => r.avg_rating);
            }
            else
            {
                restaurantRatings[restaurant.restaurant_id] = 0;
            }
        }

        topRestaurants = restaurants
            .Where(r => restaurantRatings.ContainsKey(r.restaurant_id))
            .OrderByDescending(r => restaurantRatings[r.restaurant_id])
            .Take(3)
            .ToList();
    }
    catch (Exception ex)
    {
        errorMessage = $"Az adatok betöltése sikertelen: {ex.Message}";
    }
    finally
    {
        isLoading = false;
        await JSRuntime.InvokeVoidAsync("initializeHomeSearchFunctions");
        StateHasChanged();
    }
}
```

2.18. SITE.RAZOR

A Site.razor komponens egy étterem részletes adatlapját megjelenítő Blazor komponens a GasztroTükör alkalmazásban. A komponens megjeleníti az étterem alapadatait, menüjét kategóriákra bontva, valamint a vendégek által adott értékeléseket. Bejelentkezett felhasználók számára lehetőséget biztosít új értékelések hozzáadására.

2.18.1. HTML STRUKTÚRA

Étterem fejléc szekció

Az étterem alapadatainak megjelenítése:

```
<section class="restaurant-header text-white text-center py-5"
  style="background-image: url('@restaurant.thumbnail');">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-12 col-md-10">
        <div class="info-bg">
          <h1>@restaurant.name</h1>
          <p class="category-badge">@restaurant.cuisine</p>
          <!-- További adatok... -->
        </div>
      </div>
    </div>
  </div>
</section>
```

Menü kategóriák szekció

Az étterem menüjének megjelenítése kategóriákra bontva:

```
<section class="menu-section py-5">
  <div class="container">
    @foreach (var category in categories)
    {
      var categoryItems = menuItems.Where(item => item.category_id == category.category_id
        && item.is_available && item.is_approved && item.is_deleted == false).ToList();

      if (categoryItems.Any())
      {
        <div class="menu-category mb-5">
          <h2 class="text-center mb-4">
            @(char.ToUpper(category.category_name[0]) + category.category_name.Substring(1)
              + (category.category_name == "ital" ? "ok" : "ek"))
          </h2>
          <!-- Kategória elemek megjelenítése... -->
        </div>
      }
    }
  </div>
</section>
```

Vendégvélemények szekció

Az étteremhez tartozó értékelések megjelenítése:

```
<section class="reviews-section py-5 bg-light">
  <div class="container">
    <h2 class="text-center mb-5">
      Vendégeink írták
      <span class="badge" style="background-color: #d3af27">@(reviews.Where(r => r.is_deleted == false).Count())</span>
    </h2>
  </div>
  <div class="container" style="max-height: 520px; overflow-y: auto; overflow-x: hidden;">
    <div class="row g-4" id="reviewsContainer">
      @if (reviews != null && reviews.Any())
      {
        <!-- Értékelések megjelenítése... -->
      }
      else
      {
        <h2 class="text-danger" style="text-align: center;">Még nincs értékelés ehhez az étteremhez!</h2>
      }
    </div>
  </div>
</section>
```

Értékelés modal

Csak bejelentkezett és "customer" szerepkörrel rendelkező felhasználók számára elérhető:

```
<AuthorizeView Roles="customer">
  <Authorized>
    <div class="modal fade" id="reviewModal" tabindex="-1" aria-labelledby="reviewModalLabel" aria-hidden="true">
      <!-- Modal tartalom... -->
    </div>
  </Authorized>
</AuthorizeView>
```

Lebegő gomb

Navigációs segéd az oldalon való mozgáshoz:

```
<div id="floatingButton" class="floating-reviews" @onclick="HandleFloatingButtonClick">
  Ugrás a véleményekhez
</div>
```


2.18.2. JAVASCRIPT INTEGRÁCIÓ

highlightStars(rating, category):

A megfelelő kategória (étel, szolgáltatás, tisztaság, hangulat) csillagjait aktívvá teszi az adott értékelésig.

```
// Csillagok kezelése
document.querySelectorAll('.rating-star').forEach(star => {
  // Egérrel kijelölés
  star.addEventListener('mouseover', function () {
    const rating = this.dataset.rating;
    const category = this.dataset.category;
    highlightStars(rating, category);
  });

  // További eseménykezelők...
});

// Csillagok frissítése a kijelölt érték szerint
function highlightStars(rating, category) {
  const categoryStars = document.querySelectorAll('.rating-star[data-category="'+category+'"]');
  categoryStars.forEach(s => {
    if (s.dataset.rating <= rating) {
      s.classList.add('active');
    } else {
      s.classList.remove('active');
    }
  });
}
```

showReviewModal():

Megjeleníti az étterem értékelő modális ablakát, előtte alaphelyzetbe állítja az űrlapot.

```
function showReviewModal() {
  const reviewModal = new bootstrap.Modal(document.getElementById('reviewModal'));
  resetReviewForm();
  reviewModal.show();
}
```

closeModal(modalId):

Bezárja a megadott azonosítójú Bootstrap modális ablakot.

```
function closeModal(modalId) {
  const modal = bootstrap.Modal.getInstance(document.getElementById(modalId));
  if (modal) {
    modal.hide();
  }
}
```

updateFloatingButtonState():

Frissíti a lebegő navigációs gomb állapotát és szövegét attól függően, hogy az értékelések láthatóak-e a képernyőn.

```
function updateFloatingButtonState() {
  const reviewsSection = document.getElementById('reviewsContainer');
  const floatingButton = document.getElementById('floatingButton');

  if (reviewsSection && floatingButton) {
    const reviewsTop = reviewsSection.getBoundingClientRect().top;
    const reviewsBottom = reviewsSection.getBoundingClientRect().bottom;
    const windowHeight = window.innerHeight;

    if (reviewsTop < windowHeight && reviewsBottom > 0) {
      floatingButton.textContent = 'Ugrás az oldal tetejére';
      floatingButton.setAttribute('data-action', 'top');
    } else {
      floatingButton.textContent = 'Ugrás a véleményekhez';
      floatingButton.setAttribute('data-action', 'reviews');
    }
  }
}
```

2.18.3. C# METÓDUSOK**OnInitializedAsync():**

Betölti az étterem részleteit, menüelemeit, kategóriáit és értékeléseit az URL-ben megadott név alapján, majd ellenőrzi a felhasználó bejelentkezési állapotát.

```
protected override async Task OnInitializedAsync()
{
  try
  {
    var restaurantName = name.Replace("-", " ");
    restaurant = await RestaurantService.GetRestaurantByName(restaurantName);
    // Adatok betöltése...
  }
  catch (Exception ex)
  {
    NavigationManager.NavigateTo("/");
  }
}
```

OpenReviewModal():

Megnyitja az értékelési ablakot, miután ellenőrizte a felhasználó jogosultságait és hogy nem értékelte-e már az éttermet.

```
private async Task OpenReviewModal()
{
    var authState = await AuthenticationStateTask;
    var userClaims = authState.User;

    if (userClaims.Identity?.IsAuthenticated != true)
    {
        notification.ShowMessage("Kérjük, jelentkezzen be az értékeléshez!", "yellow");
        return;
    }

    // További ellenőrzések...

    await JSRuntime.InvokeVoidAsync("showReviewModal");
}
```

HandleSubmit():

Feldolgozza és elmenti az új értékelés beküldését, miután ellenőrizte annak érvényességét.

```
private async Task HandleSubmit()
{
    try
    {
        // Validáció
        if (newReview.food_rating < 1 || newReview.service_rating < 1 ||
            newReview.cleanliness_rating < 1 || newReview.atmosphere_rating < 1)
        {
            notification.ShowMessage("Kérjük, minden kategóriában jelöljön ki legalább egy csillagot!", "yellow");
            return;
        }

        // Értékelés mentése és frissítések
        newReview.user_id = user.user_id;
        newReview.restaurant_id = restaurant.restaurant_id;
        newReview.created_at = DateTime.Now;

        var success = await ReviewService.CreateReview(newReview);
        // Sikeres mentés kezelése...
    }
    catch (Exception ex)
    {
        notification.ShowMessage("Hiba történt az értékelés mentése közben.", "red");
    }
}
```

SetRating(int rating, string category):

Beállítja a megfelelő kategóriához (étel, szolgáltatás, tisztaság, hangulat) tartozó csillagértékelést.

```
private void SetRating(int rating, string category)
{
    switch (category)
    {
        case "food":
            newReview.food_rating = rating;
            break;
        case "service":
            newReview.service_rating = rating;
            break;
        case "cleanliness":
            newReview.cleanliness_rating = rating;
            break;
        case "atmosphere":
            newReview.atmosphere_rating = rating;
            break;
    }
}
```

AverageRating():

Kiszámítja az étterem átlagos értékelését a négy kategória (étel, szolgáltatás, tisztaság, hangulat) alapján.

```
private void AverageRating()
{
    if (reviews.Any())
    {
        averageRating = reviews.Average(r =>
            (r.food_rating + r.service_rating + r.cleanliness_rating + r.atmosphere_rating) / 4.0m);
        StateHasChanged();
    }
}
```

2.19. OWNER.RAZOR

Az Owner.razor komponens a Blazor WebAssembly alkalmazás része, amely étterem tulajdonosok számára biztosít kezelőfelületet. A komponens segítségével a tulajdonosok megtekinthetik és módosíthatják éttermük adatait, kezelhetik a menüt, és hozzáférhetnek a vendégértékelésekhez.

2.19.1. HTML STRUKTÚRA

Étterem fejléc szekció

```
<body>
  <section class="restaurant-header text-white text-center py-5">
    <div class="container content">
      <div class="row justify-content-center" data-bs-toggle="modal" data-bs-target="#editDetailsModal">
        <div class="col-12 col-md-10">
          <div class="info-bg">
            <!-- Étterem alap információk -->
            <h1>@restaurant.name</h1>
            <p class="category-badge">@restaurant.cuisine</p>
            <p>@restaurant.description</p>
            <p>Minden nap @restaurant.opening_time - @restaurant.closing_time</p>
            <p>@restaurant.address</p>
            <p><i class="fas fa-envelope"></i> @owner.email</p>
            <p><i class="fas fa-phone"></i> @restaurant.phone</p>
            <!-- Értékelés csillagok -->
            <div class="rating mb-3">...</div>
            <!-- Értékelések száma -->
            <div class="d-flex align-items-center justify-content-center mt-2">...</div>
          </div>
        </div>
      </div>
    </div>
  </section>
```

Menü kategóriák szekció

```
<section class="menu-section py-5">
  <div class="container">
    <!-- Előételek -->
    <div class="menu-category mb-5">
      <h2 class="text-center mb-4">Előételek</h2>
      <div class="row" id="appetizers">
        <!-- Előételek listája -->
        @foreach (var item in appetizers.Where(item => item.is_deleted == false))
        {
          <div class="col-md-6 mb-4">
            <div class="menu-item d-flex @(item.is_approved == false || item.is_available == false ? "border border-danger" : "") @onClick="() => EditDish(item)">
              <!-- Étel képe és adatai -->
            </div>
          </div>
        }
        <!-- Új előétel hozzáadása gomb -->
        <div class="col-md-6 mb-4">
          <div class="card add-card text-center @onClick="() => PrepareNewDish("appetizers")" data-bs-toggle="modal" data-bs-target="#addDishModal">
            <!-- Tartalom -->
          </div>
        </div>
      </div>
    </div>
    <!-- Főételek -->
    <div class="menu-category mb-5">...</div>
    <!-- Desszertok -->
    <div class="menu-category mb-5">...</div>
    <!-- Italok -->
    <div class="menu-category mb-5">...</div>
  </div>
</section>
```

Vendégvélemények szekció

```
<section class="reviews-section py-5 bg-light">
  <div class="container">
    <h2 class="text-center mb-5">
      Vendégeink írták
      <span class="badge" style="background-color: #d3af27">@(reviews.Where(r => r.is_deleted == false).Count())</span>
    </h2>
    @if (reviews != null && reviews.Any())
    {
      <div class="table-responsive" style="max-height: 400px; overflow-y: auto;">
        <table class="table table-striped">
          <!-- Táblázat fejléc -->
          <thead class="table-dark sticky-top" style="text-align: center;">...</thead>
          <!-- Táblázat tartalom -->
          <tbody>...</tbody>
        </table>
      </div>
      <button class="btn btn-success mt-3 d-block mx-auto" @onclick="ExportToExcel">Exportálás</button>
    }
    else
    {
      <h2 class="text-danger" style="text-align: center;">Még nincs értékelés ehhez az étteremhez!</h2>
    }
  </div>
</section>
```

Új étel hozzáadása modal

```
<div class="modal fade login-modal" id="addDishModal" tabindex="-1" aria-labelledby="addDishModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">...</div>
      <div class="modal-body">
        <form id="addDishForm">
          <!-- Étél név, leírás, ár és kép mezők -->
        </form>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" @onclick="SaveDish">Mentés</button>
      </div>
    </div>
  </div>
</div>
```

Étel szerkesztése modal

```
<div class="modal fade login-modal" id="editDishModal" tabindex="-1" aria-labelledby="editDishModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">...</div>
      <div class="modal-body">
        <form id="editDishForm">
          <!-- Étél adatok szerkesztése mezők -->
          <!-- Státusz (elérhető/nem elérhető) radio gombok -->
        </form>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" @onclick="SaveEditedDish">Mentés</button>
      </div>
    </div>
  </div>
</div>
```

Étterem adatainak szerkesztése modal

```
<div class="modal fade login-modal" id="editDetailsModal" tabindex="-1" aria-labelledby="editDetailsModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">...</div>
      <div class="modal-body">
        <form>
          <!-- Étterem adatok mezők (név, telefon, konyha típusa, leírás, borítókép, cím, nyitvatartás) -->
        </form>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" @onclick="SaveDetails">Mentés</button>
      </div>
    </div>
  </div>
</div>
```

2.19.2. JAVASCRIPT INTEGRÁCIÓ

downloadFile(filename, content):

Excel fájl letöltését indítja el a megadott névvel és base64-kódolt tartalommal az értékelések exportálásakor.

```
function downloadFile(filename, content) {  
    var link = document.createElement('a');  
    link.download = filename;  
    link.href = "data:application/vnd.openxmlformats-officedocument.spreadsheetml.sheet;base64," + content;  
    document.body.appendChild(link);  
    link.click();  
    document.body.removeChild(link);  
}
```

2.19.3. C# METÓDUSOK

OnInitializedAsync():

Betölti az étterem tulajdonos adatait, az étterem részleteit, az ételeket kategóriák szerint és a vendégértékeléseket.

```
protected override async Task OnInitializedAsync()  
{  
    var authState = await AuthenticationStateTask;  
    var userClaims = authState.User;  
  
    if (userClaims.Identity is not null && userClaims.Identity.IsAuthenticated)  
    {  
        // Felhasználó betöltése  
        var authUser = await UserService.GetUserByEmail(userClaims.Identity.Name);  
        if (authUser != null && authUser.role == "owner")  
        {  
            user = new User { /* ... */ };  
  
            // Étterem és kapcsolódó adatok betöltése  
            restaurant = await RestaurantService.GetRestaurantByOwnerId(user.user_id);  
            if (restaurant != null)  
            {  
                // További adatok betöltése...  
            }  
        }  
    }  
}
```

PrepareNewDish(string category):

Előkészít egy új, üres menüelemet a megadott kategóriában való hozzáadásra.

```
private void PrepareNewDish(string category)
{
    currentDishCategory = category;
    editMenuItem = new MenuItem
    {
        restaurant_id = restaurant.restaurant_id,
        is_available = true,
        is_approved = false,
        is_deleted = false
    };
    thumbnailBase64 = string.Empty;
}
```

SaveDish():

Elmenti az új menüelemet a megfelelő kategória-azonosítóval és a feltöltött képpel, majd frissíti a felületet.

```
private async Task SaveDish()
{
    try
    {
        editMenuItem.category_id = currentDishCategory switch
        {
            "appetizers" => 1,
            "mainCourses" => 2,
            "desserts" => 3,
            "drinks" => 4
        };
        editMenuItem.thumbnail = thumbnailBase64;
        editMenuItem.created_at = DateTime.Now;

        await MenuItemService.CreateMenuItem(editMenuItem);
        await OnInitializedAsync();

        thumbnailBase64 = string.Empty;
        editMenuItem = new MenuItem();

        await JSRuntime.InvokeVoidAsync("eval", "document.getElementById('addDishForm').reset()");
        await JSRuntime.InvokeVoidAsync("closeModal", "addDishModal");
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt az étel hozzáadása során: {ex.Message}", "red");
    }
}
```


EditDish(MenuItem item):

Betölti a kiválasztott menüelem adatait szerkesztésre és megjeleníti a szerkesztő modális ablakot.

```
private void EditDish(MenuItem item)
{
    editMenuItem = new MenuItem
    {
        item_id = item.item_id,
        restaurant_id = item.restaurant_id,
        category_id = item.category_id,
        name = item.name,
        description = item.description,
        price = item.price,
        thumbnail = item.thumbnail,
        is_available = item.is_available,
        is_approved = item.is_approved,
        is_deleted = item.is_deleted,
        created_at = item.created_at
    };
    editThumbnailBase64 = item.thumbnail;

    JSRuntime.InvokeVoidAsync("eval", "new bootstrap.Modal(document.getElementById('editDishModal')).show();");
}
```

SaveDetails():

Menti az étterem módosított adatait, beleértve a nevet, telefonszámot, nyitvatartást és egyéb részleteket.

```
private async Task SaveDetails()
{
    try
    {
        if (!string.IsNullOrEmpty(phoneErrorMessage))
        {
            return;
        }

        if (!string.IsNullOrEmpty(editThumbnailBase64))
        {
            restaurant.thumbnail = editThumbnailBase64;
        }

        await RestaurantService.UpdateRestaurant(restaurant);

        editThumbnailBase64 = string.Empty;

        await JSRuntime.InvokeVoidAsync("closeModal", "editDetailsModal");
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt az étterem frissítése során: {ex.Message}", "red");
    }
}
```

ValidatePhone(ChangeEventArgs e):

Ellenőrzi, hogy a megadott telefonszám megfelel-e a magyar formátumnak (+36xxxxxxxxx).

```
private void ValidatePhone(ChangeEventArgs e)
{
    var phone = e?.Value?.ToString() ?? string.Empty;
    var regex = new System.Text.RegularExpressions.Regex(@"^\+36\d{9}$");
    if (!regex.IsMatch(phone))
    {
        phoneErrorMessage = "Nem megfelelő telefonszám formátum. A helyes formátum: +36123456789";
    }
    else
    {
        phoneErrorMessage = string.Empty;
    }
}
```

ExportToExcel():

Létrehoz egy Excel fájlt az étterem értékeléseivel és letölthetővé teszi a felhasználó számára.

```
private async Task ExportToExcel()
{
    try
    {
        ExcelPackage.LicenseContext = LicenseContext.NonCommercial;

        using var package = new ExcelPackage();
        var worksheet = package.Workbook.Worksheets.Add("vélemények");

        // Fejléc beállítása
        worksheet.Cells[1, 1].Value = "Email";
        // ... további fejlécek
        worksheet.Cells[1, 8].Value = "Vélemény";

        // Adatok feltöltése
        for (int i = 0; i < reviews.Count; i++)
        {
            var review = reviews[i];
            worksheet.Cells[i + 2, 1].Value = reviewUsers[review.user_id].email;
            // ... további adatok feltöltése
        }

        // Fájl letöltése
        var stream = new MemoryStream();
        package.SaveAs(stream);
        var content = stream.ToArray();
        var base64 = Convert.ToBase64String(content);

        var fileName = $"{RemoveAccents(restaurant.name.ToLower().Replace(" ", "-"))}_ertekelesek.xlsx";
        await JSRuntime.InvokeVoidAsync("downloadFile", fileName, base64);
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt az exportálás során: {ex.Message}", "red");
    }
}
```

RemoveAccents(string input):

Eltávolítja a magyar ékezeteket az éttermek nevéből, hogy fájlnevként használható legyen.

```
private string RemoveAccents(string input)
{
    var replacements = new Dictionary<char, char>
    {
        { 'á', 'a' }, { 'é', 'e' }, { 'í', 'i' }, { 'ó', 'o' }, { 'ö', 'o' },
        { 'ő', 'o' }, { 'ú', 'u' }, { 'ü', 'u' }, { 'ű', 'u' }
    };

    var result = new StringBuilder(input.Length);
    foreach (var c in input)
    {
        result.Append(replacements.ContainsKey(c) ? replacements[c] : c);
    }

    return result.ToString();
}
```

2.20. ADMINISTRATOR.RAZOR

Az Administrator.razor oldal az adminisztrátorok számára biztosít felületet az éttermek kezelésére, beleértve a jóváhagyásra váró és már jóváhagyott éttermek listázását, keresését és szerkesztését.

2.20.1. HTML STRUKTÚRA**Hero szekció:**

Az oldal tetején található keresőmezőt és címet tartalmazó szekció.

```
<section class="hero-section text-white text-center">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-12 col-md-10 col-lg-8">
        <h1 class="display-4 fw-bold mb-4 golden-text">Adminisztrátori felület</h1>
        <div class="search-container">
          <div class="input-group">
            <input type="text" id="searchInput" class="form-control form-control-lg py-3 px-4" placeholder="Keress éttermek, konyhák vagy kulcsszavak között..." aria-label="keresés">
            <button class="btn btn-lg px-4 py-3" type="button" id="searchButton">
              <i class="fas fa-search"></i>
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
```

Jóváhagyásra váró éttermek szekció:

Az éttermek adatait (név, konyha típusa, leírás, értékelés) tartalmazó kártyák.

```
<section class="restaurants-section pending-section py-3">
  <div class="container my-4">
    <h2 class="text-center text-danger mb-4">Jóváhagyásra váró éttermek</h2>
    @if (restaurants != null && restaurants.Any(r => r.is_approved == false))
    {
      <div class="row">
        @foreach (var restaurant in restaurants.Where(r => r.is_approved == false))
        {
          <div class="col-md-4">
            <div class="card" @onclick="() => EditRestaurant(restaurant)">
              
              <div class="card-body">
                <span class="category-badge mb-2">@restaurant.cuisine</span>
                <h5 class="card-title">@restaurant.name</h5>
                <div class="text-warning mb-2">
                  <!-- Értékelés csillagok -->
                </div>
                <p class="card-text">@restaurant.description</p>
                <div @onclick:stopPropagation>
                  <NavLink href="@($"{"/admin/restaurants/{restaurant.name.Replace(" ", "-").ToLower()}")" class="btn btn-custom">Bővebben...</NavLink>
                </div>
              </div>
            </div>
          </div>
        }
      </div>
    }
    else
    {
      <div class="text-center my-4">
        <h4>Nincs jóváhagyásra váró étterem.</h4>
      </div>
    }
  </div>
</section>
```

Jóváhagyott éttermek szekció:

Azokat az éttermeket jeleníti meg, amelyek már jóvá lettek hagyva.

```
<section class="restaurants-section approved-section">
  <div class="container my-4">
    <h2 class="text-center text-success mb-4">Jóváhagyott éttermek</h2>
    <div class="row">
      @if (restaurants != null && restaurants.Any(r => r.is_approved == true))
      {
        @foreach (var restaurant in restaurants.Where(r => r.is_approved == true))
        {
          <div class="col-md-4">
            <div class="card" @onclick="() => EditRestaurant(restaurant)">
              
              <div class="card-body">
                <span class="category-badge mb-2">@restaurant.cuisine</span>
                <h5 class="card-title">@restaurant.name</h5>
                <div class="text-warning mb-2">
                  <!-- Értékelés csillagok -->
                </div>
                <div class="d-flex align-items-center mb-2">
                  <i class="fas fa-comment-alt me-1 text-muted"></i>
                  <span class="small text-muted">
                    @(restaurantReviewCounts.ContainsKey(restaurant.restaurant_id) ? restaurantReviewCounts[restaurant.restaurant_id] : 0) vélemény
                  </span>
                </div>
                <p class="card-text">@restaurant.description</p>
                <div @onclick:stopPropagation>
                  <NavLink href="@($"{"/admin/restaurants/{restaurant.name.Replace(" ", "-").ToLower()}")" class="btn btn-custom">Bővebben...</NavLink>
                </div>
              </div>
            </div>
          </div>
        }
      </div>
    }
    else
    {
      <div class="text-center my-4">
        <h4>Nincs jóváhagyott étterem.</h4>
      </div>
    }
  </div>
</section>
```

Étterem szerkesztése modal:

Modális ablak az étterem adatainak szerkesztésére.

```
<div class="modal fade login-modal" id="editRestaurantModal" tabindex="-1" aria-labelledby="editRestaurantModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="editRestaurantModalLabel">Étterem szerkesztése</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Bezárás"></button>
      </div>
      <div class="modal-body">
        <form id="editRestaurantForm">
          <!-- Űrlap mezők -->
        </form>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-primary" @onclick="SaveEditedRestaurant">Mentés</button>
      </div>
    </div>
  </div>
</div>
```

2.20.2. JAVASCRIPT INTEGRÁCIÓ

initializeAdminSearchFunctions():

Kezeli a keresési funkciókat, és a valós idejű keresést.

```
function initializeAdminSearchFunctions() {
  const searchButton = document.getElementById('searchButton');
  if (searchButton) {
    searchButton.addEventListener('click', function () {
      const searchTerm = document.getElementById('searchInput').value.toLowerCase();
      const approvedCards = document.querySelectorAll('.approved-section .card');
      const pendingCards = document.querySelectorAll('.pending-section .card');
      const pendingNoResults = document.getElementById('pendingNoResultsMessage');
      const approvedNoResults = document.getElementById('approvedNoResultsMessage');

      let pendingVisible = false;
      let approvedVisible = false;

      // Szűrés a keresési feltételek alapján
      pendingCards.forEach(card => {
        const title = card.querySelector('.card-title').textContent.toLowerCase();
        const category = card.querySelector('.category-badge').textContent.toLowerCase();
        const description = card.querySelector('.card-text').textContent.toLowerCase();

        if (title.includes(searchTerm) || category.includes(searchTerm) || description.includes(searchTerm)) {
          card.closest('.col-md-4').style.display = '';
          pendingVisible = true;
        } else {
          card.closest('.col-md-4').style.display = 'none';
        }
      });

      approvedCards.forEach(card => {
        const title = card.querySelector('.card-title').textContent.toLowerCase();
        const category = card.querySelector('.category-badge').textContent.toLowerCase();
        const description = card.querySelector('.card-text').textContent.toLowerCase();

        if (title.includes(searchTerm) || category.includes(searchTerm) || description.includes(searchTerm)) {
          card.closest('.col-md-4').style.display = '';
          approvedVisible = true;
        } else {
          card.closest('.col-md-4').style.display = 'none';
        }
      });

      // Üzenetek kezelése
      if (!pendingVisible && pendingCards.length > 0) {
        if (pendingNoResults) pendingNoResults.style.display = 'block';
      } else if (pendingNoResults) {
        pendingNoResults.style.display = 'none';
      }

      if (!approvedVisible && approvedCards.length > 0) {
        if (approvedNoResults) approvedNoResults.style.display = 'block';
      } else if (approvedNoResults) {
        approvedNoResults.style.display = 'none';
      }
    });
  }

  const searchInput = document.getElementById('searchInput');
  if (searchInput) {
    searchInput.addEventListener('input', function () {
      const searchButton = document.getElementById('searchButton');
      if (searchButton) {
        searchButton.click();
      }
    });
  }
}
```

2.20.3. C# METÓDUSOK

OnInitializedAsync():

Betölti az összes étterem adatait, beleértve az értékeléseket és vélemények számát.

```
protected override async Task OnInitializedAsync()
{
    try
    {
        isLoading = true;
        StateHasChanged();

        var authState = await AuthenticationStateTask;
        var userClaims = authState.User;

        if (userClaims.Identity is not null && userClaims.Identity.IsAuthenticated)
        {
            var authUser = await UserService.GetUserByEmail(userClaims.Identity.Name);
            if (authUser != null && authUser.role == "admin")
            {
                user = new User
                {
                    email = authUser.email,
                    role = authUser.role,
                    user_id = authUser.user_id
                };

                restaurants = (await RestaurantService.GetAllRestaurants()).Where(r => r.is_deleted == false).ToList();
                foreach (var restaurant in restaurants)
                {
                    var restaurantReviews = (await ReviewService.GetReviewsByRestaurantId(restaurant.restaurant_id)).Where(r => r.is_deleted == false).ToList();

                    restaurantReviewCounts[restaurant.restaurant_id] = restaurantReviews.Count;

                    if (restaurantReviews.Any())
                    {
                        restaurantRatings[restaurant.restaurant_id] = restaurantReviews.Average(r => r.avg_rating);
                    }
                    else
                    {
                        restaurantRatings[restaurant.restaurant_id] = 0;
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        errorMessage = $"Az adatok betöltése sikertelen: {ex.Message}";
    }
    finally
    {
        isLoading = false;
        await JSRuntime.InvokeVoidAsync("initializeAdminSearchFunctions");
        StateHasChanged();
    }
}
```

EditRestaurant(Restaurant restaurant):

Betölti a kiválasztott étterem adatait szerkesztésre, és megnyitja a modális ablakot.

```
private void EditRestaurant(Restaurant restaurant)
{
    editRestaurant = new Restaurant
    {
        restaurant_id = restaurant.restaurant_id,
        owner_id = restaurant.owner_id,
        name = restaurant.name,
        cuisine = restaurant.cuisine,
        phone = restaurant.phone,
        address = restaurant.address,
        opening_time = restaurant.opening_time,
        closing_time = restaurant.closing_time,
        thumbnail = restaurant.thumbnail,
        description = restaurant.description,
        is_approved = restaurant.is_approved,
        is_deleted = restaurant.is_deleted,
        created_at = restaurant.created_at
    };
    editThumbnailBase64 = restaurant.thumbnail;

    selectedImagePath = "assets/images/image_not_available1.94c0c57d.png";

    JSRuntime.InvokeVoidAsync("eval", "new bootstrap.Modal(document.getElementById('editRestaurantModal')).show();");
}
```

SaveEditedRestaurant():

```
private async Task SaveEditedRestaurant()
{
    try
    {
        if (!string.IsNullOrEmpty(editThumbnailBase64))
        {
            editRestaurant.thumbnail = editThumbnailBase64;
        }

        if (!string.IsNullOrEmpty(phoneErrorMessage))
        {
            return;
        }

        await RestaurantService.UpdateRestaurant(editRestaurant);

        restaurants = (await RestaurantService.GetAllRestaurants()).Where(r => r.is_deleted == false).ToList();

        await JSRuntime.InvokeVoidAsync("eval", "bootstrap.Modal.getInstance(document.getElementById('editRestaurantModal')).hide();");
        StateHasChanged();
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt az étterem frissítése során: {ex.Message}", "red");
    }
}
```

ConvertSelectedImageToBase64():

```
private async Task ConvertSelectedImageToBase64()
{
    try
    {
        var imageUri = new Uri(NavigationManager.BaseUri + selectedImagePath);

        using var httpClient = new HttpClient();
        var response = await httpClient.GetAsync(imageUri);

        if (!response.IsSuccessStatusCode)
        {
            notification.ShowMessage($"Kép betöltése sikertelen: {response.StatusCode}", "red");
            return;
        }

        var imageBytes = await response.Content.ReadAsByteArrayAsync();

        editThumbnailBase64 = $"data:image/png;base64,{Convert.ToBase64String(imageBytes)}";

        notification.ShowMessage("A kép sikeresen betöltve!", "green");
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt a kép betöltése során: {ex.Message}", "red");
    }
}
```

2.21. VERIFICATION.RAZOR

Az Verification.razor oldal az adminisztrátorok számára biztosít felületet egy adott étterem adatainak, termékeinek és értékeléseinek kezelésére, beleértve a jóváhagyásra váró és már jóváhagyott termékek listázását, szerkesztését és értékelések törlését.

2.21.1. HTML STRUKTÚRA

Éttermek fejléc szekció:

Az oldal tetején található az étterem alapvető információit megjelenítő szekció.

```
<section class="restaurant-header text-white text-center py-5" style="background-image: url('@restaurant.thumbnail');">
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-12 col-md-10">
        <div class="info-bg">
          <h1>@restaurant.name</h1>
          <p class="category-badge">@restaurant.cuisine</p>
          <p>@restaurant.description</p>
          <p>Minden nap @restaurant.opening_time - @restaurant.closing_time</p>
          <p>@restaurant.address</p>
          <p><i class="fas fa-envelope"></i> @owner.email</p>
          <p><i class="fas fa-phone"></i> @restaurant.phone</p>
          <div class="rating mb-3">
            @for (int i = 0; i < 5; i++)
            {
              <i class="fa@i < Math.Floor(averageRating) ? "s" : (i < averageRating ? "s fa-star-half-alt" : "r")" fa-star" title="@Math.Round(averageRating, 2)" style="transform: none;"></i>
            }
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
```

Jóváhagyásra váró termékek szekció:

Azokat a termékeket jeleníti meg, amelyek még nem lettek jóváhagyva.

```
<section class="menu-section py-5">
  <div class="container">
    <h2 class="text-center text-danger mb-4">Jóváhagyásra váró termékek</h2>
    @if (menuItems != null && menuItems.Any(item => item.is_approved == false))
    {
      @foreach (var category in categories)
      {
        var unapprovedItems = menuItems.Where(item => item.category_id == category.category_id && item.is_approved == false).ToList();
        if (unapprovedItems.Any())
        {
          <div class="menu-category mb-5">
            <h2 class="text-center mb-4">@category.category_name</h2>
            <div class="row">
              @foreach (var item in unapprovedItems)
              {
                <div class="col-md-6 mb-4">
                  <div class="menu-item d-flex" @onclick="() => EditDish(item)">
                    <div class="menu-item-image me-3">
                      
                    </div>
                    <div class="menu-item-content flex-grow-1">
                      <h5>@item.name</h5>
                      <span class="price">@($"{item.price:n0} Ft)</span>
                      <p class="text-muted">@item.description</p>
                    </div>
                  </div>
                </div>
              }
            </div>
          </div>
        }
      }
    }
    else
    {
      <div class="text-center my-4">
        <h4>Nincs jóváhagyásra váró termék.</h4>
      </div>
    }
  </div>
</section>
```


Jóváhagyott termékek szekció:

Azokat a termékeket jeleníti meg, amelyek már jóvá lettek hagyva.

```
<section class="menu-section py-5">
  <div class="container">
    <h2 class="text-center text-success mb-4">Jóváhagyott termékek</h2>
    @if (menuItems != null && menuItems.Any(item => item.is_approved == true))
    {
      @foreach (var category in categories)
      {
        var approvedItems = menuItems.Where(item => item.category_id == category.category_id && item.is_approved == true).ToList();
        if (approvedItems.Any())
        {
          <div class="menu-category mb-5">
            <h2 class="text-center mb-4">@category.category_name</h2>
            <div class="row">
              @foreach (var item in approvedItems)
              {
                <div class="col-md-6 mb-4">
                  <div class="menu-item d-flex" @onclick="() => EditDish(item)">
                    <div class="menu-item-image me-3">
                      
                    </div>
                    <div class="menu-item-content flex-grow-1">
                      <h5>@item.name</h5>
                      <span class="price">@($"{item.price:n0} Ft)</span>
                      <p class="text-muted">@item.description</p>
                    </div>
                  </div>
                </div>
              }
            </div>
          </div>
        }
      }
    }
    else
    {
      <div class="text-center my-4">
        <h4>Nincs jóváhagyott termék.</h4>
      </div>
    }
  </div>
</section>
```

Vendégvélemények szekció:

Az étteremhez tartozó vendégvéleményeket jeleníti meg.

```
<section class="reviews-section py-5 bg-light">
  <div class="container">
    <h2 class="text-center mb-5">Vendégeink írták</h2>
    @if (reviews != null && reviews.Any())
    {
      <div class="table-responsive" style="max-height: 400px; overflow-y: auto;">
        <table class="table table-striped">
          <thead class="table-dark sticky-top" style="text-align: center">
            <tr>
              <th>Email</th>
              <th>Dátum</th>
              <th>Étel</th>
              <th>Szolgáltatás</th>
              <th>Tisztaság</th>
              <th>Hangulat</th>
              <th>Átlag</th>
              <th>Vélemény</th>
              <th>Műveletek</th>
            </tr>
          </thead>
          <tbody>
            @foreach (var review in reviews.Where(r => r.is_deleted == false))
            {
              <tr>
                <td>@reviewUsers[review.user_id].email</td>
                <td>@($"{review.created_at:yyyy. MM. dd. HH:mm}")</td>
                <td>@review.food_rating</td>
                <td>@review.service_rating</td>
                <td>@review.cleanliness_rating</td>
                <td>@review.atmosphere_rating</td>
                <td>@Math.Round(review.avg_rating, 3)</td>
                <td>@review.review_text</td>
                <td>
                  <button class="btn btn-danger btn-sm" @onclick="() => DeleteReview(review.review_id)">
                    <i class="fas fa-trash"></i> Törlés
                  </button>
                </td>
              </tr>
            }
          </tbody>
        </table>
      </div>
    }
    else
    {
      <h2 class="text-danger" style="text-align: center;">Még nincs értékelés ehhez az étteremhez!</h2>
    }
  </div>
</section>
```

2.21.2. JAVASCRIPT INTEGRÁCIÓ

closeModal(modalId):

Bezárja a megadott azonosítójú modális ablakot.

```
function closeModal(modalId) {
    const modal = bootstrap.Modal.getInstance(document.getElementById(modalId));
    modal.hide();
}
```

2.21.3. C# METÓDUSOK

OnInitializedAsync():

Betölti az étterem adatait, termékeit, kategóriáit és értékeléseit.

```
protected override async Task OnInitializedAsync()
{
    try
    {
        var restaurantName = name.Replace("-", " ");
        restaurant = await RestaurantService.GetRestaurantByName(restaurantName);
        if (restaurant != null)
        {
            menuItems = (List<MenuItem>)await MenuItemService.GetMenuItemsByRestaurantId(restaurant.restaurant_id);
            categories = (List<MenuCategory>)await MenuCategoryService.GetAllCategories();
            reviews = await ReviewService.GetReviewsByRestaurantId(restaurant.restaurant_id);
            owner = await UserService.GetUserById(restaurant.owner_id);

            foreach (var review in reviews.Where(r => r.is_deleted == false))
            {
                var reviewUser = await UserService.GetUserById(review.user_id);
                if (reviewUser != null)
                {
                    reviewUsers[review.user_id] = reviewUser;
                }
            }

            if (reviews.Any(r => r.is_deleted == false))
            {
                averageRating = reviews.Where(r => r.is_deleted == false).Average(r => r.avg_rating);
            }
        }
    }
    catch (Exception ex)
    {
        NavigationManager.NavigateTo("/");
    }
}
```

EditDish(MenuItem item):

Betölti a kiválasztott termék adatait szerkesztésre, és megnyitja a modális ablakot.

```
private void EditDish(MenuItem item)
{
    editMenuItem = new MenuItem
    {
        item_id = item.item_id,
        restaurant_id = item.restaurant_id,
        category_id = item.category_id,
        name = item.name,
        description = item.description,
        price = item.price,
        thumbnail = item.thumbnail,
        is_available = item.is_available,
        is_approved = item.is_approved,
        is_deleted = item.is_deleted,
        created_at = item.created_at
    };
    editThumbnailBase64 = item.thumbnail;

    JSRuntime.InvokeVoidAsync("eval", "new bootstrap.Modal(document.getElementById('editDishModal')).show();");
}
```

SaveEditedDish():

Mentésre kerülnek a szerkesztett termék adatai.

```
private async Task SaveEditedDish()
{
    try
    {
        if (!string.IsNullOrEmpty(editThumbnailBase64))
        {
            editMenuItem.thumbnail = editThumbnailBase64;
        }

        await MenuItemService.UpdateMenuItem(editMenuItem);
        await OnInitializedAsync();

        await JSRuntime.InvokeVoidAsync("closeModal", "editDishModal");
    }
    catch (Exception ex)
    {
        notification.ShowMessage($"Hiba történt az étel frissítése során: {ex.Message}", "red");
    }
}
```

DeleteReview(int reviewId)

Törli a megadott azonosítójú értékelést.

```
private async Task DeleteReview(int reviewId)
{
    var confirmed = await notification.ShowConfirmation("Biztosan törölni szeretné ezt az értékelést?");

    if (confirmed)
    {
        try
        {
            await ReviewService.DeleteReview(reviewId);

            reviews = reviews.Where(r => r.review_id != reviewId && r.is_deleted == false).ToList();

            if (reviews.Any())
            {
                averageRating = reviews.Average(r => r.avg_rating);
            }
            else
            {
                averageRating = 0;
            }

            notification.ShowMessage("Az értékelés sikeresen törlésre került.", "green");
            StateHasChanged();
        }
        catch (Exception ex)
        {
            notification.ShowMessage($"Hiba történt az értékelés törlése során: {ex.Message}", "red");
        }
    }
}
```

2.22. ROUTES.JSON

A routes.json fájl egy központi konfigurációs állomány a Blazor WebAssembly alkalmazásban, amely meghatározza az URL-útvonalak kezelését és a kliens oldali navigáció szabályait. Ez a fájl különösen fontos az Azure Static Web Apps szolgáltatásban történő telepítéskor, mivel ez biztosítja a kliens oldali routing megfelelő működését Azure környezetben.

2.22.1. STRUKTÚRA ÉS BEÁLLÍTÁSOK

A konfigurációs fájl JSON formátumú, amely a következő fő szekciókat tartalmazza:

2.22.1.1. ROUTES (ÚTVONALAK)

```
"routes": [  
  {  
    "route": "/admin*",  
    "serve": "/index.html"  
  },  
  ...  
]
```

Ez a szekció definiálja az alkalmazás speciális útvonalait és meghatározza, hogy az egyes URL mintákhoz milyen fájlt kell kiszolgáltatnia a szervernek.

- **route:** Az URL minta (támogatja a wildcard * karaktert)
- **serve:** A kiszolgáltató HTML fájl elérési útja

Az aktuális konfigurációban minden útvonal az index.html-hez van irányítva, ami biztosítja, hogy a kliens oldali router kezelje a navigációt.

2.22.1.2. NAVIGATIONFALLBACK (NAVIGÁCIÓS TARTALÉK)

```
"navigationFallback": {  
  "rewrite": "/index.html",  
  "exclude": [ "/images/*", "/css/*", "/js/*", "/api/*" ]  
}
```

Meghatározza az alapértelmezett viselkedést, ha egy útvonal nem található:

- **rewrite:** Átírányítja a nem létező útvonalakat az index.html-re
- **exclude:** Kivételek listája, amelyekre nem vonatkozik az átírányítási szabály

2.22.1.3. RESPONSEOVERRIDES (VÁLASZ FELÜLBÍRÁLÁSOK)

```
"responseOverrides": {
  "401": {
    "redirect": "/",
    "statusCode": 302
  },
  "404": {
    "rewrite": "/index.html",
    "statusCode": 200
  }
}
```

HTTP válaszkódok felülírására szolgáló beállítások:

- **401** (Unauthorized): Átirányítja a felhasználót a főoldalra (státuszkód: 302)
- **404** (Not Found): Átirányítja az index.html-re és 200-as státuszkódot ad vissza

2.22.2. AZ AZURE STATIC WEB APPS MIATT SZÜKSÉGES

A routes.json fájlra kifejezetten az Azure Static Web Apps szolgáltatás miatt van szükség, és a fájlnak a **wwwroot** könyvtárban kell lennie, ahol azt az Azure Static Web Apps automatikusan felismeri és alkalmazza, mivel:

- **Azure SWA routing kezelés:** Az Azure Static Web Apps alapértelmezetten nem ismeri a Blazor WebAssembly kliens oldali útválasztását. Ez a konfiguráció tájékoztatja az Azure-t, hogy hogyan kezelje a különböző URL-kéréseket.
- **Mélylinkek támogatása:** Azure Static Web Apps környezetben a közvetlen URL-hozzáférések 404-es hibát adnának e konfiguráció nélkül, mivel a szerver nem tudná, hogy a **/restaurants/etterem-neve** jellegű kéréseket is az index.html-nek kell kiszolgáltatnia.
- **SPA architektúra támogatása:** Ez a konfiguráció biztosítja, hogy az Azure megfelelően kezelje az egyoldalas alkalmazás (SPA) struktúrát, és minden navigációs kérést a Blazor alkalmazás gyökerére irányítson.

2.23. KÉPERNYŐTERVEK

Bejelentkezés/Regisztráció

GasztroTükör

BEJELENTKEZÉS X

Email cím

Jelszó

Bejelentkezés

Elfelejtett jelszó?

FOOTER

GasztroTükör

Regisztráció

Email cím

Jelszó

Jelszó megerősítése

Felhasználó típusa

☐ Vendég ☒ Tulajdonos

Étterem neve

Telefonszáma

Konyha típusa

Rövid leírás

Borítókép

Fájl kiválasztása

Cím

Nyitási ideje

12:00 20:00

Regisztráció

Bejelentkezés Regisztráció

The image shows a web application interface with a registration modal. The modal is titled "Regisztráció" and has a red header bar with a close button (X). It contains the following fields and elements:

- Email cím:** A text input field.
- Jelszó:** A password input field with a toggle icon (eye) to the right.
- Jelszó:** A second password input field with a toggle icon (eye) to the right.
- Felhasználó típusa:** Two radio buttons labeled "Vendég" (selected) and "Tulajdonos".
- Regisztráció:** A red button at the bottom of the modal.

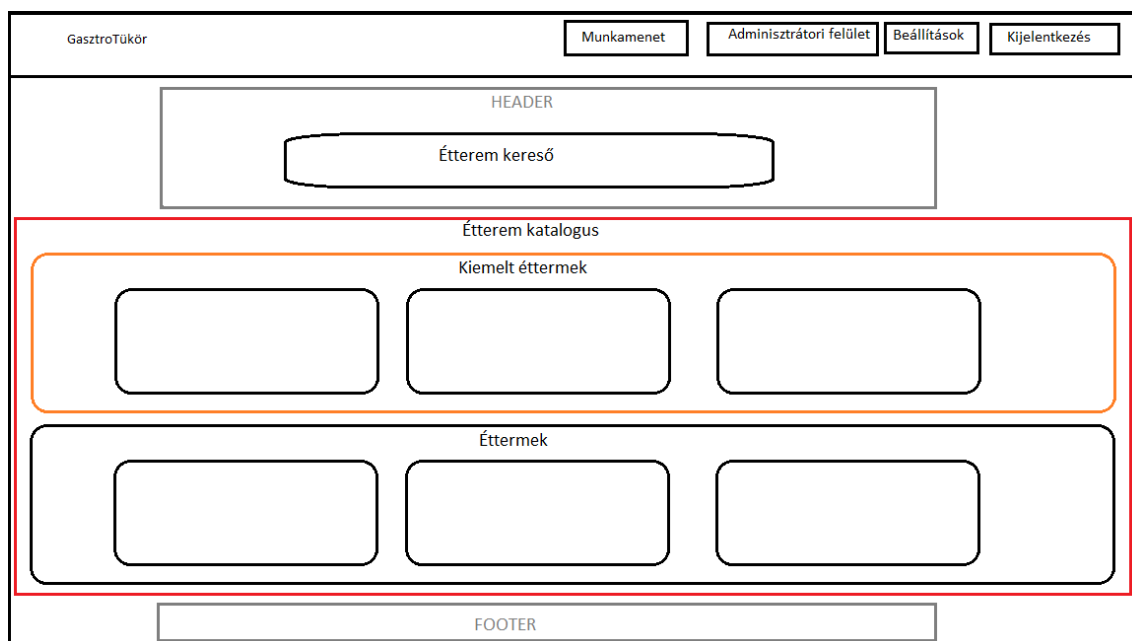
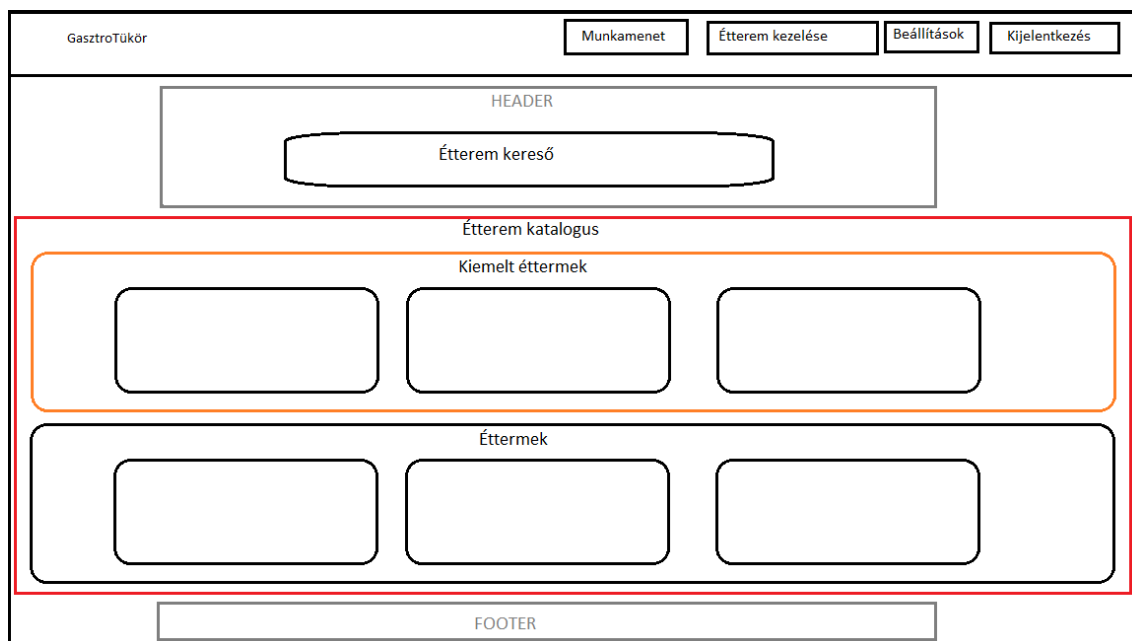
The background shows a header with "GasztroTükör" and buttons for "Bejelentkezés" and "Regisztráció". A red rectangle highlights the main content area of the page.

Főképernyő

The image shows the main page layout of the application. It includes the following sections:

- Header:** Contains the "GasztroTükör" logo and buttons for "Bejelentkezés" and "Regisztráció".
- Search Bar:** A large rounded rectangle labeled "Étterem kereső".
- Restaurant Catalog:** A section titled "Étterem katalogus" containing a sub-section "Kiemelt éttermek" with three placeholder boxes, and another section "Éttermek" with three placeholder boxes.
- Footer:** A bar at the bottom labeled "FOOTER".

A red rectangle highlights the main content area, which includes the search bar and the restaurant catalog.



Admin/Étterem kezelés

GasztroTükör
Munkamenet
Adminisztrátori felület
Beállítások
Kijelentkezés

HEADER

Étterem kereső

Étterem katalogus

JÓVÁHAGYÁSRA VÁRÓ ÉTTERMEK

JÓVÁHAGYOTT ÉTTERMEK

FOOTER

GasztroTükör
Munkamenet
Étterem kezelése
Beállítások
Kijelentkezés

HEADER

Étterem leírás/adatok szerkesztése

Étlap

+

Új előétel

+

Új főétel

+

Új desszert

+

Új előétel

+

Új főétel

+

Új desszert

Vendégeink írták

FOOTER

Beállítások

GasztroTükör

Beállítások

Kijelentkezés

Beállítások

Profil Adatok

Email cím

Jelenlegi jelszó

Új jelszó

Új jelszó megerősítése

Fiók törlése

Mentés

FOOTER

Admin

beállítások

GasztroTükör

Beállítások

Kijelentkezés

Beállítások

Profil Adatok Felhasználók kezelése

Email	Szerepkör	Műveletek
csordas.patrik.cristian@ganziskola.hu	ADMIN	Adminisztrátor nem törölhető
szoke.daniel@ganziskola.hu	ADMIN	Adminisztrátor nem törölhető
marton.bence.david@ganziskola.hu	ADMIN	Adminisztrátor nem törölhető
customer1@pelda.hu	customer	Törlés
customer2@pelda.hu	customer	Törlés
customer3@pelda.hu	customer	Törlés
customer4@pelda.hu	customer	Törlés
info@labellaitalia.hu	owner	Törlés

FOOTER

Étterem (új étel hozzáadása)

94

Vendégként étterem értékelés

Étterem értékelése

Étel ☆☆☆☆☆

Szolgáltatás ☆☆☆☆☆

Tisztaság ☆☆☆☆☆

Hangulat ☆☆☆☆☆

Vélemény

Értékelés küldése

2.24. BEJELENTKEZÉSI ADATOK

Email	Szerepkör	Jelszó
csordas.patrik.cristian@ganziskola.hu	admin	admin
szoke.daniel@ganziskola.hu	admin	admin
marton.bence.david@ganziskola.hu	admin	admin
customer1@pelda.hu	customer	customer
customer2@pelda.hu	customer	customer
info@labellaitalia.hu	owner	labellaitalia
info@sushimester.hu	owner	sushimester
info@eltacoloco.hu	owner	eltacoloco
info@lebistrotfrancais.hu	owner	lebistrotfrancais
info@bajorudvar.hu	owner	bajorudvar
info@tasteofindia.hu	owner	tasteofindia
info@casaportuguesa.hu	owner	casaportuguesa
info@olivafataverna.hu	owner	olivafataverna
info@koreanbbqhouse.hu	owner	koreanbbqhouse
info@thaikert.hu	owner	thaikert
info@huszarcsarda.hu	owner	huszarcsarda
info@americandiner.hu	owner	americandiner
info@scandinaviankitchen.hu	owner	scandinaviankitchen
info@cedrusgrill.hu	owner	cedrusgrill
info@russianvodkahouse.hu	owner	russianvodkahouse

2.25. FEJLESZTÉSI LEHETŐSÉGEK

A GasztroTükör platform jelenlegi állapota szilárd alapot biztosít a további fejlesztésekhez, azonban számos területen van lehetőség a bővítésre és finomításra. Az alábbiakban néhány javaslatot fogalmaztunk meg, amelyek hozzájárulhatnak a felhasználói élmény javításához és a platform funkcionalitásának bővítéséhez:

1. Közösségi funkciók bővítése

- **Felhasználói profilok testreszabása:** Lehetővé tehetnénk a felhasználók számára, hogy személyre szabott profilekat hozzanak létre, például profilképpel, kedvenc éttermek listájával és egyéni preferenciákkal.

2. Értékelési rendszer fejlesztése

- **Értékelések hitelességének növelése:** Bevezethetünk egy moderációs rendszert, amely segít kiszűrni a nem releváns vagy nem megfelelő értékeléseket.

3. Keresési és szűrési funkciók fejlesztése

- **Továbbfejlesztett keresőmotor:** A keresési funkció bővítése árkategóriák, nyitvatartási idő vagy értékelési szempontok alapján.

4. Étteremtulajdonosok számára nyújtott eszközök

- **Statisztikai elemzések:** Az étteremtulajdonosok számára részletes statisztikák biztosítása a látogatottságról, értékelésekről és trendekről.
- **Promóciós lehetőségek:** Hirdetési és promóciós eszközök bevezetése, amelyek mégjobban segítenek az éttermek népszerűsítésében.

5. Felhasználói visszajelzések integrálása

- **Visszajelzési rendszer:** Egy egyszerű és gyors visszajelzési lehetőség biztosítása, amely segít az oldal folyamatos fejlesztésében.

2.26. TESZTELESI TERV

A tesztelési terv célja az oldal funkcióinak ellenőrzése, a hibák feltárása és a felhasználói élmény optimalizálása.

2.26.1. TESZTELESI CÉLOK

- Az oldal alapvető funkcióinak ellenőrzése.
- A felhasználói élmény értékelése különböző eszközökön.
- A biztonsági és adatvédelmi szempontok felülvizsgálata.

2.26.2. TESZTELESI TÍPUSOK

2.26.2.1. FUNKCIONÁLIS TESZTEK

- **Étterem keresése és listázása:** Ellenőrizni, hogy a kereső megfelelően működik-e, és a találatok relevánsak-e.
- **Értékelések megtekintése és hozzáadása:** Biztosítani, hogy a vendégek értékelései megfelelően jelennek meg, és új értékelések hozzáadása hibamentes.
- **Felhasználói regisztráció és bejelentkezés:** Tesztelni a regisztrációs és bejelentkezési folyamatokat, beleértve a jelszó-visszaállítást is.
- **Adminisztrációs funkciók:** Ellenőrizni, hogy az adminisztrátorok képesek-e új éttermeket hozzáadni, meglévőket szerkeszteni vagy törölni.

2.26.2.2. EGYSÉGTESZTEK

- **Backend logika:** Tesztelni az adatbázis műveleteket, például az értékelések mentését és lekérdezését.
- **Érvényesítések:** Ellenőrizni, hogy az űrlapok megfelelően kezelik-e a hibás vagy hiányos adatokat.

2.26.2.3. INTEGRÁCIÓS TESZTEK

- **Frontend és backend összekapcsolása:** Biztosítani, hogy az értékelések beküldése és megjelenítése zökkenőmentes.
- **Jogosultságkezelés:** Tesztelni, hogy a különböző felhasználói szerepkörök megfelelő jogosultságokkal rendelkeznek-e.

2.26.3. TESZTELÉSI KÖRNYEZET

- **Böngészők:** Chrome, Firefox, Edge, Safari.
- **Eszközök:** Asztali számítógépek, táblagépek, okostelefonok.
- **Operációs rendszerek:** Windows, macOS, iOS, Android.

2.26.4. TESZTESETEK

2.26.4.1. REGISZTRÁCIÓ/BEJELENTKEZÉS

Teszt eset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Sikeres regisztráció	1. Regisztrációs űrlap megnyitása 2. Válassza ki a felhasználó típusát 3. Töltse ki az összes mezőt 4. Küldje el a regisztrációját	Sikeres regisztráció visszajelzése, Utasítás a bejelentkezésre
2.	Hibás email-cím kezelése	Hibás email cím megadása pl.: ezegyhibasemail@gmail.com	Hibaüzenet: "Az email címből hiányzik a @-jel"
3.	Duplikált regisztráció	Már egy regisztrált címmel való regisztrálás	Hibaüzenet: "Az email cím már létezik az adatbázisban"
4.	Sikeres bejelentkezés	Bejelentkezés egy helyes email/jelszó kombinációval	Sikeres bejelentkezés visszajelzés
5.	Hibás bejelentkezés	Bejelentkezés egy hibás email/jelszó kombinációval	Hibaüzenet: "Helytelen email cím és/vagy jelszó" vagy "Ez az email cím még nincs regisztrálva"

2.26.4.2. ÉTTEREM KERESÉS / LISTÁZÁS

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Éttermek listázása / megjelenítése	1. Főoldal megnyitása	Megjelennek a hitsített éttermek
2.	Étterem keresés	1. Írja be egy étterem nevét vagy egy étterem konyha típusát vagy egy étterem leírás részletét a keresőmezőbe	Csak azok az éttermek megjelenítése, amelyek kártyája tartalmazza a kereső mezőbe beírt szöveget
3.	Nem létező étterem keresése	1. Pl.: Kereső mezőbe beírt szöveg: "xxxZZxz étterem" vagy "furfangos indiai ételek"	"Nincs találat a megadott keresési feltételre." szöveg megjelenítése az éttermek helyett
4.	További információ egy étteremről	1. Kívánt étterem megkeresése 2. Az étterem kártyáján lévő "Bővebben..." gomb megnyomása 3. Átírányítás a választott étterem oldalára	Sikeres átirányítás a választott oldalára

2.26.4.3.ÉRTÉKELÉSEK

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Értékelés hozzáadása	1. Jelentkezzen be egy vendég típusú fiókba 2. Nyissa meg az értékelni kívánt étterem oldalát 3. Kattintson az étterem információs részén a "Értékeljen minket!" fölötti csillagokra 4. Töltse ki az értékelő űrlapot 5. Küldje el az értékelését	"Köszönjük, hogy megosztotta véleményét." visszajelzés
2.	Üres szöveges értékelés beküldése	1. Hagyja üresen a szöveges értékelés mezőt 2. Küldje el értékelését	Hibaüzenet: "Kérjük, töltse ki ezt a mezőt"
3.	Már egy értékelt étterem újra értékelése	1. Értékeljen egy éttermet 2. Próbálja meg értékelni ugyan azt az éttermet	Hibaüzenet: "Ön már írt értékelést ehhez az étteremhez!"

2.26.4.4. PROFIL BEÁLLÍTÁSOK / ÉRTÉKELÉSEK MÓDOSÍTÁSA

Teszt eset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Sikeres jelszó megváltoztatása	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. Adja meg a jelenlegi jelszavát a "Jelenlegi jelszó" mezőbe majd adja meg az új jelszavát a megfelelő mezőkbe 4. Mentse el az új jelszavát	"A jelszó sikeresen módosítva." visszajelzés
2.	Sikeres fiók törlése	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. Adja meg a jelenlegi jelszavát a "Jelenlegi jelszó" mezőbe 4. Nyomjon rá a "Fiók törlése" gombra 5. Erősítse meg a fiókja törlését a jobb oldali felugró "sávba"	"Sikeresen törölte fiókját" visszajelzés
3.	Sikertelen jelszó módosítás	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. Adjon meg egy hibás jelszót a "Jelenlegi jelszó" mezőbe majd adja meg az új jelszavát a megfelelő mezőkbe 4. Mentse el az új jelszavát	Hibaüzenet: "Jelszó módosítás sikertelen: A jelenlegi jelszó helytelen."
4.	Sikertelen fiók törlés	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. Adjon meg egy hibás jelszót a "Jelenlegi jelszó" mezőbe 4. Nyomjon rá a "Fiók törlése" gombra 5. Erősítse meg a fiókja törlését a jobb oldali felugró "sávba"	Hibaüzenet: "Fiók törlése sikertelen: Hibás jelenlegi jelszó."
5.	Értékeléseim megjelenítése	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. A felrúgó ablakba kattintson az "Értékeléseim" opciót	A felhasználó összes értékeléseinek megjelenítése
6.	Meglévő értékelés módosítása	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban	"Az értékelés sikeresen frissítve." visszajelzés

		3. A felrúgó ablakba kattintson az "Értékeléseim" opciót 4. Keresse meg a szerkeszteni kívánt értékelést 5. Nyomja meg a "Szerkesztés" gombot 6. A felugró ablakba írja át az értékelése szöveget 7. Mentse el a szerkesztett értékelését	
7.	Meglévő értékelés törlése	1. Jelentkezzen be 2. Nyomja meg a "Beállítások" gombot a felső navigációssávban 3. A felrúgó ablakba kattintson az "Értékeléseim" opciót 4. Keresse meg a szerkeszteni kívánt értékelést 5. Nyomja meg a "Törlés" gombot. 6. A felugró sávba erősítse meg a törlést	Az értékelés törlése az "Értékeléseim" és az értékelt étterem értékelései közül, illetve "Az értékelést sikeresen törölte"

2.26.4.5. ÉTTEREM KEZELÉSE

Tesztelési lépés	Tesztelési cél	Lépések	Elvárt eredmény
1.	Étterem kezelési felület megnyitása	1. Jelentkezzen be egy „Étterem tulajdonos” típusú fiókkal 2. Nyomja meg a "Étterem kezelése" gombot a felső navigációssávban	Megnyílik az fiókhoz tartozó étterem kezelési oldala
2.	Étterem információk szerkesztése	1. Jelentkezzen be egy „Étterem tulajdonos” típusú fiókkal 2. Nyomja meg a "Étterem kezelése" gombot a felső navigációssávban 3. Az oldal tetején az étterem jelenlegi információt tartalmazó sávra rákattintva megnyíló űrlapon szerkessze az étterme információit 4. Mentse el a változtatásokat	Bezáródik a szerkesztői felület és frissül a változtatott rész
3.	Menü szerkesztése	1. Jelentkezzen be egy „Étterem tulajdonos” típusú fiókkal 2. Nyomja meg a "Étterem kezelése" gombot a felső navigációssávban 3. Görgessen az oldalon ahhoz a menüelemhez, amit szerkeszteni szeretne és nyomjon rá az menü elem kártyára 4. A felugró ablakba módosítsa a menü elemet kedvére 5. Mentse el a változtatásokat	Bezáródik a szerkesztői felület és frissül a menü elem

4. Új Menü elem hozzáadása	1. Jelentkezzen be egy „Étterem tulajdonos” típusú fiókkal 2. Nyomja meg a "Étterem kezelése" gombot a felső navigációssávban 3. Görgessen az oldalon ahhoz a Menü elem típushoz, amit szeretne bővíteni 4. A felugró ablakba töltse ki a hozzá adni kívánt elem adataival 5. Mentse el az új elemet	Bezáródik a szerkesztői felület és frissül a menü
-----------------------------------	---	--

3. FELHASZNÁLÓI DOKUMENTÁCIÓ

3.1. A PROGRAM CÉLJÁNAK ÉS LÉNYEGESEBB FUNKCIÓINAK ÖSSZEFOGLALÁSA

A GasztroTükör egy étterem értékelő weboldal, amely lehetőséget biztosít a vendégek számára, hogy a már meglátogatott éttermekről tapasztalat alapján visszajelzést adjanak az étteremtulajdonosoknak. Ezzel a lehetőséggel a feltörekvő és már hírnévvel rendelkező éttermek tudják növelni népszerűségüket. A GasztroTükör célja, hogy egyszerre szolgálja ki az éttermek marketingcéljait és a vendégek tájékozódását esetleges új éttermeiről.

A program fő funkciói:

- Vendég: regisztráció, bejelentkezés, éttermek keresése, értékelések olvasása és írása, saját értékelés szerkesztése, fiókadatok megtekintése és módosítása.
- Étterem tulajdonos: regisztráció, bejelentkezés, saját étterem létrehozása, étterem adatainak szerkesztése, menük feltöltése és szerkesztése, vendégértékelések megtekintése.
- Adminisztrátor: felhasználók kezelése, éttermek és menük kezelése, vélemények kezelése, jogosultságok kezelése.

3.2. SZÜKSÉGES HARDVERESZKÖZÖK ÉS SZOFTVEREK FELSOROLÁSA

Hardverkövetelmények

- Asztali számítógép vagy laptop, mobilkészülék.
- Stabil internetkapcsolat

Szoftverkövetelmények

- Böngésző (legfrissebb verzió): Google Chrome, Mozilla Firefox, Microsoft Edge
- Operációs rendszer: Napjainkban használt operációs rendszerek és verzióik (Windows, MacOS, Linux, Android, iOS)

3.3. TELEPÍTÉS ÉS INDÍTÁS LÉPÉSEINEK ISMERTETÉSE

A GasztroTükör egy webes alkalmazás, nem szükséges telepíteni. Megfelelő internetkapcsolat és egy böngésző szükséges a használatához.

Használatba vétel lépései:

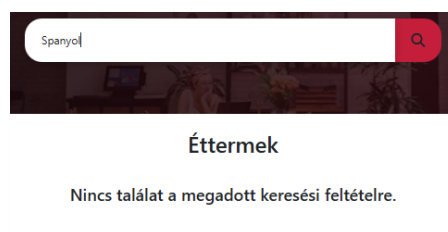
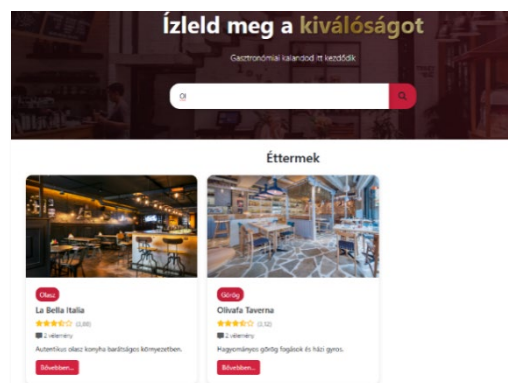
- Nyissa meg az internetböngészőt (Google Chrome, Mozilla Firefox, Microsoft Edge)
- A címsorba írja be a weboldal címét: <https://www.gasztrotukor.hu>
- Az oldal betöltődése után megjelenik a kezdőlap.

3.4. A PROGRAM RÉSZLETES BEMUTATÁSA

3.4.1. VENDÉG FUNKCIÓK:

3.4.2. KERESŐMEZŐ MŰKÖDÉSE

- A keresőmező mint ahogy a mezőben is látható három féle részben keres. Étterem neve, étteremnek a leírása, és a konyha típusa.
- Amit ide beír a felhasználó azt vizsgálja a program és a szerint jeleníti meg az éttermeket. Lehet az egy kulcsszó vagy akár csak egy betű, ha egyezést talál bármely étterem nevével/leírásával/konyhatípusával akkor megjeleníti a megfelelő éttermet.
- Van olyan opció is amikor a keresésre nincs megfelelő találat akkor az oldal ezt jelzi, már a mezőbe íráskor szortírozza a találatokat így a keresés gombra nem kell rákattintani.



3.4.2.1. REGISZTRÁCIÓ ÚJ VENDÉGKÉNT

- Email cím
 - Egy létező email cím megadása
- Jelszó
 - Válasszon egy olyan jelszót amely minimum 8, maximum 20 karakterből áll.
 - A jelszónak tartalmaznia kell minimum 1-db nagy betűt (pl: T).
 - A jelszónak tartalmaznia kell minimum 1-db számot.(0-9)
 - A jelszónak tartalmaznia kell egy speciális karaktert a felsoroltak közül (!@#\$\$%^&*_._-).

The registration form is titled 'Regisztráció' and includes the following fields and options:

- Email cím:** A text input field containing 'teszt@pelda.hu'.
- Jelszó:** A text input field containing 'Teszt12!' with a toggle icon for password visibility.
- Jelszó megerősítése:** A text input field containing 'Teszt12!' with a toggle icon for password visibility.
- Felhasználó típusa:** Two radio buttons: 'Vendég' (selected) and 'Tulajdonos'.
- Regisztráció:** A red button at the bottom.

3.4.2.2. SIKERES REGISZTRÁCIÓ

A regisztráció sikerességéről az oldal értesítést küld e-mail-ben.

The email is from 'GasztroTükör' and is addressed to 'teszt@pelda.hu'. The content includes:

- Greeting: 'Kedves teszt@pelda.hu!'
- Message: 'Köszönjük, hogy regisztrált a GasztroTükör oldalra. Reméljük, hogy élvezni fogja az oldal nyújtotta lehetőségeket.'
- Signature: 'Üdvözlettel, A GasztroTükör csapata'
- Footer: '© 2025 GasztroTükör - Minden jog fenntartva' and 'Ez egy automatikusan küldött üzenet. Kérjük, ne válaszoljon erre az e-maillal'

3.4.2.3. BEJELENTKEZÉS

Amint megkapta az értesítést hogy a regisztrációja sikeres volt azután már betud jelentkezni a fiókjába. A megadott E-mail cím és jelszó kitöltése után a „bejelentkezés” gombra kattintva

The login form is titled 'Bejelentkezés' and includes the following fields and options:

- Email cím:** A text input field containing 'teszt@pelda.hu'.
- Jelszó:** A text input field containing 'Teszt12!' with a toggle icon for password visibility.
- Bejelentkezés:** A red button.
- Elfelejtett jelszó?:** A link below the login button.

3.4.2.4. ELFELEJTETT JELSZÓ

- Amennyiben esetleg nem emlékezne a jelszávára amit regisztrációkor adott meg, akkor a bejelentkezés fülénél tudja használni az elfelejtett jelszó funkciót amennyiben az E-mail címét helyesen kitöltötte.
- Amint rákattint az „Elfejtett jelszó?” lehetőségre, az oldal jobb oldalán megfog jelenni egy értesítési fül, ahol lehetősége van új jelszót igényelni.
- Ha az „Igen” lehetőséget választotta akkor az oldal elküldi E-mailben az új automatikusan generált jelszavát.
- Ezzel az automatikusan generált jelszóval, és a meglévő email-címével betud jelentkezni.

The screenshot shows a login form with two input fields: 'Email cím' (Email address) containing 'teszt@pelda.hu' and 'Jelszó' (Password) with masked characters. Below the fields is a red 'Bejelentkezés' (Login) button. At the bottom, the text 'Elfejtett jelszó?' (Forgot password?) is visible.

The screenshot shows a yellow confirmation dialog box with the text 'Biztosan szeretne új jelszót igényelni?' (Are you sure you want to request a new password?). It has two buttons: a green 'Igen' (Yes) button and a red 'Nem' (No) button.

The screenshot shows an email from 'GasztroTükör' to 'teszt@pelda.hu'. The email body states: 'Kedves teszt@pelda.hu! Az új jelszava: WbTBsx%Ydr\$e'. It also includes a note: 'Ez egy automatikusan generált jelszó. A biztonság érdekében javasoljuk, hogy bejelentkezés után a Beállítások menüpontban változtassa meg.' (This is an automatically generated password. For security reasons, we recommend changing it after login in the Settings menu). The email is signed 'A GasztroTükör csapata' (The GasztroTükör team). The footer contains copyright information: '© 2025 GasztroTükör - Minden jog fenntartva' and a note: 'Ez egy automatikusan küldött üzenet. Kérjük, ne válaszoljon erre az e-maillal' (This is an automatically sent message. Please do not reply to this email).

3.4.2.5. JELSZÓ MEGVÁLTOZTATÁSA

- Érdeemes viszont a beállításokba ezt az automatikusan generált jelszót átállítani, hogy gördülékenyebb legyen a bejelentkezés.
- Itt az automatikusan generált jelszót bemásoljuk a „jelenlegi jelszó” helyére és megadhatjuk az „új jelszót”, ha mindennel végeztünk a mentés gombra kattintva elmenthetjük a módosításokat.

3.4.2.6. FIÓK TÖRLÉSE

Amennyiben arra a döntésre jutna, hogy nem szeretne az oldalon bármilyen formában szerepelni, akkor tudja a felhasználói fiókját törölni. Ezt bejelentkezés után a beállítások menüpontban a „Fiók törlése” gombra kattintva tudja megtenni.

FIGYELEM: Ez a művelet nem vonható vissza azután, hogy az „Igen” opciót választotta. Ekkor minden meglévő étterem értékelése/véleménye törlődik, a fiók a megadott email címmel -és jelszóval törlődik.

3.4.2.7. ÉRTÉKELÉS KÜLDÉSÉNEK LÉPÉSEI

- Bejelentkezés után az éttermek közül kiválasztja azt, amelyet szeretne értékelni és rákattint a „Bővebben...” gombra az étteremnél.
- Ekkor az oldalon megfog jelenni az étterem oldala, ahol láthatja a különféle étterem információkat, étterem kínálatát és árait, és az oldal alján lévő értékeléseket.

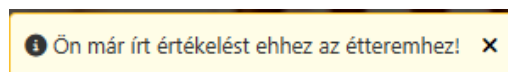


- Hogyha értékelést szeretne írni az adott étteremről azt az étterem adatainál a csillagokra kattintva teheti meg az „értékeljen minket” fül felett.



- Ezzel a lehetőséggel előjön egy értékelés fül ahol személyes tapasztalatait oszthatja meg mind csillag mind személyes vélemény alapján.
- Értékelés küldése gomra kattintva elküldi az értékelést ami az étterem oldal alján megjelenik.

FIGYELEM: Egy felhasználó minden étteremhez csak egyszer tud értékelést küldeni.



FIGYELEM: Minden felhasználó a már meglévő értékelését tudja szerkeszteni vagy akár törölni a „beállítások” -on belül a „értékeléseim” menüpontban.

Étterem	Dátum	Értékelés	Vélemény	Műveletek
Huszár Csárda	2025. 04. 13.	Étel: 5/5 Szolgáltatás: 3/5 Tisztaság: 4/5 Hangulat: 1/5 Átlag: 3,2/5	Példa értékelés	☒ Szerkesztés ☐ Törölés

FIGYELEM: Nem küldhet úgy értékelést, hogy bármely csillaggal értékelhető opció vagy az írásra kijelölt hely nem tartalmaz minimum 1 karaktert.

3.4.3. ÉTTEREM TULAJDONOS FUNKCIÓK

3.4.3.1. ÉTTEREM REGISZTRÁCIÓ

- Email cím
 - Egy létező email cím megadása
- Jelszó
 - Válasszon egy olyan jelszót amely minimum 8, maximum 20 karakterből áll.
 - A jelszónak tartalmaznia kell minimum 1-db nagy betűt (pl: T).
 - A jelszónak tartalmaznia kell minimum 1-db számot.(0-9)
 - A jelszónak tartalmaznia kell egy speciális karaktert a felsoroltak közül (!@#\$\$%^&*_.-).

A felhasználó típust tegye át „Tulajdonosra”. Ezzel megerősíti, hogy étterem tulajdonos és az étterem regisztrációjához szükséges lenyíló mezőket töltsse ki.

FIGYELEM: Regisztrációkor nem maradhat üres mező, különben a regisztráció sikertelen lesz.

Regisztráció

Email cím
peldaetterm@pelda.hu

Jelszó
Etterem12!

Jelszó megerősítése
Etterem12!

Felhasználó típusa
☐ Vendég ☒ Tulajdonos

Étterem neve
Teszt Étterem

Telefonszám
+36112345678

Konyha típusa
Magyar

Rövid leírás
Teszt leírás

Borítókép
Fájl kiválasztása tesztetterm.jpg

Cím
1234 Budapest Teszt Elek Utca 1

Nyitási ideje
10:00

Zárási ideje
18:00

Regisztráció

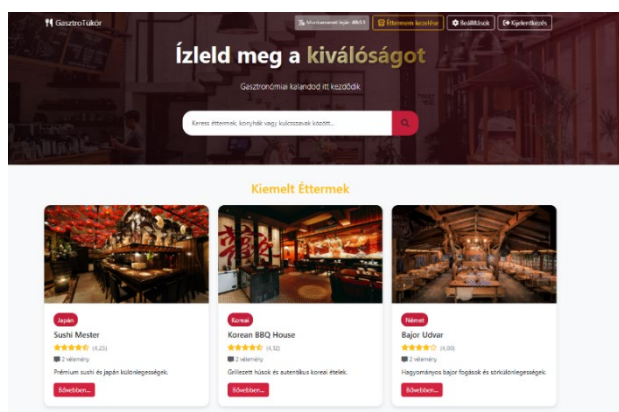
- Étterem neve
 - Adja meg az éttermének nevét, tartalmazhat nagybetűt, számot.
- Telefonszám
 - Adja meg az étterem telefonszámát amit későbbiekben a vendégek a nyitva tartás alatt elérnek.
- Konyha típusa
 - Adja meg hogy milyen az éttermének a konyha típusa, milyen kultúra alapján készítik az ételeket.
- Rövid leírás
 - Röviden foglalja össze az éttermének a leírását, mik kaphatóak az étteremben.
- Borítókép
 - Válasszon az étterméről egy képet, ami a borítóképként fog szerepelni, a kép mérete nem haladhatja meg az 500 kilobájt.
- Cím
 - Adja meg az étterem címét ahol helyileg is megtalálható az étterem (Írányítószám/Város/Utca/Házszám)
- Nyitás/Zárás ideje
 - Adja meg az étterem nyitvatartását.

FIGYELEM: Amint étteremtulajdonosként regisztrált, az étterme nem fog megjelenni mivel azt az oldal adminisztrátorainak jóvá kell hagynia. Étterem tulajdonosként nem tud véleményt írni saját és mások étterméhez.

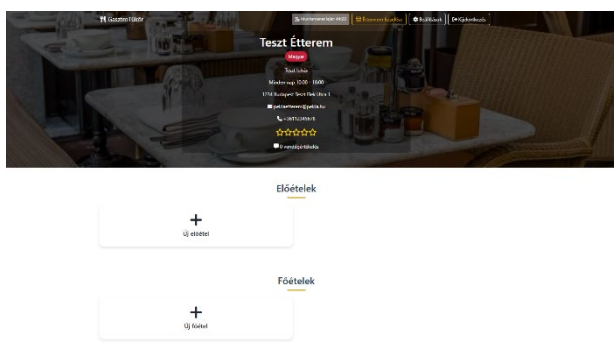


3.4.3.2. ÉTTEREM KEZELÉSE

- Miután regisztrálta az éttermét és bejelentkezett, akkor ez az oldal fogja várni.
- Ez a felület az étterem tulajdonos felület, ahol saját éttermét tudja karbantartani, változtatásokat elvégezni.
- A saját éttermét az alábbi menüpontban „Éttermem kezelése” gombnál tudja megtekinteni vagy akár szerkeszteni.



- Ha kiválasztotta az étterem kezelése menüpontot a navigációs sávban akkor az oldal átfogja önt irányítani az étterme oldalára.



3.4.3.3. ÉTTEREM ADATAINAK MÓDOSÍTÁSA

- Ha az étterem publikus adatait szeretné módosítani akkor azt a „Éttermem kezelése” menüpontban az étterem adataira kattintva megteheti.



- Itt a megjelent adatokat tudja módosítani.
 - Étterem neve
 - Telefonszám
 - Konyha típusa
 - Rövid leírás
 - Borítókép
 - Cím
 - Nyitás ideje
 - Zárás ideje

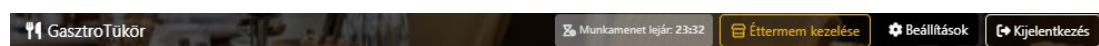
FIGYELEM: Az étterem e-mail címét nem tudja módosítani, válasszon regisztrációkor olyan e-mail címet, amit a későbbiekben nem kíván módosítani vagy akár cserélni.

The 'Szerkesztés' modal form contains the following fields:

- Név:** Text input with 'Teszt Étterem'.
- Telefonszám:** Text input with '+36112345678'.
- Konyha típusa:** Text input with 'Magyar'.
- Rövid leírás:** Text area with 'Teszt leírás'.
- Borítókép:** File selection area with 'Fájl kiválasztása' and 'Nincs fájl kiválasztva'.
- Cím:** Text input with '1234 Budapest Teszt Elek Utca 1'.
- Nyitás:** Time selection input with '10:00'.
- Zárás:** Time selection input with '18:00'.
- Mentés:** Red button at the bottom right.

3.4.3.4. ÉTTEREM TULAJDONOS BEÁLLÍTÁSOK

Az étterem tulajdonos profil beállításokat a navigációssávban a „Beállítások” menüpontban találja.



Erre kattintva lehetősége van új jelszót beállítani.

Itt láthatja az email címét és megtudja változtatni jelenlegi jelszavát, ha helyesen adta meg a jelenlegi jelszavát és az új jelszó megfelel az előírásoknak akkor a „Mentés” gombra kattintva, elmentheti a módosításokat.

FIGYELEM: A profiladatok „email cím” része nem módosítható.

The 'Beállítások' modal form contains the following fields:

- Profiladatok:** Tab header.
- Email cím:** Text input with 'peldaetterem@pelda.hu'.
- Jelenlegi jelszó:** Password input with masked characters and a visibility toggle.
- Új jelszó:** Password input with masked characters and a visibility toggle.
- Új jelszó megerősítése:** Password input with masked characters and a visibility toggle.
- Fiók törlése:** Red button.
- Mentés:** Red button at the bottom.

3.4.3.5. ÉTTEREM TULAJDONOS FIÓK TÖRLÉSE

- Amennyiben arra a döntésre jutna, hogy nem szeretne az oldalon bármilyen formában szerepelni, akkor a tudja a étterem tulajdonos fiókját törölni. Ezt bejelentkezés után a beállítások menüpontban a „Fiók törlése” gombra kattintva tudja megtenni.
- Ekkor megjelenik egy értesítési fül ahol ha az „Igen” választási lehetőségre kattint, akkor a fiókja törlésre kerül.

FIGYELEM: Ez a művelet nem vonható vissza azután, hogy az „Igen” opciót választotta. Ekkor az étterem tulajdonosi fiók, a fiókhoz tartozó étterem, és az étteremben szereplő adatok törlődnek. A Fiók a megadott E-mail cím és jelszóval törlődik.

3.4.3.6. ÉTTEREM FELTÖLTÉSE ADATOKKAL

Az étterem menüjét feltöltheti változatosan.

- Feltölthet előételeket, főételeket, desszerteket, italokat.

Minden ételtípusnál van egy étel hozzáadása opció amire rákattintva előjön az étel feltöltéséhez szükséges ablak.

- Éttermet feltöltheti változatosan, ha nem kíván valamelyik lehetőséggel élni akkor arra a lehetőségre nem tölt fel ételt. Viszont ha nem tölt fel az egyik lehetőségre ételt az a lehetőség nem fog megjelenni (Példa: Ha nem fog feltölteni az előétel menüponthoz ételt akkor az étterménél a menü a „Főételek” menüponttal fog kezdődni és az „Előételek” rejtve marad.)

3.4.3.7. ÉTEL FELTÖLTÉSÉNEK LÉPÉSEI

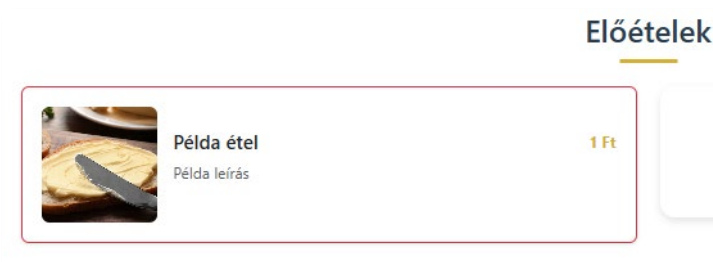
Ha rá kattint az „új előétel”, „új főétel”, „új desszert”, „új ital” menü pontra akkor megjelenik az étel feltöltéséhez szükséges adat beviteli ablak.

- Itt megadhatja az étel nevét, rövid leírását, árát, és az ételről egy képet ami nem haladja meg az 500 kilóbájtot.

FIGYELEM: Kitöltés után kattintson a „Mentés” gombra különben az étel feltöltés nem fog megtörténni.

FIGYELEM: Töltse ki az összes lehetőséget, hiányos vagy üres ételt nem tud hozzáadni az étterméhez.

- Amint rákattint a „Mentés” gombra a kiválasztott ételtípushoz megjelenik a feltöltött étel.
- Ekkor még az étel nem került publikálásra mivel azt az adminisztrátoroknak jóvá kell hagynia.
- A vörös keret is azt jelzi hogy az étel még nem lett jóváhagyva.
- Amint az adminisztrátorok jóváhagyták a publikálást akkor a vörös kereszt átvált zöldre.



3.4.3.8. FELTÖLTÖTT ÉTEL SZERKESZTÉSE

Ha a feltöltött étel árát, képét, leírását vagy éppen az elérhetőségét szeretné megváltoztatni arra is van lehetőség.

- Kattintson arra az ételre, amit szerkeszteni kíván
- Ekkor megfog jelenni egy ablak ahol tudja módosítani az étel adatait.
- Itt tudja módosítani az étel nevét, leírását, árát, megváltoztatni a képét és a státuszát. Amint módosította a kívánt változtatásokat kattintson a „Mentés” gombra különben a változtatás nem lesz érvényes.
- Státusz opció azt takarja, hogy elérhető-e az étteremben az az étel vagy sem. Hiszen lehet hogy egy szezonális ételt tölt fel ami nem az egész évben van a kínálatban így ha a „Nem elérhető” opciót válassza és úgy menti el az ételt akkor az étel nem fog megjelenni a kínálatai között hanem rejtve marad.

Szerkesztés

Étel neve
Példa étel

Leírás
Példa leírás

Ár
1

Borítókép
Fájl kiválasztása Nincs fájl kiválasztva

Státusz
☒ Elérhető
☐ Nem elérhető

Mentés

FIGYELEM: Ha módosítani kívánja a kínálatát, a megjelenő ablakba nem maradhat üres opció, legyen az az étel neve, leírása, ára, borítóképe. Egy esetben maradhat a borítókép „Nincs fájl kiválasztva” opción ha az étel státuszát módosítja, minden ellenkező esetben kikell tölteni.

3.4.3.9. ÉTTEREM TULAJDONOS VÉLEMÉNYEK SZEKCIÓ

Ha kíváncsi hogy milyen véleménnyel vannak az étterméről a vendégek azt megtudja tekinteni az oldal alján lévő „Vendégeink írták” opciónál.

Vendégeink írták **2**

Email	Dátum	Étel	Szolgáltatás	Tisztaság	Hangulat	Átlag	Vélemény
customer2@pelda.hu	2025. 03. 15. 02:22	3	4	2	5	3,500	A kiszolgálás kedves, de az étel nem volt ínycsiklandó. Az étterem tiszta és kellemes környezetet biztosít.
customer1@pelda.hu	2025. 03. 15. 02:22	5	4	5	3	4,250	Az étel fantasztikus volt, de a kiszolgálás lassú volt. A hely tiszta, de a hangulat lehetett volna jobb.

Exportálás

Itt egy részletes értékelést kaphat a vendégeitől, akik szántak időt értékelni az Ön által vezetett éttermet.

- Láthatja, hogy ki írt véleményt az étterméről
- Láthatja, hogy mikor írt véleményt
- Láthatja a 4 különböző értékelés csillagos eredményét
 - Étel értékelése 1-5 csillag
 - Szolgáltatás értékelése 1-5 csillag
 - Tisztaság értékelése 1-5 csillag
 - Hangulat értékelése 1-5 csillag
- Láthatja az átlagot, ami ebből a 4 értékelésből született és beleszámít az étterem csillagos értékelésébe.
- Láthatja a vendég véleményét, mit látott pozitívan és mi az, ami negatív hatást keltett az értékelésében.

FIGYELEM: Értékelést étterem tulajdonosként nem tud törölni, ha bármilyen olyan értékelést lát ami nem került moderálásra, vagy nem felel meg az irányelveknek akkor kérem keresse meg az oldal üzemeltetőjét emailben: info@gasztrotukor.hu.

3.4.3.10. ÉTTEREM VÉLEMÉNYEK MENTÉSE

Minden étterem tulajdonosnak van lehetősége a saját vendégeinek a véleményét kimenteni Microsoft Excel formátumban. Ez a lehetőség elősegíti az éttermek fejlődését hiszen hatnaponta vagy akár hetente összevethetik a kapott értékeléseket és ezzel fejlődési visszajelzésre tehetnek szert.

- Étterem kezelése menüpontban, a saját étterem oldalán a „vendégeink írták” opciónál találja az „exportálás” gombot.

Vendégeink írták 2							
Email	Dátum	Étel	Szolgáltatás	Tisztaság	Hangulat	Átlag	Vélemény
customer2@pelda.hu	2025. 03. 15. 02:22	3	4	2	5	3,500	A kiszolgálás kedves, de az étel nem volt ínycsiklandó. Az étterem tiszta és kellemes környezetet biztosít.
customer1@pelda.hu	2025. 03. 15. 02:22	5	4	5	3	4,250	Az étel fantasztikus volt, de a kiszolgálás lassú volt. A hely tiszta, de a hangulat lehetett volna jobb.

Exportálás

- Az „Exportálás” gombra kattintva letöltheti a „etteremneve_ertekelesek.xlsx” fájlt amiben láthatja Excel formátumban az étteremről a vendégvéleményeket.

3.4.4. ADMINISZTRÁTOR FUNKCIÓK

3.4.4.1. ADMINISZTRÁTOR BEJELENTKEZÉS

A programnak három adminisztrátora van. Az adminisztrátori fiókok nem regisztráció alapján kezelhetők, hanem manuálisan lettek a jogosultságok megadva.

Adminisztrátori bejelentkezés email címek és hozzá tartozó jelszó:

- csordas.patrik.cristian@ganziskola.hu
- szoke.daniel@ganziskola.hu
- marton.bence.david@ganziskola.hu

A jelszó mind a három adminisztrátori fiókhoz megegyező: **admin**

3.4.4.2. ADMINISZTRÁTOR PROFIL BEÁLLÍTÁSOK



Válassza ki a „beállítások” menüpontot a navigációs sávban.

Itt ugyan úgy, mint az étterem tulajdonosnál és a vendég felhasználónál a jelenlegi jelszó megváltoztatható egy új jelszóra.

Viszont a „Fiók törlése” opció nem található mivel ezek a fiókok nem törölhetők.

A profiladatokon kívül az adminisztrátorok tudják a felhasználókat is kezelni a „Felhasználók kezelése” opciónál.

3.4.4.3. ADMINISZTRÁTOR FELHASZNÁLÓK KEZELÉSE

Ennél az opciónál az adminisztrátor rálátást kap az oldalon jelenleg regisztrált felhasználókról, és azoknak a szerepkörükről.

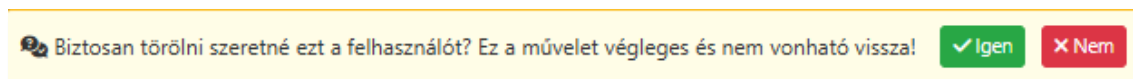
- Láthatja a regisztrált felhasználók email címét.
- Láthatja a regisztrált felhasználók szerepkörét: admin, customer, owner
- Műveletet hajthat végre a felhasználókon. Ez alól csak az admin profilok kivételek hisz azokat nem lehet törölni.

Beállítások		
Felhasználók kezelése		
Email	Szerepkör	Műveletek
csordas.patrik.cristian@ganziskola.hu	admin	Adminisztrátor nem törölhető
szoke.daniel@ganziskola.hu	admin	Adminisztrátor nem törölhető
marton.bence.david@ganziskola.hu	admin	Adminisztrátor nem törölhető
customer1@pelda.hu	customer	Törés
customer2@pelda.hu	customer	Törés
info@labellaitalia.hu	owner	Törés
info@sushimester.hu	owner	Törés
info@eltacoloco.hu	owner	Törés
info@leblotfrancs.hu	owner	Törés
info@bajordvar.hu	owner	Törés
info@tasteofindia.hu	owner	Törés
info@casaportuguesa.hu	owner	Törés
info@olivafataverna.hu	owner	Törés
info@koreanbbqhouse.hu	owner	Törés
info@thalkert.hu	owner	Törés
info@huszarcasarda.hu	owner	Törés
info@americandiner.hu	owner	Törés
info@scandinaviakitchen.hu	owner	Törés
info@cedrusgrill.hu	owner	Törés
info@russianvodkahouse.hu	owner	Törés
teszt@pelda.hu	customer	Törés
peldaetterem@pelda.hu	owner	Törés

3.4.4.4. ADMINISZTRÁTOR FELHASZNÁLÓK TÖRLÉSE

Amennyiben a regisztrált felhasználó nem felel meg az oldal irányelveinek, akkor az adminisztrátorok jogosultak a fiók törlésére, legyen az vendég fiók, vagy akár étterem tulajdonos fiók.

Ha ilyenre kerül a sor akkor az adminisztrátor rákattint a „Törés” gombra és megjelenik egy értesítési panel az oldal jobb oldalán.



Ha az „Igen” opciót választja az adminisztrátor akkor a kiválasztott fiók törlésre kerül.

FIGYELEM: Ez a művelet nem vonható vissza azután, hogy az „Igen” opciót választotta. Ekkor az adminisztrátor által elbírált döntés következtében, a fiókhoz tartozó profil, a fiókhoz tartozó étterem vélemények, vagy a fiókhoz tartozó étterem törlésre kerül. A Fiók a megadott E-mail cím és jelszóval törlődik.

3.4.4.5. ADMINISZTRÁTORI FELÜLET KEZELÉSE

Ha az éttermeket akarja kezelni akkor válassza a navigációs sávban az „Adminisztrátori felület” opciót.

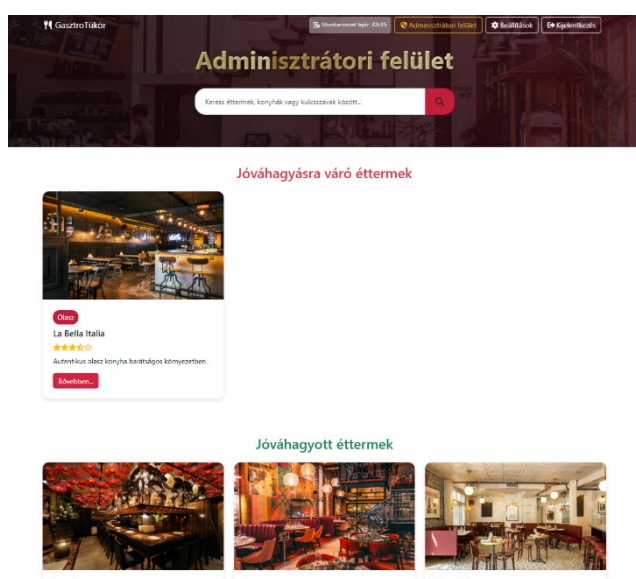


Ekkor átkerül az Adminisztrátori nézetre.

Itt láthat két féle étterem típust.

- A „jóváhagyásra váró éttermek”
- A „jóváhagyott éttermek”

A jóváhagyásra váró étterem az olyan étterem, ami nemrég regisztrált az oldalra, de még az adminisztrátorok nem hagyták jóvá, hogy publikus étterem



legyen az oldalon. De akár olyan is lehet, hogy az étterem inaktív vagy változtatások várhatóak az étteremmel kapcsolatba és az étterem tulajdonos kérte, hogy ne legyen publikus az étterem.

A jóváhagyott éttermek viszont olyan éttermek akik regisztráció után adminisztrátori döntés végett publikált éttermek lettek és megtalálhatóak az oldalon a vendégek számára.

3.4.4.6. ADMINISZTRÁTORI FELÜLET ÉTTERMEK ADATAINAK KEZELÉSE

Ha az adminisztrátor változtatásoka szeretne elvégezni egy étterem adataival kapcsolatban akkor azt megteheti.

- Kattintson a kiválasztott étterem kártyájára
- Ekkor megjelenik a kezelési felület, ahol az étterem adatait tudja módosítani.

3.4.4.7. ÉTTEREM SZERKESZTÉSE ÉS ANNAK LÉPÉSEI

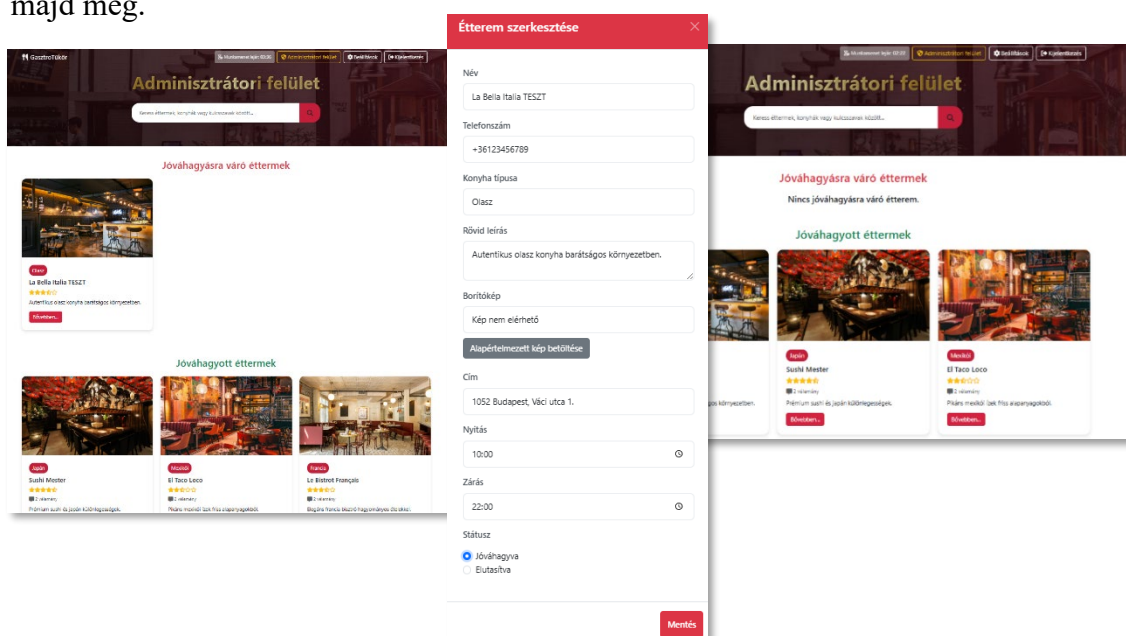
Változtatásokat tud végrehajtani ezek a pontok alapján

- Név
- Telefonszám
- Konyha típusa
- Rövid leírás
- Borítókép ha nem megfelelő akkor egy alapértelmezett „kép nem elérhető” cserélheti.
- Cím
- Nyitás
- Zárás
- Státusz

FIGYELEM: Minden változtatás csak akkor érvényes, ha nincs üresen hagyott beviteli mező, ez alól csak a borítókép kivétel.

Státusz változtatásánál amennyiben az étterem jóváhagyva opció van kijelölve és úgy kattint a „mentés” gombra akkor a vendégek előtt megjelenik az

étterem és az adminisztrátori felületen átkerül a „Jóváhagyott éttermek” közé. Viszont, ha a státusz az „Elutasítva” opción van és úgy kerül mentésre akkor az étterem rejtve lesz a vendégek elől és az adminisztrátori felületen a „Jóváhagyásra váró éttermeknél” jelenik majd meg.

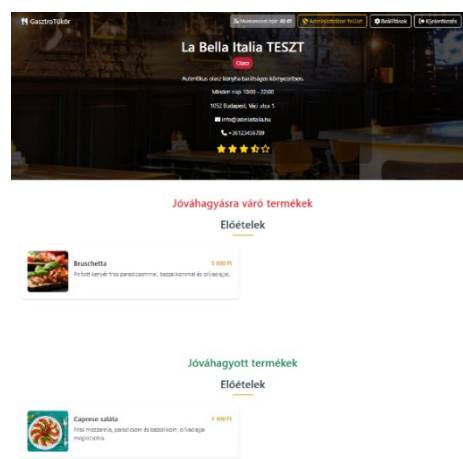


3.4.4.8. ADMINISZTRÁTORI FELÜLET ÉTTEREM MENÜJÉNEK KEZELÉSE

Ha az adminisztrátor felülvizsgál egy éttermet és az egyik étel vagy kínálat nem felel meg vagy esetleg hiba lenne a feltöltött kínálatban, akkor azt tudja szerkeszteni.

Szerkesztés lépései

- Válassza ki az adott éttermet adminisztrátori nézetben
- Az étteremnél kattintson a „Bővebben” lehetőségre.
- Ekkor az oldal átirányítja önt az étterem adminisztrátori felületére.
- Itt láthatja a jóváhagyott és a jóváhagyásra váró ételeket.
- Itt a szerkeszteni kívánt ételre rákattintva megjelenítheti a szerkesztési ablakot.



3.4.4.9. A SZERKESZTÉS LÉPÉSEINEK FOLYAMATA

- Szerkesztheti az étel nevét
- Szerkesztheti az étel leírását
- Szerkesztheti az étel árát
- Ha nem megfelelő a kép akkor egy „kép nem elérhető” képre cserélheti.
- Szerkesztheti a státuszát.

Szerkesztés

Étel neve

Bruschetta

Leírás

Pirított kenyér friss paradicsommal, bazsalikommal és olí

Ár

3000

Borítókép

Kép nem elérhető

Alapértelmezett kép betöltése

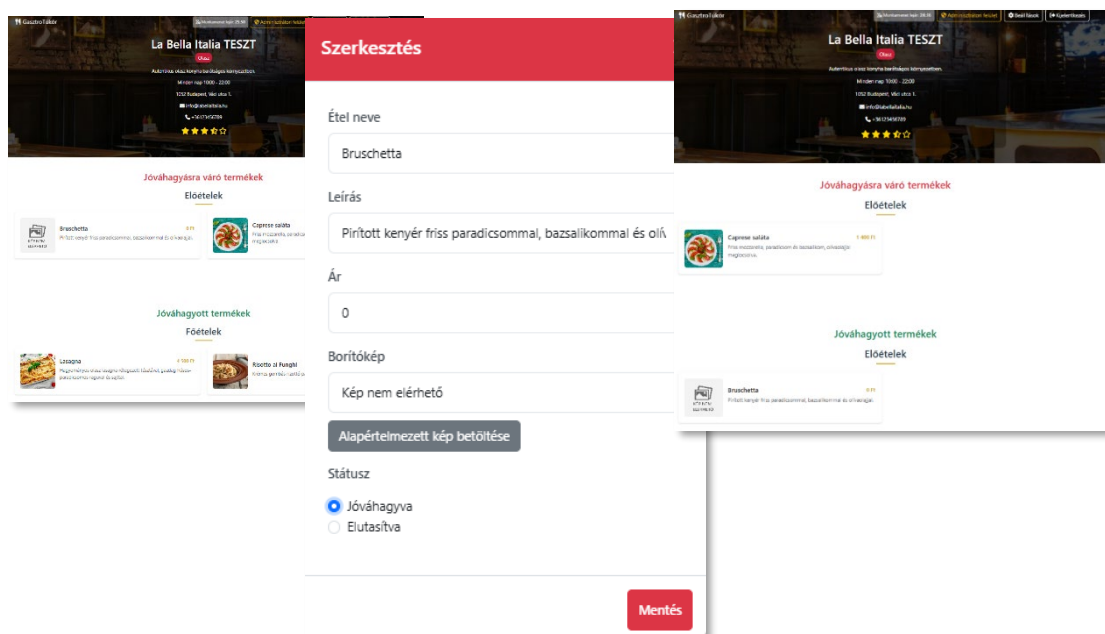
Státusz

☐ Jóváhagyva
 ☒ Elutasítva

Mentés

A státusznál ha a „Jóváhagyva” van kijelölve és úgy menti el a szerkesztést akkor az adminisztrációs felületen a jóváhagyott termékekhez kerül a termék.

Viszont ha a státusznál az „Elutasítva” opció van kijelölve akkor a termék a „Jóváhagyásra váró termékek”-hez kerül és rejtve marad a vendégek számára. Lásd az ábrán.



3.4.4.10. ADMINISZTRÁTORI FELÜLET VENDÉGVÉLEMÉNY KEZELÉSE

Adminisztrátorok jogosultak felülvizsgálni minden étterem vélemény szekcióját és moderálni azt. Adminisztrátori felületben kiválasztjuk a kívánt éttermet, rákattint az étteremnél a „Bővebben” opcióra, és legörget a „Vendégeink írták” opcióhoz.

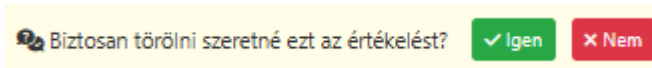
Vendégeink írták									
Email	Dátum	Étel	Szolgáltatás	Tisztaság	Hangulat	Átlag	Vélemény	Műveletek	
customer2@pelda.hu	2025. 03. 15. 02:22	3	4	2	5	3,500	A kiszolgálás kedves, de az étel nem volt ínycsiklandó. Az étterem tiszta és kellemes környezetet biztosít.	Törölés	
customer1@pelda.hu	2025. 03. 15. 02:22	5	4	5	3	4,250	Az étel fantasztikus volt, de a kiszolgálás lassú volt. A hely tiszta, de a hangulat lehetett volna jobb.	Törölés	

Itt láthatóak a vendégek által írt vélemények. Ha az adminisztrátor úgy dönt, hogy nem megfelelő a vélemény formátuma vagy ütközik az oldal irányelveivel akkor a véleményt törölheti a „Törölés” gombbal, ha esetleg ez a szankció nem elég akkor az adminisztrátor látja, hogy milyen e-mail címről írták az adott véleményt, és az e-mail címhez tartozó felhasználói fiókot is törölheti.

Amint az adminisztrátor rákattint a „Törlés” gombra akkor megjelenik egy értesítési panel.

Ha az „Igen” opciót választja az adminisztrátor akkor a vendég

vélemény törlésre kerül és az értékelés nem fog beleszámítani az étterem csillagos értékeléseibe.



3.4.5. AZ OLDALT SEGÍTŐ TÁJÉKOZÓDÁSI OPCÍÓK

3.4.5.1. GASZTROTÜKÖR GOMB ÉS JELENTŐSÉGE



A GasztróTükör gomb jelentős és a navigációs sávban egy fix pozíciót elfoglaló tájékoztató gomb. Bármilyen módon vagyunk legyen az vendég felhasználói felület, étterem tulajdonos felület, vagy admin felület, akár egy külsős megfigyelő, aki nem regisztrál és nem is jelentkezik be az oldalon ez a gomb mindig minden körülmények között a Fő oldalra vezet vissza.

3.4.5.2. MUNKAMENET FIGYELŐ

Ez az opció a navigációs sávban jelenik meg bejelentkezés után. Nem számít, hogy vendégként, étteremtulajdonosként, vagy akár adminisztrátorként jelentkezünk be az oldalba ez a számláló bejelentkezéstől kezdve 45 percen át visszaszámol. Így ellenőrizhetjük mennyi időt töltöttünk az oldalon.

Vendég fiók



Étterem tulajdonos fiók



Adminisztrátori fiók



3.4.5.3. MUNKAMENET LEJÁRT ÉS ANNAK KÖVETKEZMÉNYE

Amint a visszaszámláló 45 percet visszaszámolt az oldalon lévő felhasználót szerepkörtől függetlenül kijelentkezteti az oldalból. Ekkor egy ilyen oldal fogadja a felhasználót. Ez az opció arra szolgál hogyha esetleg bejelentkezve marad a felhasználó viszont elfelejt kijelentkezni akkor 45 perc után biztonságosan kijelentkezteti és így nem lesz leterhelt az oldal, valamilyen szinten a felhasználók adatait is védjük ezzel az opcióval.



3.4.6. KAPCSOLATTARTÁS

A kapcsolattartási lehetőségeket minden oldal alján, a láblécben megtalálod, így bármelyik részoldalról könnyedén elérhetsz bennünket.

Ha kérdésed merül fel, technikai problémába ütköztél, vagy csak szeretnél velünk megosztani egy ötletet, visszajelzést, bátran vedd fel velünk a kapcsolatot az oldal láblécében található E-mail címen: info@gasztrotukor.hu.

Célunk, hogy minden látogatónk és felhasználónk elégedett legyen, ezért minden üzenetet igyekszünk a lehető leghamarabb megválaszolni.



4. ÖSSZEFOGLALÁS

4.1. KÖSZÖNETNYILVÁNÍTÁS

Ezúton szeretnénk kifejezni őszinte hálánkat és köszönetünket Szalainé Török Edit és Méhes József tanárainknak, akik szakmai tudásukkal, támogatásukkal és bátorításukkal végig segítették vizsgaremekünk elkészítését.

Szalainé Török Edit tanárnő türelme, útmutatásai és értékes tanácsai nélkül ez a munka nem jöhetett volna létre. Köszönjük, hogy mindig készséggel állt rendelkezésünkre, és iránymutatásaival hozzájárult fejlődésünkhöz.

Méhes József tanár úr szakmai felkészültsége és inspiráló hozzáállása hatalmas segítséget jelentett munkánk során. Köszönjük a biztatást, az értékes visszajelzéseket és azt a rengeteg tudást, amelyet átadott nekünk.

Hálásak vagyunk, hogy ilyen elhivatott és támogató tanároktól tanulhattunk, és hogy a segítségükkel sikeresen megvalósíthattuk vizsgaremekünket.

4.2. ELKÉSZÜLT PROJEKTÜNK ÉRTÉKELÉSE

Rendkívül büszkéek vagyunk a csapatunk által elért eredményekre! Sikerült egy kiváló alapot létrehoznunk a GasztroTükör platformhoz, amely szilárd alapot biztosít a jövőbeli fejlesztésekhez. Weboldalunk teljesíti az összes kitűzött célunkat, és meggyőződésünk, hogy ez a munka hosszú távon is jelentős értéket képvisel majd.

A fejlesztési folyamat során számtalan technikai ismeretet szereztünk a Blazor WebAssembly keretrendszer használatával kapcsolatban, és bár szembesültünk kihívásokkal, ezek mind elősegítették szakmai fejlődésünket. Célunk egy könnyen használható, átlátható étterem-értékelő platform létrehozása volt, ahol a felhasználók egyszerűen találhatnak és oszthatnak meg véleményeket éttermekről.

Természetesen vannak még területek, amelyeken fejlődnünk kell. A felhasználói felület néhány eleme további finomításra szorul, és néhány tervezett funkció még nem érték el a tökéletes állapotot. Ezek a kihívások azonban további fejlődési lehetőségeket kínálnak számunkra.

5. IRODALOMJEGYZÉK

- ❖ <https://getbootstrap.com/>
- ❖ <https://fontawesome.com/icons/>
- ❖ <https://www.flaticon.com/>
- ❖ <https://stackoverflow.com/>
- ❖ <https://www.geeksforgeeks.org/>
- ❖ <https://developer.mozilla.org/>
- ❖ <https://w3schools.com/>
- ❖ <https://learn.microsoft.com/en-us/azure/app-service/>
- ❖ <https://unsplash.com/>
- ❖ <https://pinterest.com/>